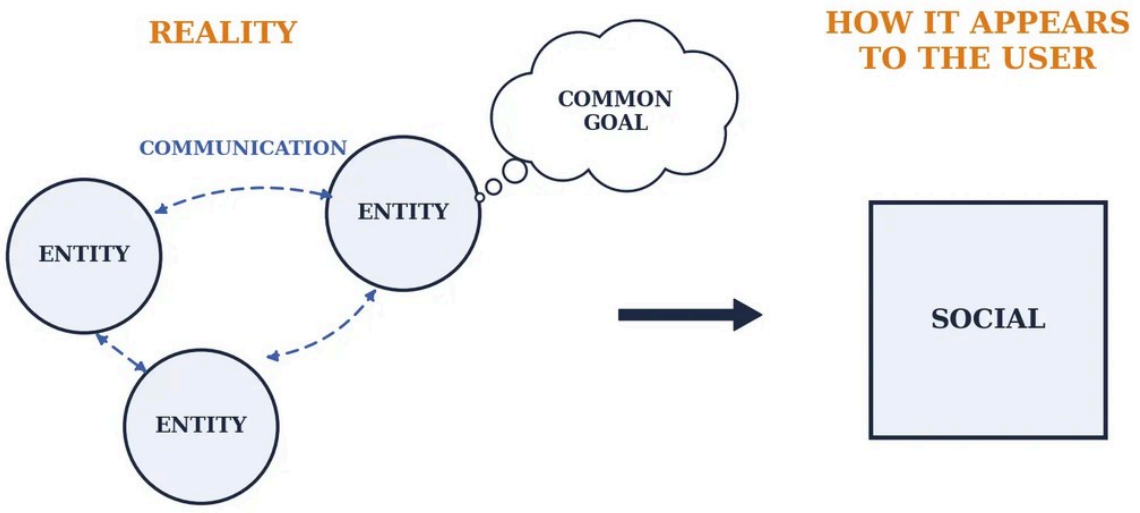
Dependable distributed system 1

Distributed system

> **DISTRIBUTED SYSTEM**
>
> A distributed system is a piece of software that ensures that a collection of indipendant computers appears to its users as a single coherent system.

Usually a distributed system needs to complete a task, or goal, for which more than half ($\frac{2}{3}$) of the entities need to agree.



Distributed system are useful:

- To increase performance, coping with the increasing demand of users in both processing power and data storage, and reducing latency;
- To build dependable services, coping with failures.

A distributed system is characterized by:

1. **A collection of autonomous computing elements.**
    1. *Challenges*: the absence of a global clock, group membership (open vs closed) and the overlay networks (structured vs unstructured)
2. **A single coherent system**, where *coherent* means that it behaves according to the expectations of its users.
    1. *Challenge*: failures.

Dependability

> **DEPENDABILITY**
>
> Dependability is the ability of a system to deliver a service that can justifiably be trusted, i.e. the ability to avoid service failures that are more frequent than is acceptable.

The characteristics of dependability are:

1. **Availability:** ready to be used;
2. **Reliability:** continuity of correct service;
3. **Safety:** no catastrophic consequences,
4. **Integrity:** no system alteration;
5. **Maintainability:** possibility to modify and repair;
6. **Robustness:** keep performing well even if the environment changes.

$$\text{Fault} \xrightarrow{\text{causes}} \text{error} \xrightarrow{\text{generates}} \text{failure}$$

Dependable distributed systems

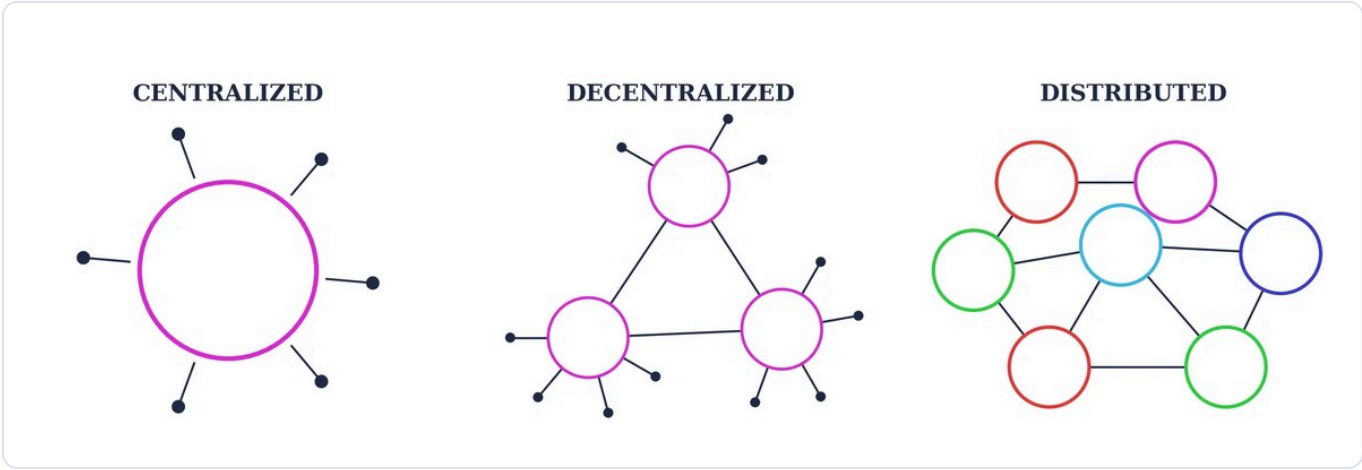
The characteristics and challenges of these systems are:

1. **Heterogeneity** [not defined];
2. **Openness**: the capability of a system to be extended and re-implemented;
3. **Security**: confidentiality, integrity and availability;
4. **Scalability**: the system keeps running with adequate performance even if the number of resources and/or users grows by orders of magnitude;
5. **Fault tolerance**;
6. **Concurrency**: multiple access to shared resources, coordination and synchronisation;
7. **Transparency**.

Blockchain

A dependable distributed system can be:

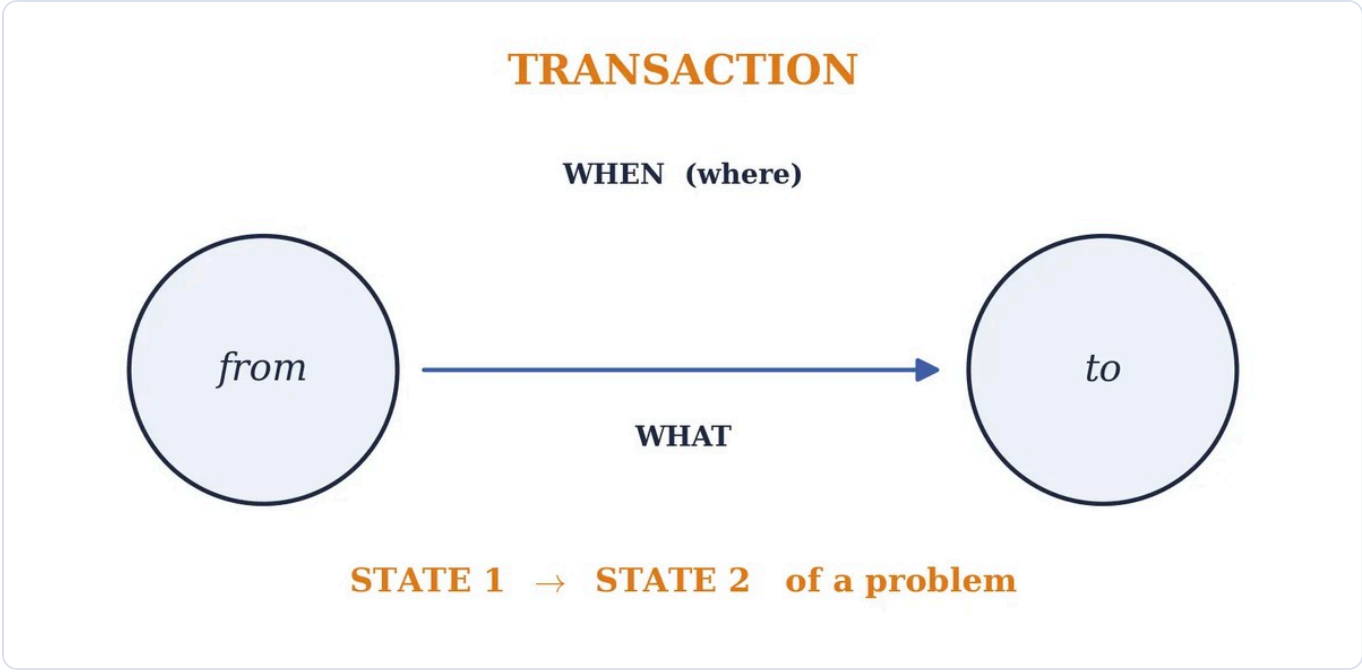
- **Centralized**: there is a single central node on which all the other peripheral nodes depend. If the central node fails, the entire system collapses;
- **Decentralized:** There are multiple intermediate-level nodes, each of which manages its own subset of peripheral nodes. Control is divided among multiple entities, reducing dependance on a single central point;
- **Distributed:** All nodes are connected to each other in a peer-to-peer network (or mesh), where there is no central authority or single coordinator and each node collaborates to achieve a common goal. This is the key model for ensuring high fault tolerance and reliability in modern distributed systems.



CONSENSUS

A **consensus** algorithm allows a group of distributed nodes to agree on a common value or state, even when some nodes fails or behave maliciously. It must satisfy:

- **Agreement**: all honest nodes agree on the same value;
- **Validity**: the decided value satisfies the protocol rules;
- **Termination**: every honest node reaches a decision.



A transaction is made of several phases; it is atomic when all its phases are executed as a single unit.

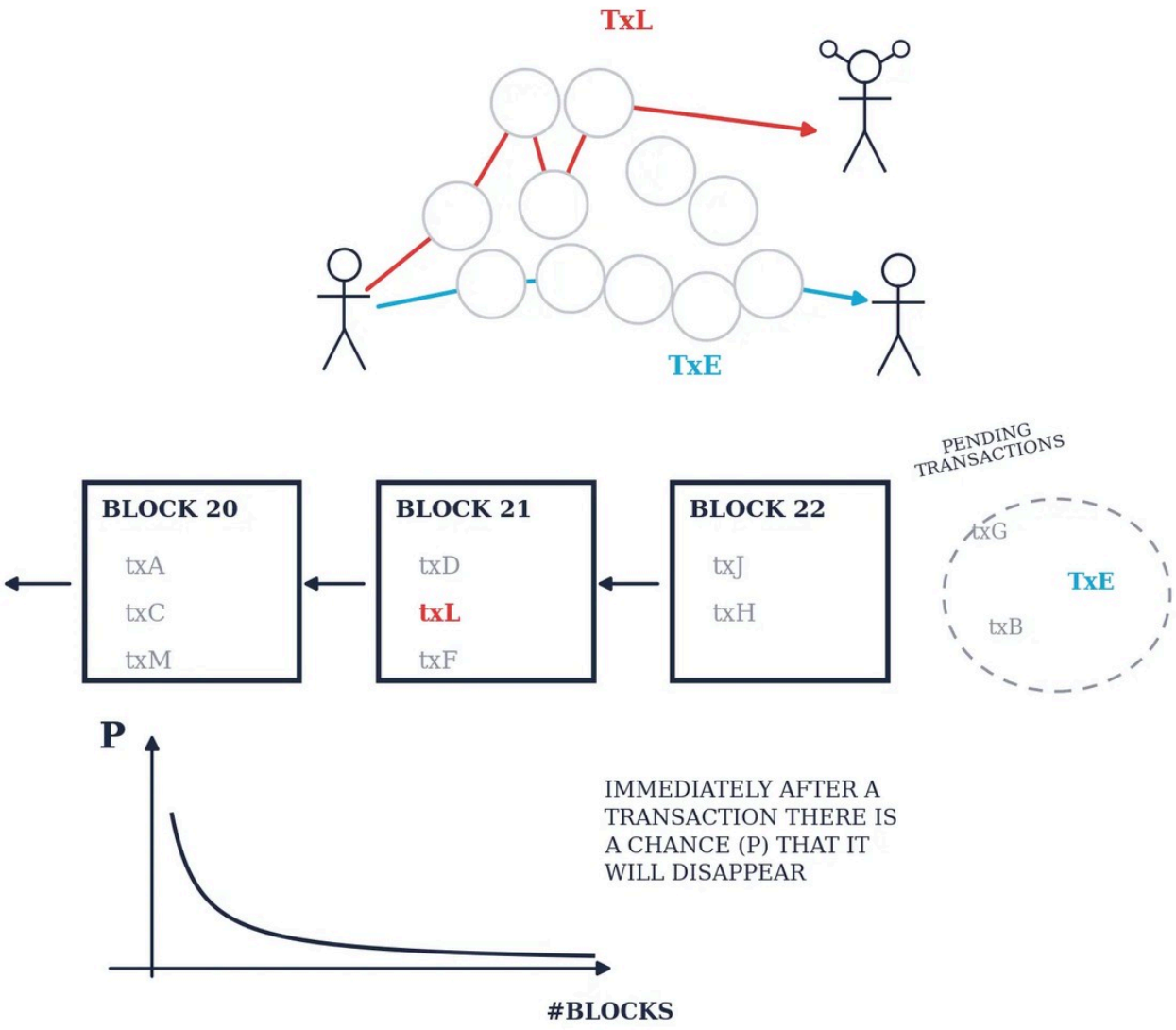
LEDGER OF TRANSACTIONS

The **ledger of transactions** is the record of all the transactions made. It is easier to keep updated and synchronised in a centralised architecture.

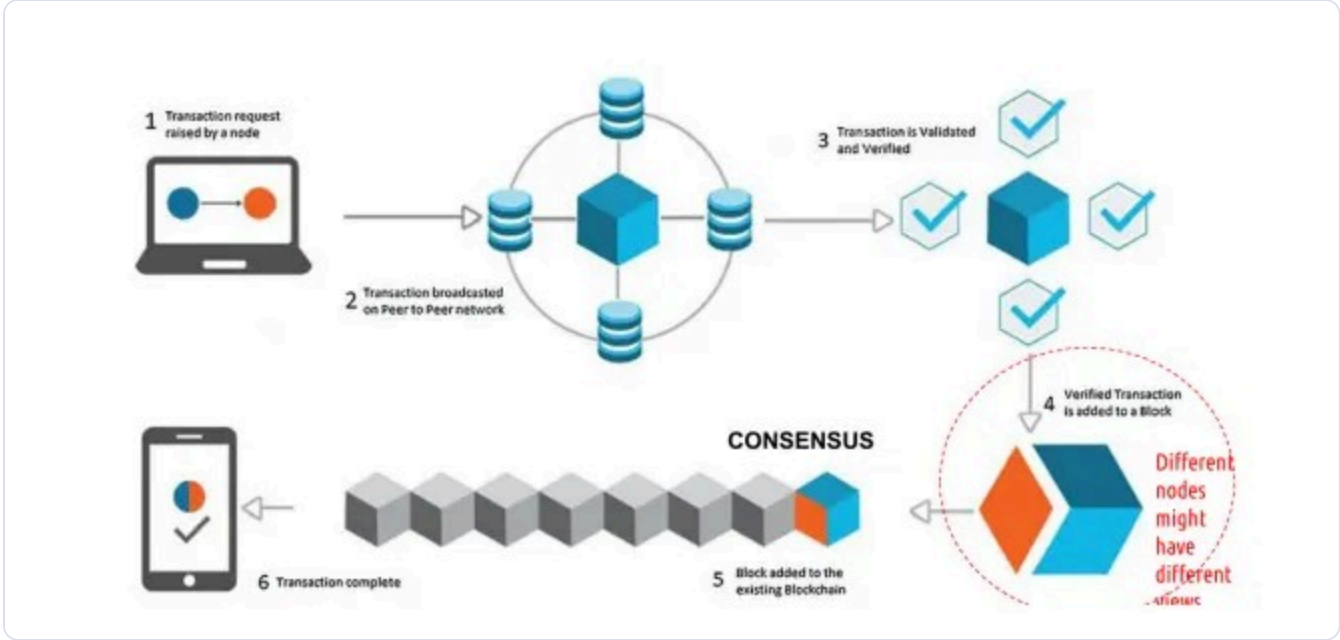
BLOCKCHAIN

A **blockchain** is a distributed ledger that records transactions in an immutable, verifiable and permanent way.

If two irreconcilable transactions happen, only the first one recorded is considered legitimate.


TxL
TxE
PENDING TRANSACTIONS
BLOCK 20
txA
txC
txM
BLOCK 21
txD
txL
txF
BLOCK 22
txJ
txH
txG
TxE
txB
P
IMMEDIATELY AFTER A TRANSACTION THERE IS A CHANCE (P) THAT IT WILL DISAPPEAR
#BLOCKS

How a transaction gets into a blockchain



Modelling distributed system

A distributed system pursues its goal by running a distributed algorithm, i.e. a piece of software that takes care of communication, coordination and resource sharing. The road to a distributed abstraction has three steps:

1. **System model**: describe the relevant elements in an abstract way, identify their intrinsic properties and characterize their interactions;
2. **Distributed abstraction**: design a protocol that captures a recurring interaction pattern of distributed applications;
3. **Implementation and deployment of the algorithm**, possibly as part of the middleware. This step is system dependent (language, architectural pattern) and belongs to Software Engineering and Lab.

Abstraction are important because they capture properties common to a large and significant range of systems, help to distinguish the fundamental from the accessory, and prevent designers from reinventing the same solution for slight variants of the same problem. An *architectural style* (components with well-defined required and provided interfaces, their connections, the data exchanged, their configuration) is treated in Software Engineering. Here components are abstracted by their specification and interfaces, and we focus on distributed abstractions and their algorithmic aspects.

Composition model

Protocols are written in pseudo.code that reflects a reactive computing model: components of the same process communicate by exchanging events and an algorithm is a set of event handlers that react to incoming events and possibly trigger new ones.

$$
\begin{aligned}
&\textbf{upon event } \langle c_1, \text{Event}_1 \mid \text{att}_1, \text{att}_2, \ldots \rangle \textbf{ do}\\
&\quad \text{do something;} \quad //\text{send-some-event}\\
&\quad \textbf{trigger } \langle c_2, \text{Event}_2 \mid \text{att}_1, \text{att}_2, \ldots \rangle;\\
&\textbf{upon condition do} \quad //\text{an-internal-event}\\
&\quad \text{do something;}
\end{aligned}
$$

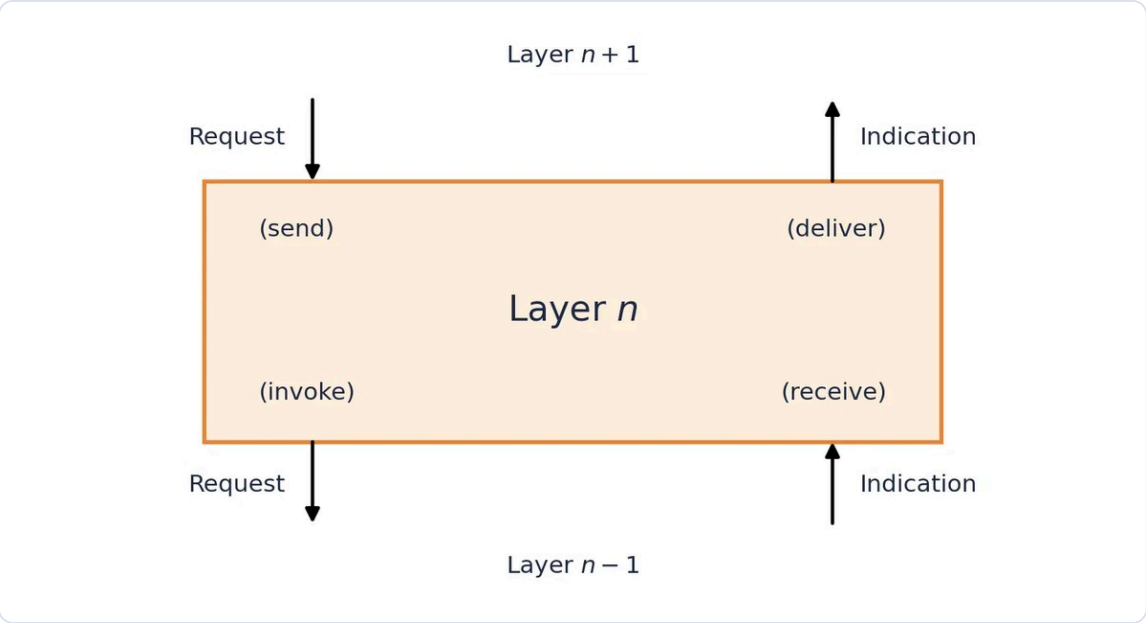
Event-Driven Algorithm Components

- `upon event ⟨ component, EventName | attributes ⟩`: This acts as an event handler. It triggers the execution of the subsequent code block whenever a specific event is received or intercepted by the component.
- `component` **(e.g.,** c_1, rbc**)**: Identifies the specific module, layer, or instance (such as a communication abstraction or broadcast instance) responsible for handling or generating the event.
- `EventName` **(e.g., Broadcast, Deliver, Send)**: Specifies the type of action or message event being processed (e.g., initiating a broadcast or delivering a payload).
- `attributes` **(e.g.,** m, p, att_1**)**: Represents the parameters or payload carried by the event, such as messages, process identifiers, or state variables.
- `trigger ⟨ ... ⟩`: Used to emit or dispatch a new event toward another component, layer, or communication channel.

- `upon condition`: Evaluates an internal boolean guard or local system state condition, triggering execution immediately when the condition becomes true.

Events are of three kinds, and form the programming interface between layers:

- **Request**: used by a component c1 to request a service from c2 (e.g. disseminate a message in a group);
- **Confirmation**: used by a component to confirm the completion of a request;
- **Indication**: used by a component c1 to deliver information to c2 (e.g. the notification of a message delivery).



Example: job handler and layering

Component: JobHandler, instance jh.
Request:
$\langle jh, \text{Submit} \mid \text{job} \rangle$: requests a job to be processed.
Indication:
$\langle jh, \text{Confirm} \mid \text{job} \rangle$: confirms that the job has been (or will be) processed.
Property JH1 (Guaranteed response):
every submitted job is eventually confirmed.

Two implementations satisfy this specification. In the *synchronous* one the job is processed before being confirmed; in the *asynchronous* one it is only stored in a buffer, which is why the confirmation says "has been or will be" processed.

ALGORITHM: SYNCHRONOUS JOB HANDLER

$$
\begin{array}{l}
\textbf{Algorithm: } \text{synchronous job handler} \\
\textbf{Upon event } \langle jh, \text{Submit} \mid \text{job} \rangle \textbf{ do} \\
\quad \text{process}(\text{job}); \\
\quad \text{trigger } \langle jh, \text{Confirm} \mid \text{job} \rangle;
\end{array}
$$

- **Event Trigger (`Submit`):** When a job submission request arrives via $\langle jh, \text{Submit} \mid \text{job} \rangle$, the handler handles it straight away.
- **Execution Flow:**
  - It executes the task synchronously: `process(job);`
  - Once processing is fully completed, it triggers the confirmation event: `trigger` $\langle jh, \text{Confirm} \mid \text{job} \rangle$;
- **Characteristics:** It is **blocking**. The system processes the request sequentially and remains tied up until the job is completely executed before sending a confirmation back.

ALGORITHM: ASYNCHRONOUS JOB HANDLER

$$
\begin{array}{l}
\textbf{Algorithm: } \text{asynchronous job handler} \\
\textbf{Upon event } \langle jh, \text{Init} \rangle \textbf{ do} \\
\quad \text{buffer} := \emptyset; \\
\textbf{Upon event } \langle jh, \text{Submit} \mid \text{job} \rangle \textbf{ do} \\
\quad \text{buffer} := \text{buffer} \cup \{\text{job}\}; \\
\quad \text{trigger } \langle jh, \text{Confirm} \mid \text{job} \rangle; \\
\textbf{Upon } \text{buffer} \neq \emptyset \textbf{ do} \\
\quad \text{job} := \text{selectjob}(\text{buffer}); \\
\quad \text{process}(\text{job}); \\
\quad \text{buffer} := \text{buffer} \setminus \{\text{job}\};
\end{array}
$$

- **Initialization (`Init`):**
  - Upon initialization, it creates an empty data structure to store incoming tasks: `buffer := ∅;`
- **Receiving Requests (`Submit`):**
  - When a job arrives, instead of processing it right away, it appends the job to the buffer (`buffer := buffer ∪ {job}`) and immediately triggers a confirmation ($\langle jh, \text{Confirm} \mid \text{job} \rangle$). This decouples request acceptance from actual execution.
- **Background Processing (`upon buffer ≠ ∅`):**
  - A continuous internal guard monitors the buffer. Whenever it is not empty, the system selects a job (`selectjob(buffer)`), processes it, and removes it from the queue (`buffer := buffer \ {job}`).
- **Characteristics:** It is **non-blocking**. The system accepts and confirms requests instantly while queuing them up for background processing.

LAYERING

Module:
Name: TransformationHandler, **instance** th.
Events:
Request: $\langle th, \text{Submit} \mid \text{job} \rangle$: Submits a job for transformation and for processing.
Indication: $\langle th, \text{Confirm} \mid \text{job} \rangle$: Confirms that the given job has been (or will be) transformed ar
Indication: $\langle th, \text{Error} \mid \text{job} \rangle$: Indicates that the transformation of the given job failed.
Properties:
TH1: Guaranteed response: Every submitted job is eventually confirmed or its transformation fa
TH2: Soundness: A submitted job whose transformation fails is not processed.

ALGORITHM: JOB-TRANSFORMATION BY BUFFERING

Algorithm: Job-Transformation by Buffering
Implements: TransformationHandler, instance th.
Uses: JobHandler, instance jh.

Upon event $\langle th, \text{Init} \rangle$ **do**
  $\text{top} := 1; \quad \text{bottom} := 1;$
  $\text{handling} := \text{FALSE}; \quad \text{buffer} := [\perp]^M;$

Upon event $\langle th, \text{Submit} \mid \text{job} \rangle$ **do**
  if $\text{bottom} + M = \text{top}$ **then**
    trigger $\langle th, \text{Error} \mid \text{job} \rangle$;
  else
    $\text{buffer}[\text{top} \bmod M + 1] := \text{job};$
    $\text{top} := \text{top} + 1;$
    trigger $\langle th, \text{Confirm} \mid \text{job} \rangle$;

Upon $\text{bottom} < \text{top} \wedge \text{handling} = \text{FALSE}$ **do**
  $\text{job} := \text{buffer}[\text{bottom} \bmod M + 1];$
  $\text{bottom} := \text{bottom} + 1;$
  $\text{handling} := \text{TRUE};$
  trigger $\langle jh, \text{Submit} \mid \text{job} \rangle$;

Upon event $\langle th, \text{Confirm} \mid \text{job} \rangle$ **do**
  $\text{handling} := \text{FALSE};$

This algorithm maintains a buffer of size M to temporarily store incoming jobs and send them one at a time to the underlying JobHandler (jh):

- **Initialization (`Init`)**
  - The `top` and `bottom` indices are set to 1, pointing to the insertion and extraction positions, respectively.

- `handling := FALSE` indicates that the system is currently not processing any jobs.
- `buffer := [\bot]^M` initializes an empty circular buffer of size M filled with null elements ($\perp$).

- **Receiving the Request (`Submit`)**
  - When a new job arrives via the event $\langle th, \text{Submit} \mid \text{job} \rangle$, the algorithm checks whether the buffer is full by evaluating the overflow condition $\text{bottom} + M = \text{top}$.
    - **If the buffer is full:** It rejects the request by triggering the error event $\langle th, \text{Error} \mid \text{job} \rangle$.
    - **If the buffer is not full:** It stores the job at the correct position using the modulo operator ($\text{top} \bmod M + 1$), increments the `top` index, and immediately confirms the submission by triggering $\langle th, \text{Confirm} \mid \text{job} \rangle$.
- **Asynchronous Sending to the Lower Level ($\text{bottom} < \text{top} \wedge \text{handling} = \text{FALSE}$)**
  - This internal guard continuously monitors whether there are pending jobs in the buffer ($\text{bottom} < \text{top}$) and whether the system is free ($\text{handling} = \text{FALSE}$).
  - **When both conditions are verified:** It retrieves the next job, increments the `bottom` index, sets `handling := TRUE` to prevent simultaneous transmissions, and forwards the job to the lower-level handler via `trigger` $\langle jh, \text{Submit} \mid \text{job} \rangle$.
- **Handling Confirmation from the Lower Level (`jh, Confirm`)**
  - Upon receiving the completion signal $\langle jh, \text{Confirm} \mid \text{job} \rangle$ from the lower-level handler (jh), the `handling` flag is reset to `FALSE`, allowing the system to process the next job from the buffer.

Processes, messages and algorithms

Π denotes the **set of processes**. Unless stated otherwise it is static and every process knows the identities of all the others. Sometimes a function $rank : \Pi \to \{1, \ldots, N\}$ gives each process a unique index; in the code, **`self`** is the name of the process executing it. **Processes** communicate by *exchanging messages over links*. Messages are uniquely identified (e.g. by a sequence number or local clock together with the process identifier), hence all messages ever exchanged by an algorithm are unique.

DISTRIBUTED ALGORITHM

A **distributed algorithm** is a distributed collection of automata, one per process. The automaton regulates how the process executes its computation steps, i.e. how it reacts to a message; every process runs the same automaton. An execution is a sequence of steps executed by the processes.

SAFETY AND LIVENESS

A **safety** property states that the algorithm does not do anything wrong: it can be violated at some time t and never be satisfied again after t (equivalently, if it is violated in an execution E, there is a partial execution E' of E such that it is violated in every extension of E'). A **liveness** property ensures that eventually something good happens: for any time t there is some hope that it can be satisfied at some $t' \geq t$.

Failure models

Failure	Behaviour
Crash	the process stops its execution
Omission	the process does not send (or receive) a message that it should, according to its algorithm
Crash-recovery	the process crashes and stops sending messages, but might recover later
Eavesdropping	the process leaks information obtained in an algorithm to an outside entity
Arbitrary (Byzantine)	the process may deviate arbitrarily from its algorithm

Crash-stop. The process crashes at time t and never recovers. A process is *faulty* if it crashes at some time during the execution, and *correct* if it never crashes and executes an infinite number of steps.

FAULT TOLERANCE, RESILIENCE

Fault tolerance means designing an algorithm that works despite faults, by assuming that at most f of the N processes are faulty. The relation between f and N is called **resilience**; an upper bound f means that any number of processes up to f may fail. *Crash-stop* does not preclude recovery: a recovered process is simply no longer part of the system, and the algorithm must not rely on it to make progress.

Crash-recovery. A process is *faulty* if it crashes and never recovers, or if it keeps crashing and recovering infinitely often; otherwise it is correct, even if it crashed and recovered a finite number of times. A characteristic of this model is *amnesia*: a crashed process loses its internal state, so after recovering it might send messages that contradict those sent before the crash. To cope with this, every process is assumed to have a stable storage (a log), accessed through $\mathrm{store}()$ and $\mathrm{retrieve}()$, and to be aware upon recovery that it crashed and recovered.

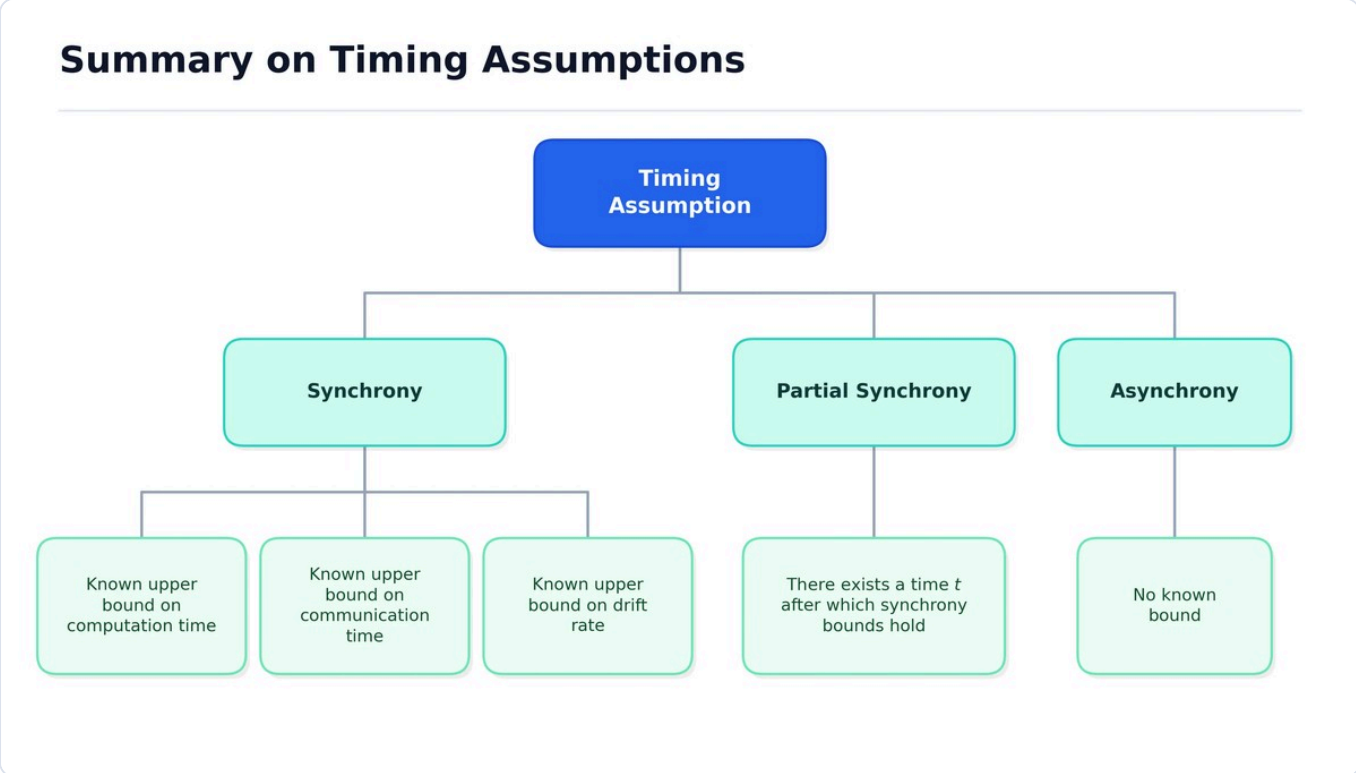


The second one crashes and recovers infinitely often so it is faulty. The third one is correct and the fourth one crashes and recovers finitely so it is correct.

Timing assumptions

Model	Assumption
Synchronous	known upper bounds on (i) the time to execute a basic step, (ii) message transit time, (iii) the drift of local clocks w.r.t. real time
Asynchronous	no known bound on processes or links
Partially synchronous	there exists a time t after which the synchrony bounds hold

Synchronous systems: enable timed failure detection, measuring transit delays, time-based coordination, worst-case performance guarantees and synchronised clocks. The major problem is the coverage of the synchrony assumption: it is hard to build a system in which the bounds hold with high probability. **Asynchronous systems**: even without physical clocks, the passage of time can be measured through the transmission and delivery of messages: time is defined with respect to communication. This is *logical time*, and the resulting notion of clock is a logical clock. **Partial (eventual) synchrony**: distributed systems are usually synchronous most of the time, with periods of bounded asynchrony. This is captured by eventual synchrony: there is an unknown time t after which the system becomes synchronous. It does not mean that the whole system (hardware, software, network) is synchronous forever after t, nor that the system starts asynchronous and later becomes synchronous. What we need is a period of synchrony long enough for the algorithm to terminate.



Abstracting communication

A link represents the network components of the distributed system: every pair of processes is connected by a bidirectional link. The physical topology (fully connected, bus, ring, switched network, ...) may be complex, in which case a routing algorithm realizes the abstraction. Under the crash-failure assumption we consider three links: **fair-loss, stubborn and perfect**. The system model is the following: two processes (sender and receiver); messages can be lost and take an unpredictable time to arrive; processes can crash; the time taken by a process to execute an operation is bounded (the bound may be unknown).

> **REMARK**
>
> Deliver is different from receive. A message is received at a port of the network and stored in a buffer; then an algorithm runs to make sure that the properties of the required link abstraction hold, and only afterwards is the message delivered.

Fair-loss links

Messages can get lost, but if a message is continuously re-transmitted, under certain conditions it is delivered.

Stubborn links

They guarantee continuous re-transmission of messages until they are delivered, so the same message can be received multiple times.

Perfect links

They ensure that messages are delivered reliably, without duplication and without creating non-existent messages.

Time in distributed systems

In a distributed system the processes run on different nodes interconnected by mean of a network (LAN, WLAN...). This processes cooperate to complete a computation and communicate only through message-based communications. Many applications require a perfect ordering between events and synchronization to terminate correctly. **So time is a critical factor for distributed system.**

Why time is so important?

We want to measure the time, but why? because many algorithm depends on time and we have many problems related to time:

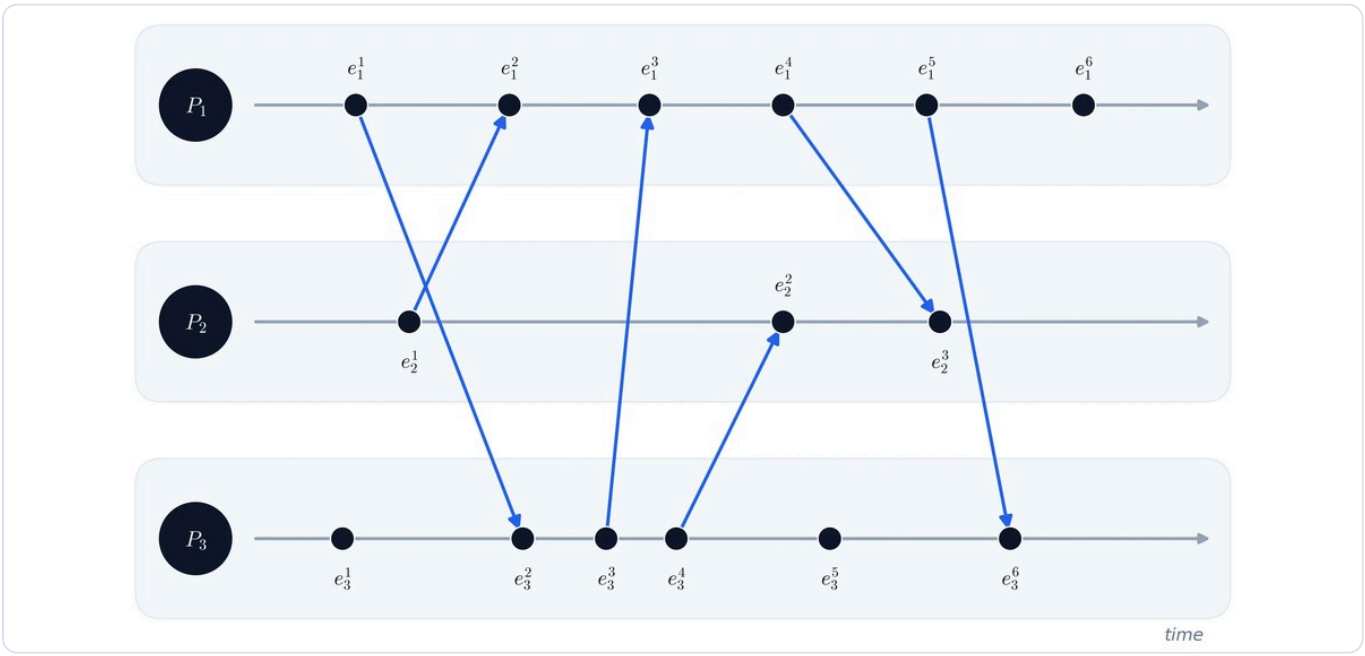
1. **Data consistency**: Ensure that different replicas of a database reflect updates in the correct order;
2. **Authentication**: Validate tokens and time-based certificates;
3. **Double processing avoidance**: Prevent the same operation from being performed twice on different nodes. To solve these problem ordering events is fundamental.

System model

A distributed system is composed by a set $\Pi = \{p_1, \dots, p_n\}$ of n processes. Each *process* runs on a single mono-processor machine with no shared memory. Each process p_i has a *state* s_i that is changed by the actions it takes during algorithm execution. The state s_i includes all the values of variables maintained by p_i. Each process can communicate with other processes only by exchanging messages. Each process generates a sequence of events:

- Internal event, an event that transforms the process state;
- External event. send/receive message;
- e_i^k, k-th event generated by p_i.

The evolution of the computation can be represented with a space-time diagram:



We denote with $\rightarrow_i$ the ordering relationship between two events in the same process p_i: $e \rightarrow_i e' \iff e$ happened before e' in p_i.

LOCAL HISTORY

Sequence of events produced by a process: $\text{history}(p_1) = h_1 = \langle e_1^1, e_1^2, e_1^3, e_1^4, e_1^5, e_1^6 \rangle$

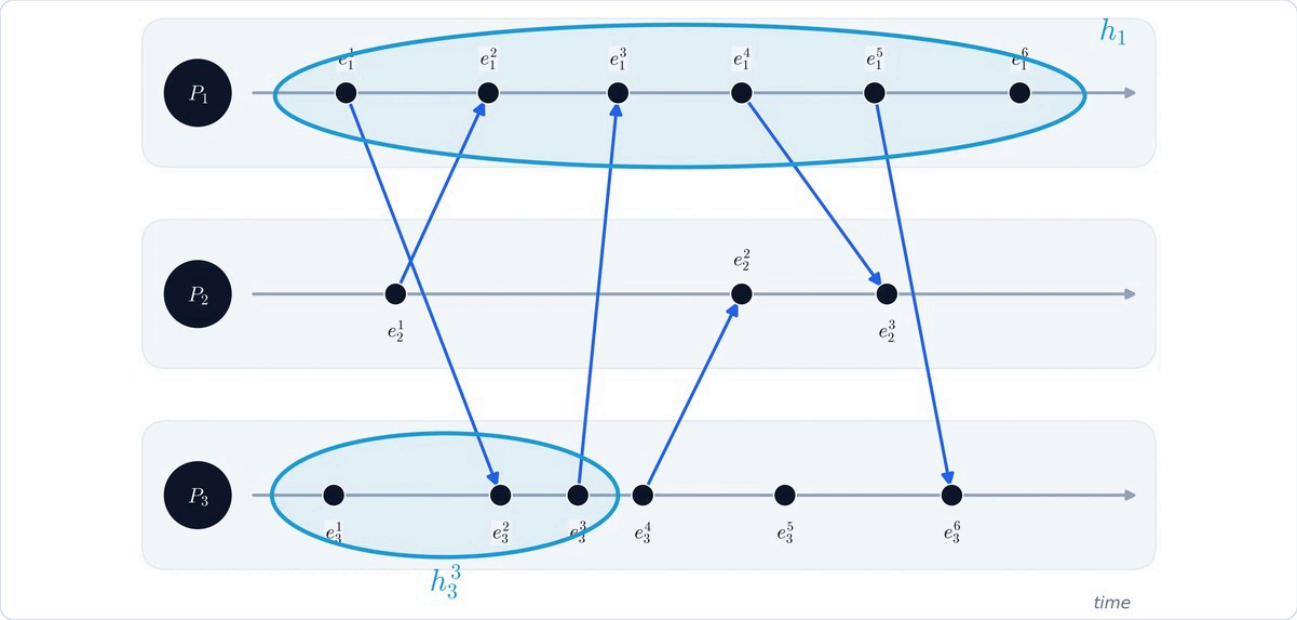
PARTIAL LOCAL HISTORY

Prefix of local history $h_1^m = e_1^1 \dots e_1^m$

GLOBAL HISTORY

Set containing every local history: $H = \bigcup_{i=1}^{n} h_i$

The first (and bigger) circle is the local history of p_1, the second circle is a partial local history of h_3^3



Time-stamping and physical clocks

The naive solution to build a global history is *time-stamping*: every process labels each event with the value of its own physical clock. Events of the same process are always ordered this way, **but for processes on different machines the network and processing delays make it impossible to realise a common clock shared by all of them**. What can be done is to synchronize the physical clocks, through suitable algorithms, up to a certain accuracy (synchronization error), and to timestamp events with the synchronized local clock. A **physical clock** is a counting register incremented at every tick of an oscillator (hardware clock $H_i(t)$). The operating system reads the hardware $H_i(t)$ and produces a software clock $C_i(t) = \alpha H_i(t) + \beta$ which approximates the real time t at process p_i. *Example*: $C_i(t)$ may be implemented by a 64-bit word, representing nanoseconds that have elapsed at time t. Generally this clock is not completely accurate: it can be different from the real time t, it can be different at any process due to the precision of the approximation. C_i can be used such as timestamp for event produced

by p_i. To distinguish two events, its resolution (the time between two consecutive increments) must satisfy $T_{\text{resolution}} < T$ between the two events.

Parameters affecting the Clock Synchronization accuracy

We have two different parameter that affect the clock synchronization accuracy:

- **Skew**: the difference in time between two clocks $\rightarrow |C_i(t) - C_j(t)|$;
- **Drift rate**: the gradual misalignment of once synchronized clocks caused by the slight inaccuracies of the time keeping-mechanism. Difference for time unit with respect to an ideal clock. The standard of time is the **UTC** $\rightarrow$ *U*niversal *T*ime *C*oordinated) is the base of any international measure. Based on international atomic time: 1 sec = time a cesium atom needs for 9192631770 state transitions.

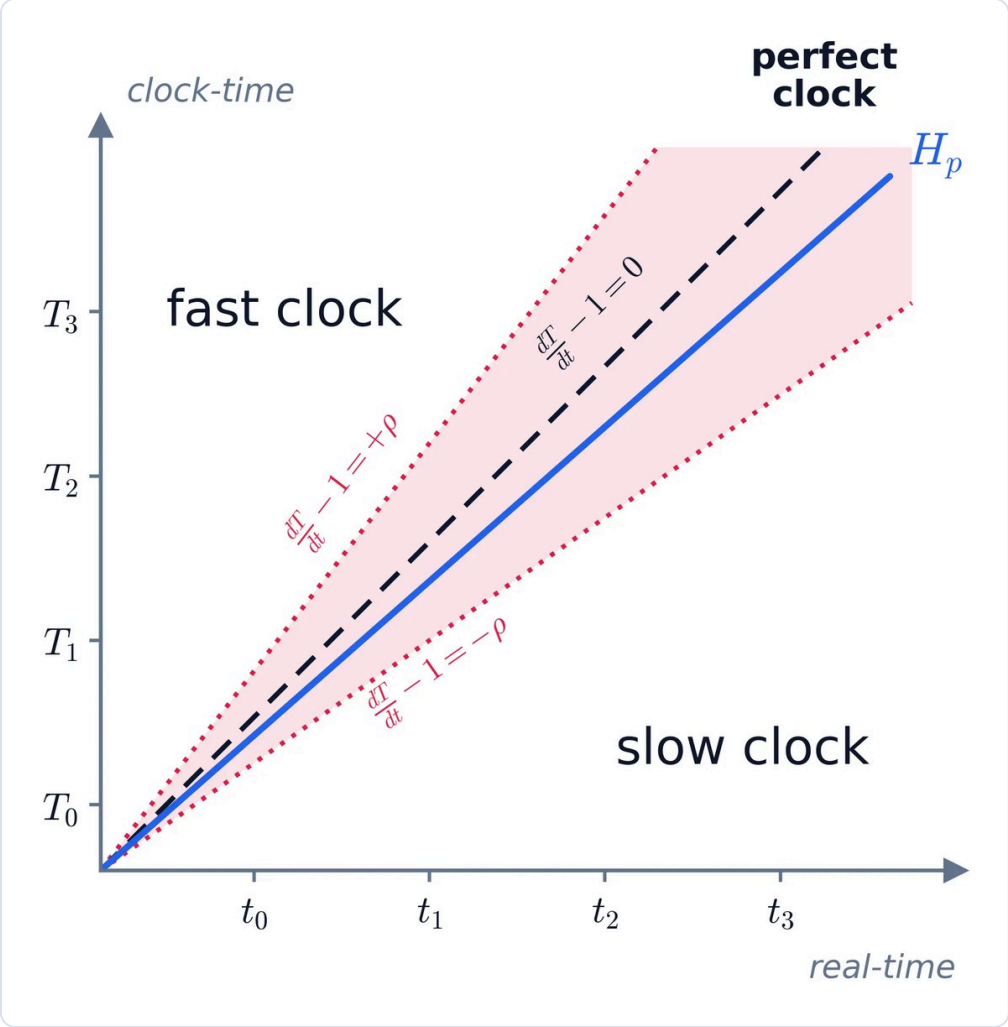
We have two type of synchronization:

1. **External Synchronization**: processes synchronize their clock C_i with an authoritative external source S. Let $D > 0$ be the synchronization bound and S be the source of UTC. Clocks C_i for $i = 1, \dots, N$ are **externally synchronized** with a time source S (UTC) if for each time interval I: $|S(t) - C_i(t)| < D$ for $i = 1, 2, \dots, N$ and for real time t in I. We say that clocks C_i are accurate within the bound of D.
2. **Internal Synchronization**: all the processes synchronize their clocks C_i between them. Let $D > 0$ be the synchronization bound and let C_i and C_j the clocks at processes p_i and p_j respectively. Clocks are **internally synchronized** in a time interval I: $|C_i(t) - C_j(t)| < D$ for $i, j = 1, 2, \dots, N$ and for all time t in I. We say that clocks C_i, C_j agree within the bound of D.

Internally synchronized clocks are not necessarily externally synchronized: they may drift collectively from the external source. Conversely, processes externally synchronized within D are internally synchronized within 2D (by the triangle inequality).

Correct clock

A *hardware clock* H is **correct** if its drift rate is within a limited bound of $\rho > 0$, respecting: $1 - \rho \leq \frac{dC}{dt} \leq 1 + \rho$. In the presence of a correct hardware clock H we can measure a time interval $[t, t']$ ($\forall t' > t$) introducing only a limited error. So we can quantify and limit the error: $(1 - \rho)(t' - t) \leq H(t') - H(t) \leq (1 + \rho)(t' - t)$. On the left of the formula we have the lower limit, which describes the case when the hardware clock is slow: the interval of time measured by the hardware will be shorter than the real time, but the formula guarantees that it cannot be less than $(1 - \rho)(t' - t)$.



A HARDWARE CLOCK IS CONSIDERED CORRECT IF ITS SPEED DEVIATION RESPECTS THIS BAND BETWEEN $(1 - \mathrm{P})$ AND $(1 + \mathrm{P})$. THANKS TO THIS INEQUALITY, WE KNOW WITH MATHEMATICAL CERTAINTY THAT, MEASURING AN INTERVAL WITH A CLOCK WITH A CERTAIN DRIFT P, THE INTRODUCED ERROR WILL REMAIN CONFINED WITHIN VERY PRECISE AND PREDICTABLE LIMITS.!

Software clocks have to be **monotone**: $t' > t \implies C(t') > C(t)$. The monotonic property can be guaranteed by choosing opportune α and β. (Note that α and β can be functions of time).

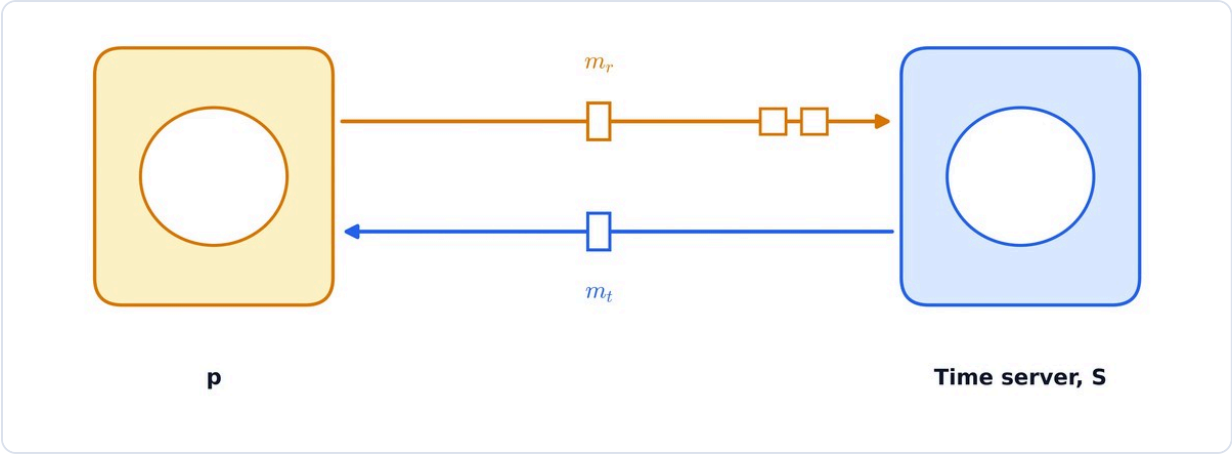
Clock failure:

- *Crash failure*: clock simply stops;
- *Arbitrary failure*: clock behaves arbitrarily (e.g. Y2K bug: the day after the 31/1/1999 becomes 1/1/1900 rather than 1/1/2000)

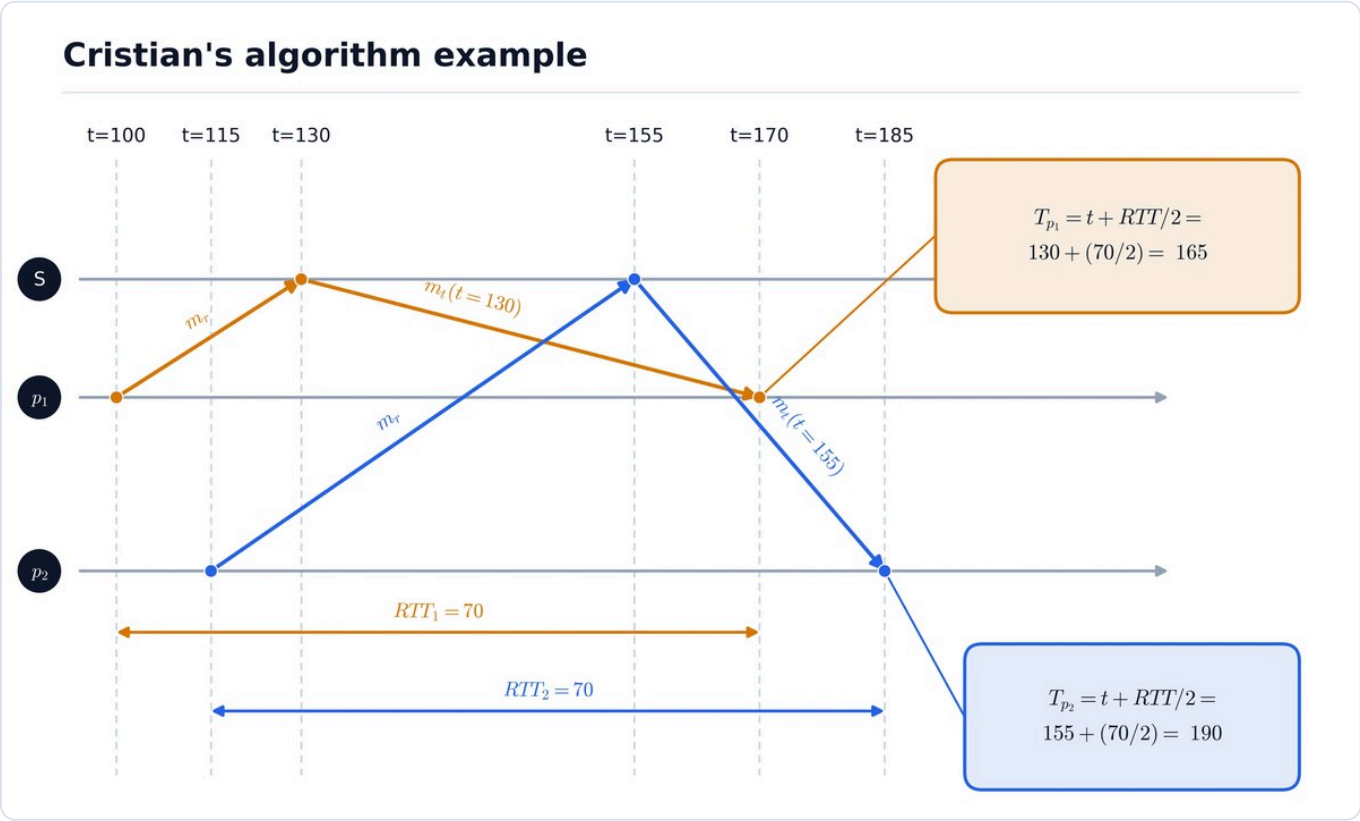
Synchronization algorithm based on time server

Christian's algorithm (external synchronization)

- **Operation**: A process p sends a time request (ask the current time) through a message m_r to a time server S (connected to a UTC source) and receives the timestamp t via m_t.



- **Timing**: The process sets its clock to $t + \frac{\mathrm{RTT}}{2}$, where RTT (Round Trip Time) is the total round-trip time for the message.
- **Accuracy**: The maximum estimated error is $\pm\left(\frac{\mathrm{RTT}}{2} - \min\right)$, where $\min$ is the minimum estimated transmission time on the network.
- **Limitations**: Represents a single point of failure; requires the use of redundant server clusters and authentication mechanisms. A time server can crash and can be attacked. There could exist periods in which the synchronization is not possible $\rightarrow$ ask to multiple servers at the same time.

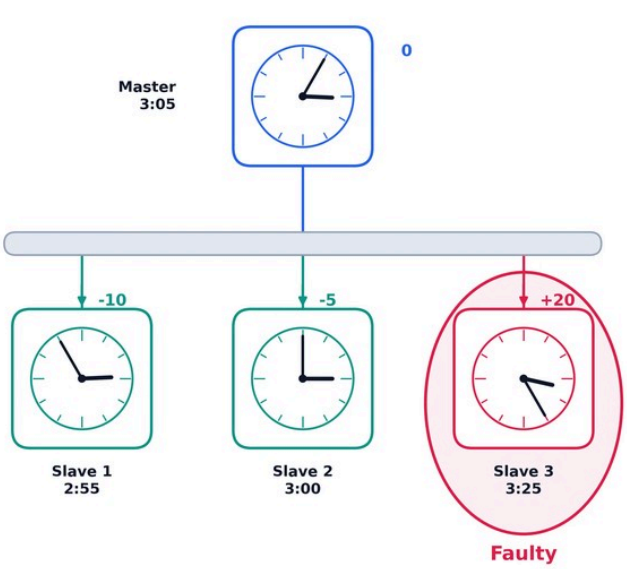


Berkeley Algorithm (Internal Synchronization)

- **Structure**: Master-slave architecture.
- **Algorithm steps**:
  1. The master p_m periodically polls the slaves, sending its own timestamp t_1 and receiving the nodes' local timestamps t_2 in return.

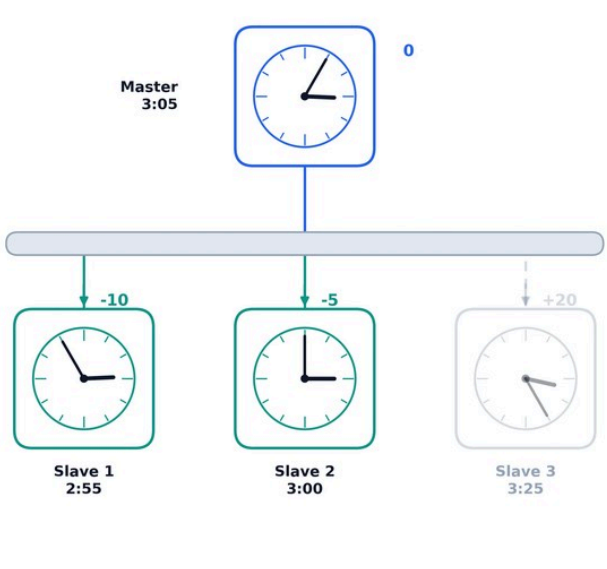
2. The master calculates the differences Δ ($\Delta = \frac{t_1+t_3}{2} - t_2$ where t_3 is the local clock when the master receive the timestamps t_2 in return) between its own clock and that of each process.
3. The master discards the values of faulty processes (whose difference exceeds a threshold γ).
4. It calculates the average of the valid values Δp_i and sends each process the algebraic correction to apply using $\mathrm{Adj}_{p_i} = \mathrm{avg} - \Delta p_i$.

- **Slowdown management**: To avoid jumping back in time (violating causality and monotonicity), excess clocks are not set to a past value, but are slowed down by temporarily hiding hardware clock interrupts.



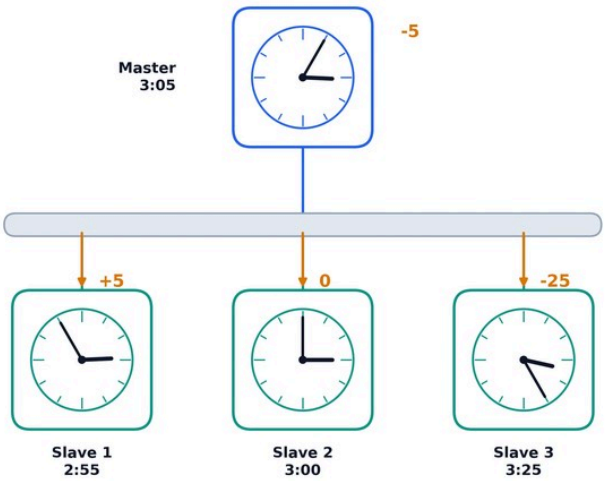
Measuring the differences

- $\Delta p_m = 3:05 - 3:05 = 0$
- $\Delta p_1 = 2:55 - 3:05 = -10$
- $\Delta p_2 = 3:00 - 3:05 = -5$
- $\Delta p_3 = 3:25 - 3:05 = +20$



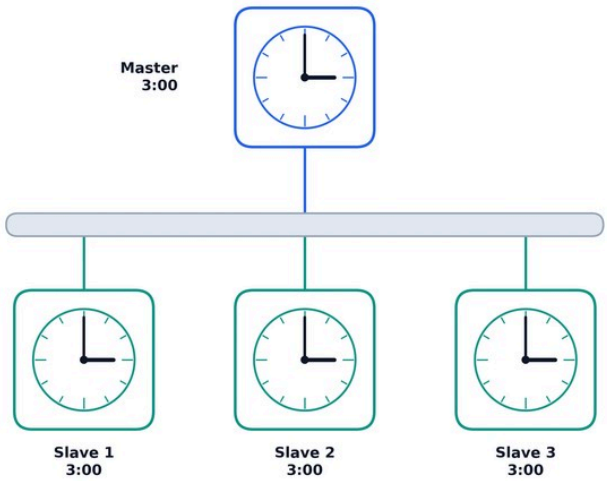
Computing the average

- $\mathrm{Avg} = (0 - 10 - 5)/3 = -5$



Compute and send the Correction

- $\text{Adj}_m = \text{Avg} - \Delta p_m = -5 - 0 = -5$
- $\text{Adj}_1 = \text{Avg} - \Delta p_1 = -5 - (-10) = 5$
- $\text{Adj}_2 = \text{Avg} - \Delta p_2 = -5 - (-5) = 0$
- $\text{Adj}_3 = \text{Avg} - \Delta p_3 = -5 - 20 = -25$



Apply the correction

- All clocks now read 3:00

- **Accuracy**: the protocol accuracy depends on the maximum round-trip time. The master does not consider clock values associated to RTT grater than the maximum one.
- **Fault tolerance**:
  - If the master crashes, another master is elected (in an unknown time);
  - It is tolerant to arbitrary behavior (eg. slaves that send wrong values);
  - Master process consider a certain number of clock values and these values do not differ between them over a certain threshold;

WHAT DOES SLOWING DOWN A CLOCK MEAN?

Imagine that the average calculated by the master determines that the correct clock is 12:00, but a slave (having a faulty or fast clock) reads 12:05. It's not possible to directly set a passed value to the slave clocks. If we told the slave to suddenly "jump" from 12:05 to 12:00, a logical disaster would ensue:

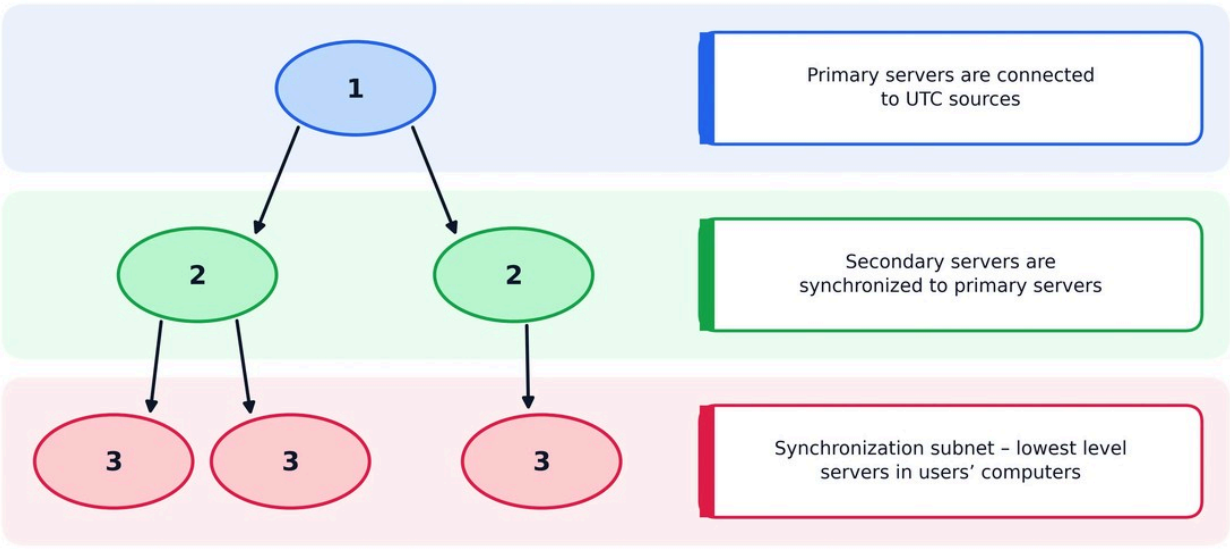
- The *monotonicity* of time would be violated (time must always advance, never go backward: $t' > t \implies C(t') > C(t)$).
- The *cause-effect order* (causality) would be violated: events that logically happened earlier could end up with a later timestamp than events that happened later, creating massive conflicts in logs and databases. **Solution**: slow down the clock! Since we can't make the clock "jump" back in time, the only legitimate solution is to slow down the slave's clock speed until the rest of the system "*reach it*". Computers update their software clocks using periodic hardware interrupts (electronic signals repeated at regular intervals, e.g., every millisecond, that tell the operating system to add a "tick" to the time counter). To slow down the clock, the operating system hides (ignores) a certain number of these interrupts. The system calculates how many specific interrupts to skip based on the amount of

> accumulated delay and the original tick rate. Practical example: If the counter normally ticks 1000 times per second, we decide to ignore some of these signals. This way, for a certain period, time on that machine will run slower than normal, allowing the average time calculated by the master to "catch up" with it without leaping into the past.

Network time protocol (NTP)

NTP is a standard de facto for **external clock synchronization** of distributed system on Internet. It synchronizes clients with UTC. NTP employs several security mechanisms (e.g. mechanisms for authentication of time references) usually they are not required in a local area network. Based on a remote reading procedure like Christian's Algorithm, NTP specification adds to the basic algorithm mechanisms for clustering, filtering and evaluating data quality in order to minimize the synchronization. Stratum Hierarchy:

1. **Stratum 1** (Primary servers): Connected directly to the primary UTC time sources.
2. **Stratum 2** (Secondary servers): Synchronized hierarchically with the primary servers.
3. **Stratum 3** *and above*: Synchronization subnets for end-user computers.



The NTP hierarchy is reconfigurable in the presence of faults:

- Primary server that loses its connection with UTC-signal can become a secondary server;
- Secondary server that loses its connection with a primary server (e.g. a crash of the primary server) can contact and connect itself to another primary.

NTP Synchronizations models

- **Multicast**: server periodically send its actual time to its leaves in the LAN. Leaves set their time using the received time assuming a certain delay. *It is used in quick LANs but it shows a low accuracy.*
- **Procedure call**: server replies to request with its actual timestamp (like Christian's algorithm). *High accuracy and it is useful when it is not available hw multicast.*

- **Symmetrical**: used to synchronize between pairs of time servers using messages containing timing information. *Only used in high level of hierarchy*.

Time in asynchronous system

In an asynchronous distributed system, global **physical time is not directly observable** in a deterministic way due to the unpredictability of network communication delays. Clocks can only be synchronized with a certain degree of probability, and an accuracy bound can only be calculated by knowing rigorously the upper and lower limits of the transmission delays.

Logical Clocks and Distributed Mutual Exclusion

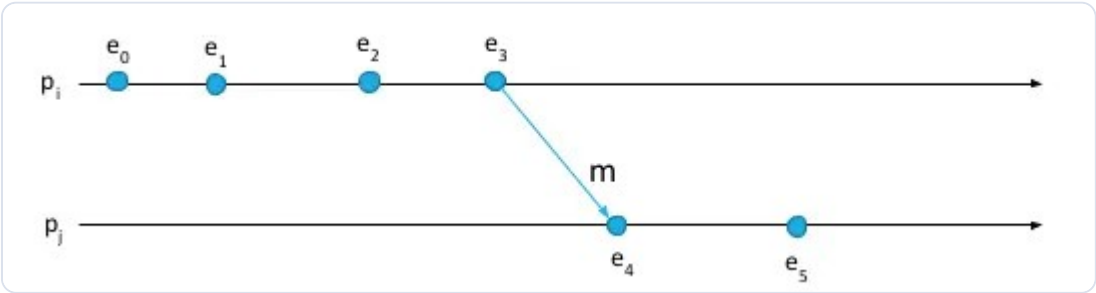
Synchronizing physical clocks in distributed systems is complex, and accuracy depends heavily on estimating network communication delays, which are often difficult to calculate precisely. The key insight: In many distributed applications, it is not essential to know the exact time an event occurred, but it is essential to determine with certainty the order in which events occurred. **Goal**: Find a reliable way to order events without having to rely on physical clock synchronization.

Happened-before relation

- We note with $\rightarrow_i$ the ordering relation between events in a process p_i
- We note with $\rightarrow$ the happened-before between any pair of events
- Two events occurred at some process p_i happened in the same order as p_i observes them
- When p_i sends a message to p_j the send event happens before the deliver event

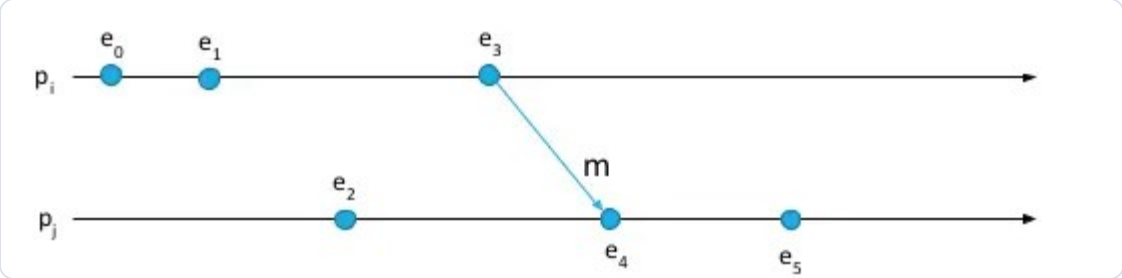
Formal definition ($e \rightarrow e'$): Two events e and e' are related by the happened-before relation if one of the following conditions is true:

1. **Same process**: $\exists p_i | e \rightarrow_i e'$
   - e and e' occur in the same process p_i and e precedes e' in time.
2. **Communication**: $\exists m | e = send(m) \text{ and } e' = deliver(m)$
   - e corresponds to the sending of a message ($\text{send}(m)$) by one process and e' corresponds to the reception/delivery of the same message ($\text{deliver}(m)$) by another process ($\text{send}(m) \rightarrow \text{deliver}(m)$).
3. **Transitivity**: $\exists e, e', e'' | (e \rightarrow e') \wedge (e'' \rightarrow e')$
   - There exists an intermediate event e'' such that $e \rightarrow e''$ and $e'' \rightarrow e'$.



Happened-before relation imposes a partial order over events of the execution history:

- If two events e_i and e_j are not linked by any cause-effect relationship, they are said to be **concurrent** ($e_i \parallel e_j$).
- In a distributed system, for any pair of events only one of three conditions is always true:
  1. $e_i \rightarrow e_j$;
  2. $e_j \rightarrow e_i$;
  3. $e_i \parallel e_j$.

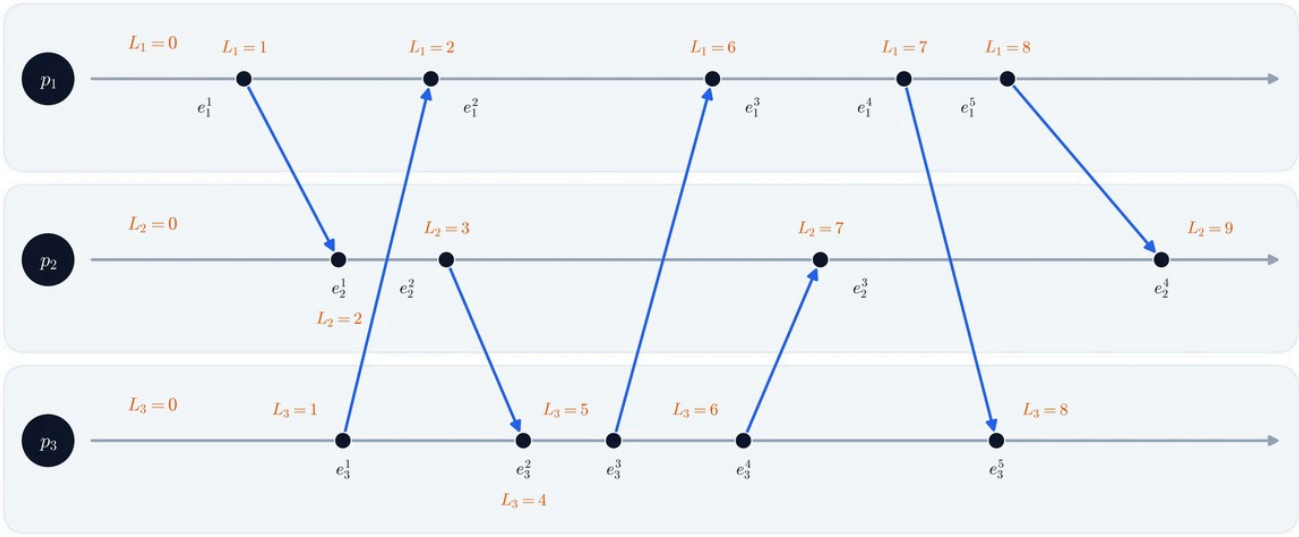


Logical clock and scalar logical clock

The Logical Clock, introduced by Lamport, is a software counter that monotonically increases its value. A logical clock L_i can be used to timestamp events: $ts_e = L_i(e)$ is the "logical" timestamp assigned by a process p_i to events e using its current logical clock with a property: if $e \rightarrow e'$ then $ts_e < ts_{e'}$. Implementation Rules:

- Each process p_i initializes its local clock to zero ($L_i = 0$);
- The process increments L_i by 1 each time it generates a local event (send or receive);
- **Sending**: When p_i sends a message m, it appends the current timestamp value $ts = L_i$;
- **Receiving**: When p_i receives a message with a timestamp ts, it updates its clock by calculating the maximum and incrementing it: $L_i = \max(ts, L_i) + 1$ (before generating the receive event).

Example:



e_i^j is $j-th$ event of process p_i, L_i is the logical clock of p_i.

- **System Overview:** The diagram illustrates Lamport's Logical Clocks in a distributed system consisting of three processes (p_1, p_2, and p_3).
- **Initial State:** All processes start with their logical clocks set to zero ($L_1 = 0$, $L_2 = 0$, $L_3 = 0$).
- **Local Events & Internal Increments:** As time progresses locally within each process, events occur and local counters increment sequentially (e.g., L_3 goes from 1 to 4).
- **Message Passing & Synchronization (Blue Arrows):** When a message is sent from one process to another, it carries the sender's current timestamp. Upon reception, the receiving process updates its own clock by

taking the maximum between its current local clock and the incoming timestamp, then adding 1 ($\max(\text{local}, \text{msg}) + 1$).

$e_1^1 \rightarrow e_1^2$ and timestamps reflect this property $e_1^1 \parallel e_3^1$ and respective timestamps have the same value $e_1^2 \parallel e_3^1$ but respective timestamps have different values

Properties and Limitations:

- **Guaranteed Property**: If $e \rightarrow e'$, then their timestamp satisfies the inequality $ts_e < ts_{e'}$.
- **Critical Bound**: The implication is one-way. The converse is not true: if $ts_e < ts_{e'}$, we cannot deduce with certainty that $e \rightarrow e'$. With scalar clocks, it is not possible to determine whether two events are causally related or concurrent ($e \parallel e'$).

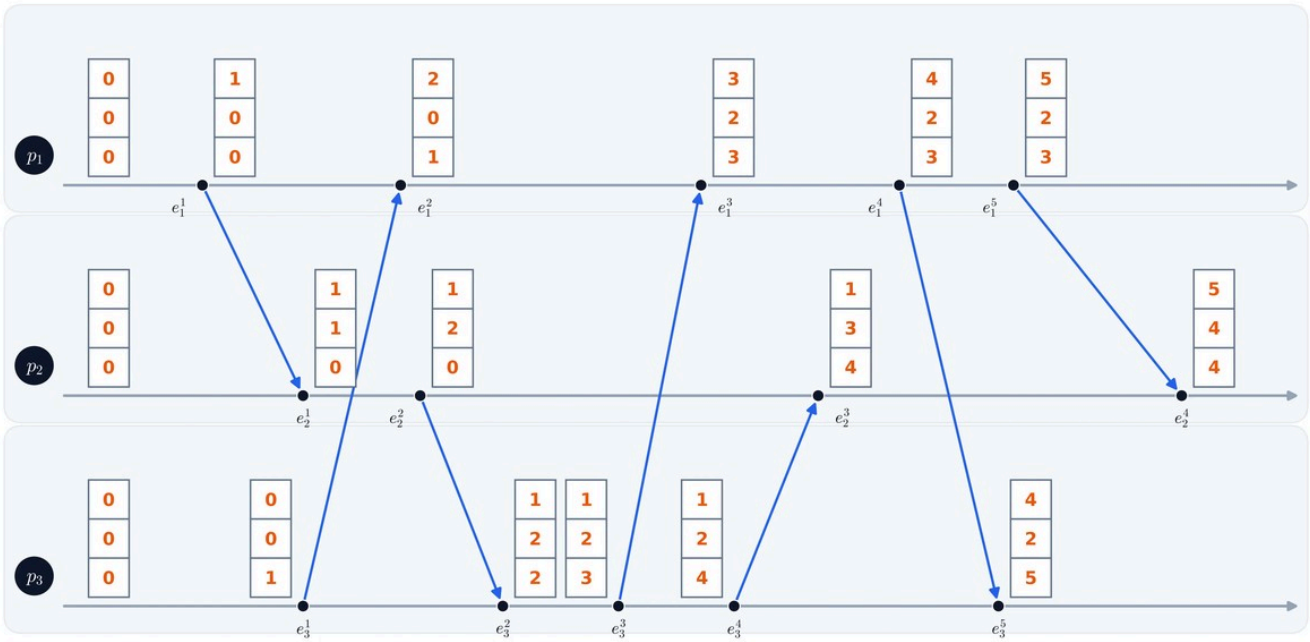
Vector clock

Vector Clock for a set of N processes is composed by an array of N integer counters. Each process p_i maintains a Vector Clock V_i and timestamps events by mean of its Vector Clock. Similarly to scalar clock, Vector Clock is attached to message m, but in this case the timestamp will be an integer vector. Vector Clock allows nodes to order events in happens-before just looking at their timestamps:

- Scalar clocks: $e \rightarrow e' \implies L(e) < L(e')$;
- Vector clocks: $e \rightarrow e' \iff L(e) < L(e')$.

Definition and Implementation:

- In a system with N processes, each process p_i maintains a vector of integers V_i of size N.
- $V_i[i]$ tracks the number of events autonomously generated by p_i.
- $V_i[j]$ (with $i \neq j$) represents the updated knowledge that p_i has about the events generated by p_j.
- Generating an event: p_i increments its component in the vector: $V_i[i] = V_i[i] + 1$.
- **Sending**: The message is stamped by attaching the entire vector $ts = V_i$.
- **Receiving**: The receiving process updates each component of its vector by maximizing the received vector: $V_i[j] = \max(ts[j], V_i[j])$ for each $j = 1 \dots N$, and then increments its component.

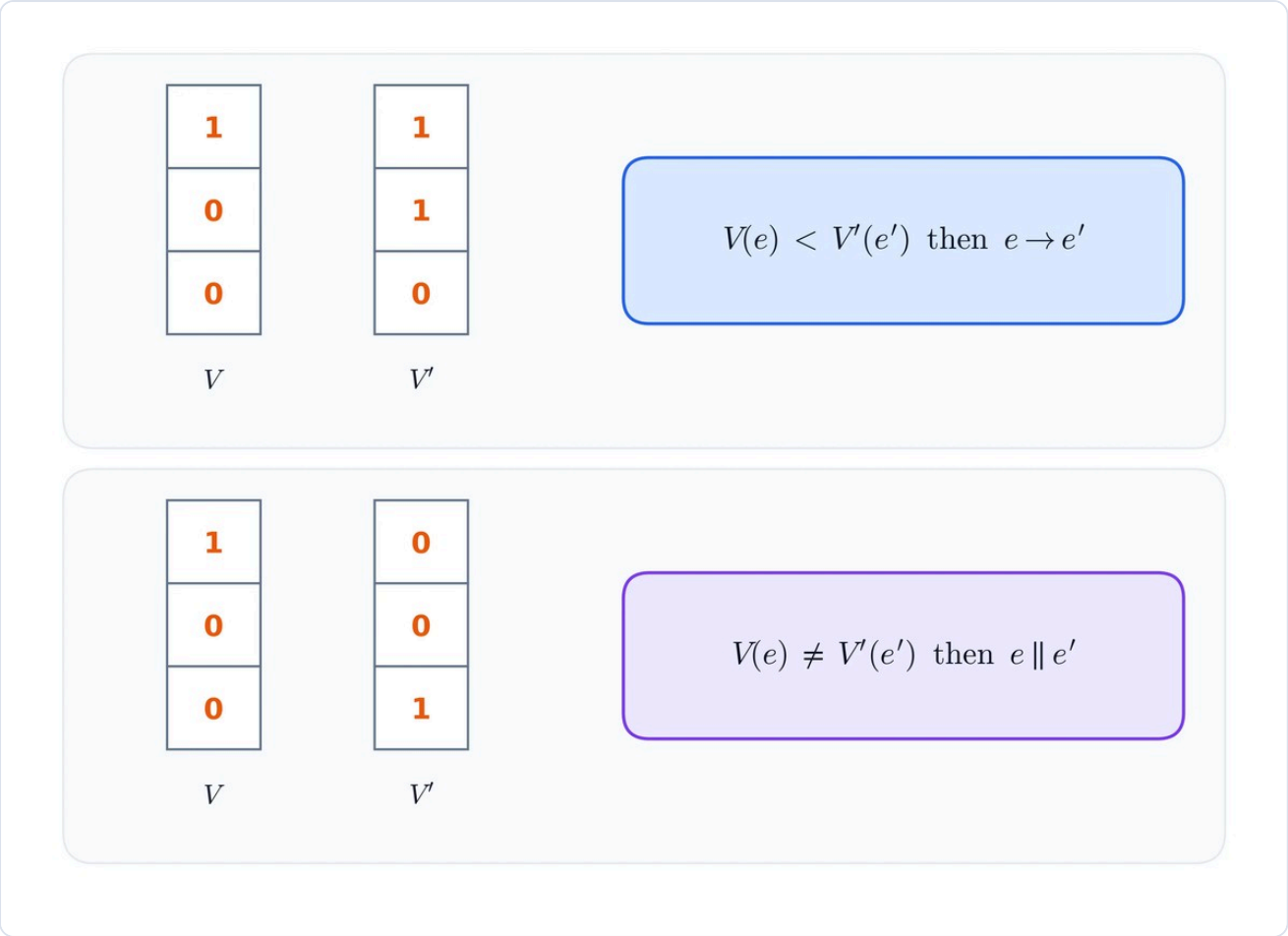


A Vector Clock V_i:

- $V_i[i]$ represents the number of events produced by p_i.
- $V_i[j]$ with $i \neq j$ represents the number of events generated by p_j that p_i knows.

Properties of Vector Clocks

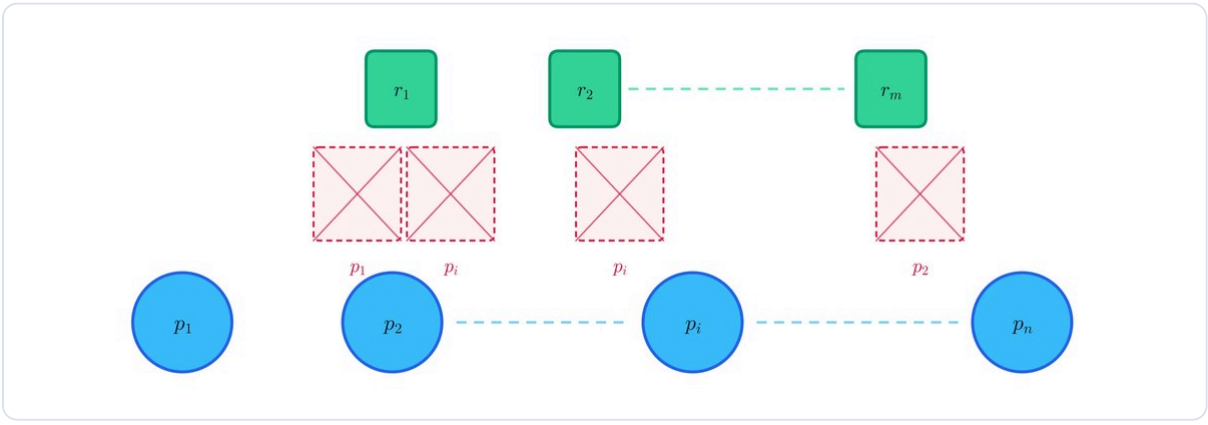
- **Equality ($V = V'$):** $V = V' \iff V[j] = V'[j] \quad \forall j = 1 \dots N$
- **Less than or equal ($V \leq V'$):** $V \leq V' \iff V[j] \leq V'[j] \quad \forall j = 1 \dots N$
- **Causal precedence ($V < V'$):** The event associated with V happened before the event associated with V' if and only if: $V < V' \iff (V \leq V') \wedge (V \neq V')$ Equivalently, expressed in components: $\forall i = 1 \dots N, \quad V'[i] \geq V[i] \; \exists i \in \{1 \dots N\} \mid V'[i] > V[i]$



Differently from Scalar Clock, Vector Clock allows to determine if two events are concurrent or related by a happened-before relation.

Distributed mutual exclusion

Let us consider a set of processes $\Pi = \{p_1, \ldots, p_n\}$ and a set of resources $R = \{r_1, \ldots, r_m\}$. The system is asynchronous, the process will be always correct and can communicate by exchanging messages on top of perfect point to point links.



Processes need to access resources exclusively and we need to design a distributed abstraction that allows them to coordinate to get access to resources. Distributed Mutual Exclusion is a fundamental problem in distributed systems. Imagine having a set of processes distributed across different machines connected via a network (with

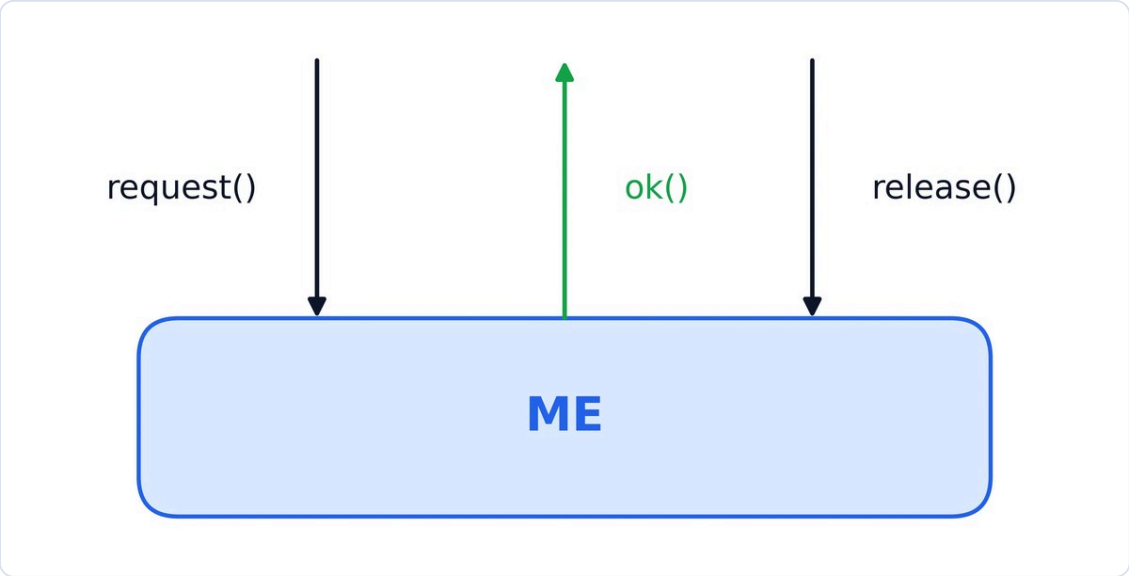
no shared memory) that need access to a shared resource (called a Critical Section or CS, such as a file to write, a database, or a printer). The mutual exclusion problem involves designing a coordination mechanism that allows processes to access it in a safe and orderly manner.

Events

- `request()`: it issues a request to enter into the critical section;
- `ok()`: it notifies the process that it can now access the critical section;
- `release()`: it is invoked to leave the critical section and to allow someone else to enter.

Properties

- **Mutual exclusion:** at any time t, at most one process p is running the critical section;
- **No-deadlock:** there always exists a process p able to enter the critical section;
- **No-starvation:** every `request()` and `release()` operation eventually terminate.



Lamport's algorithm

The Lamport Algorithm for Distributed Mutual Exclusion is one of the most important classical methods in distributed systems for regulating access to a shared resource (Critical Section, or CS) using scalar logical timestamps. In this algorithm, each process p_i in the system maintains two fundamental structures:

- A *local logical clock counter* (ck).
- A *local request queue* (Q) in which all requests for access to the Critical Section (both its own and those of other processes) are stored in chronological order.

The Phases of the Algorithm

Phase 1: The Request

- When a process p_i wishes to access the Critical Section, it increments its logical clock and sends a `REQUEST` message (containing the timestamp t and its identifier id) to **all** other processes in the system.
- Simultaneously, p_i inserts its request into its local queue Q.

Phase 2: Reception and Response (Ack)

- When any other process p_j receives the `REQUEST` message from p_i:
  - Inserts p_i's request into its local queue Q.
  - Responds by immediately sending an acknowledgement (**ACK**) message to p_i. (Note: The ACK travels with its own next logical timestamp.)

Phase 3: Entering the Critical Section A process p_i can enter the Critical Section **only if** three conditions are simultaneously true:

1. Its request is at the top of its queue Q (meaning it has the **smallest timestamp** of all pending requests).
2. p_i has already received an ACK message from *all* other processes in the system.
3. The timestamps of received ACKs are strictly greater than the timestamp of its own request. *Why is this rule necessary?* Because the arrival of an ACK from another node guarantees p_i that that node has seen its request and has recorded a higher logical clock; therefore, no other node will be able to insert a request with a timestamp "in the past" that should have taken precedence.

Phase 4: The Release

- Once the work within the Critical Section is complete, process p_i:
  - Sends a `RELEASE` message to all other processes.
  - Removes its request from its local Q queue.
- When other processes receive the `RELEASE` message from p_i, they also remove p_i's request from their local Q queues. This refreshes the queues and frees the resource for the next process in the queue.

PICTURES OF THE EXAMPLE

Safety proof

Let's assume, for the sake of argument, that both p_i and p_j manage to enter the Critical Section. According to the rules of Lamport's algorithm, for a process to enter the CS, two things must be simultaneously true:

- The process must have received an ACK from *all* other nodes in the system.
- Its request must have the absolute smallest timestamp within the local queue (Q). If we assume that p_i sent a request with a timestamp of t_i and p_j with a timestamp of t_j:
- One of the two must necessarily have the temporally smallest timestamp than the other (assume $t_i < t_j$).
- Since logical timestamps create a total order (thanks to scalar counters and process IDs in case of ties), the queues of all nodes will reflect exactly the same priority order.
- Consequently, p_j will never be able to consider its own timestamp t_j as "the smallest" if the timestamp of p_i (t_i) which arrived first is in the queue, or p_j has not yet received the ACK from all nodes because it is blocked by the priority of p_i.

Properties of the algorithm

- **Fairness (Absence of Starvation):** Requests are fulfilled strictly in the chronological order established by logical timestamps (*happened-before* relationship). First come, first served.
- **Fairness (Mutual Exclusion):** Two processes are guaranteed never to enter the CS at the same time, precisely because the combination of ordered timestamps and universal ACKs creates a global and consistent view of

the queues on all nodes.

- **Message Cost:** For each single access to the Critical Section, the Lamport algorithm requires a fixed total of $3(N-1)$ **messages**, divided as follows:
- $N-1$ `REQUEST` messages sent to all other nodes.
- $N-1$ `ACK` response messages.
- $N-1$ `RELEASE` messages.

Ricart-Agrawala's algorithm

The *Ricart-Agrawala algorithm* is an evolution and optimization of the Lamport algorithm to solve the Distributed Mutual Exclusion problem. While the Lamport algorithm requires sending `REQUEST`, `ACK`, and `RELEASE` messages separately (for a total of $3(N-1)$ messages), Ricart-Agrawala merges and reduces message exchange by leveraging dynamic responses called `REPLYs`.

Each process maintains the following information about its state:

- `State`: The current state of the process, which can be:
  - `NCS` (Non Critical Section): The process is not interested in the resource (initial state).
  - `Requesting`: The process is requesting entry into the Critical Section.
  - `CS`: The process is currently within the Critical Section.
- `num`: The local logical clock counter (scalar).
- `Last_Req`: The timestamp of the last request sent by the process.
- `#replies`: A counter that tracks permissions (`REPLY`) received from other nodes (initially 0).
- `Q`: A queue of pending requests (requests received from others that have been decided not to respond to yet).

The Phases of the Algorithm

Phase A: When a process wants to enter the Critical Section

1. Process p_i changes its state to `Requesting`, increments its logical clock, and sets `num`.
2. It sends a `REQUEST(num, i)` message (containing its timestamp and ID) to **all** other $N-1$ processes in the system.
3. It **waits** patiently until it receives permission from all other nodes, that is, until the `#replies` counter reaches the value $N-1$.
4. Once it has obtained all permissions, it enters the Critical Section (`State = CS`).

Phase B: When a process p_j receives a `REQUEST(t, i)` request from p_i When a node p_j receives a request from another node p_i, it must decide whether to respond immediately or postpone the response, comparing priorities (based on timestamp and process ID: the lowest `{timestamp, id}` pair takes precedence):

- **Conditions for postponing the response (deferring the `REPLY`):** Process p_j **does not** respond immediately if it is in one of these two situations:

1. p_j is currently in the Critical Section (`State == CS`).
2. p_j is in the `Requesting` state **and** has a higher priority than p_i (i.e., its `Last_Req` timestamp/ID is less than that of p_i: `{Last_Req_j, j} < {t, i}`). *In this case*, p_j puts p_i's request into its local pending queue `Q`.

- **When to respond immediately:** If p_j is not interested in the resource (`State == NCS`) or has a lower priority than p_i, then it **immediately sends a `REPLY` message** to p_i.

Phase C: When the process leaves the Critical Section (Release) Once it has finished its work in the Critical Section, process p_i:

1. It scrolls through its local queue `Q` for requests it had queued and sends a `REPLY` message to each of those processes.
2. It empties the `Q` queue, returns the state to `NCS`, and resets the `#replies` counter.

for the example -> DDS lecture5B

Coordinator, Token and Quorum Algorithms for Mutual Exclusion

We keep the same setting as before: a set of processes Π, a single resource, an asynchronous system, correct processes and perfect point-to-point links. After the logical clock-based algorithms, this lecture presents the other three approaches: coordinator-based, token-based and quorum-based. *Basic idea:* There exist a special process, called coordinator, that collects requests and grant permission to enter into the critical section.

Process code

$$
\begin{aligned}
&\textbf{Init:}\\
&\quad \text{state} := \text{idle};\\
&\quad \text{coordinator} := \text{getCoordinatorId}();\\
&\\
&\textbf{upon event } \text{request}() \textbf{ do}\\
&\quad \text{state} := \text{waiting};\\
&\quad \text{trigger pp2pSend}(\text{REQ}, i) \text{ to coordinator};\\
&\\
&\textbf{upon event } \text{pp2pDeliver}(\text{GRANT_CS}) \textbf{ do}\\
&\quad \text{state} := \text{CS};\\
&\quad \text{trigger ok}();\\
&\\
&\textbf{upon event } \text{release}() \textbf{ do}\\
&\quad \text{state} := \text{idle};\\
&\quad \text{trigger pp2pSend}(\text{REL}, i) \text{ to coordinator};
\end{aligned}
$$

This is the pseudocode that runs on every generic process p_i that needs access to a shared resource (Critical Section).

- `Init` **(Initialization):**
  - `state = idle`: Initially, the process is in an "idle" state (it does not need the resource).
  - `coordinator = getCoordinatorId()`: The process locates and stores the identifier of the node acting as coordinator in the system.
- `upon event request()` **(When the application requests the resource):**
  - `state = waiting`: The process changes its state to "waiting".
  - `trigger pp2pSend(REQ, i) to coordinator`: Sends a point-to-point `REQ` request message (containing its ID i) to the coordinator.
- `upon event pp2pDeliver(GRANT_CS)` **(When permission arrives from the coordinator):**
  - `state = CS`: The process enters the Critical Section state (can use the resource).
  - `trigger ok()`: Notifies the local application that it is now safe to proceed.
- `upon event release()` **(When the application has finished using the resource):**
  - `state = idle`: The process returns to the idle state.

- `trigger pp2pSend(REL, i) to coordinator`: Sends a `REL` release message to the coordinator to notify it that it has freed the resource.

Coordinator code

Init:
pending := ∅;
current_in_CS := ⊥;

upon event pp2pDeliver(REQ, j) from p_j **do**
pending := pending ∪ {p_j};

when pending ≠ ∅ ∧ current_in_CS = ⊥ do
candidate := select_process(pending);
pending := pending \ {candidate};
current_in_CS := candidate;
trigger pp2pSend(GRANT_CS) to candidate;

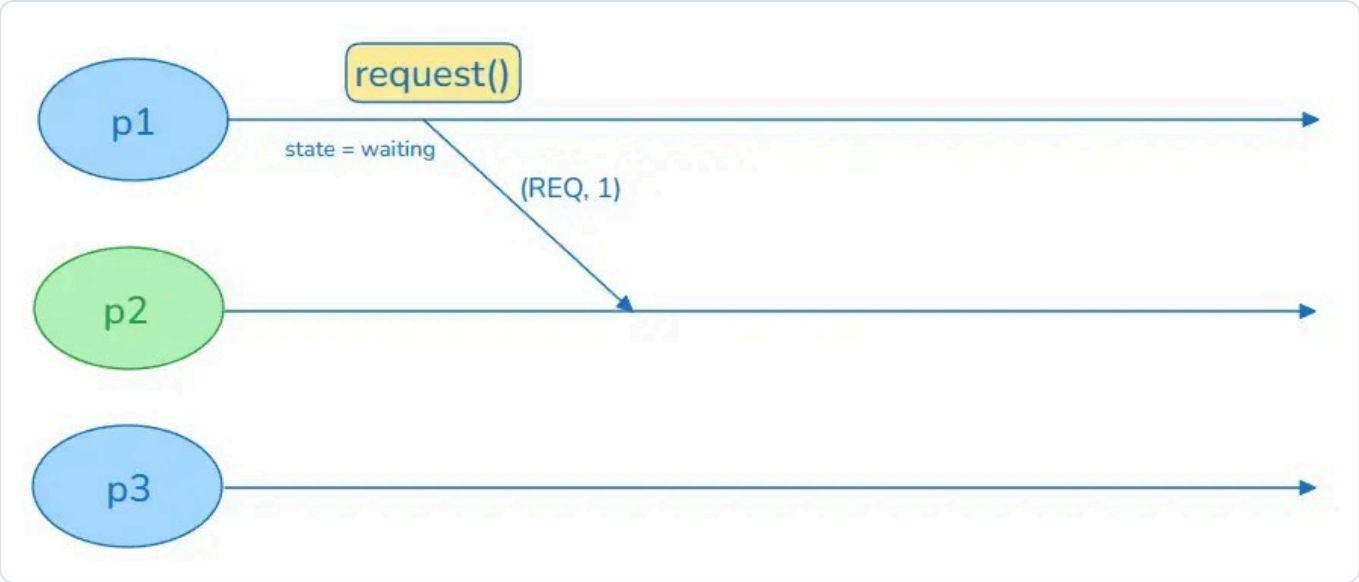
upon event pp2pDeliver(REL, j) from p_j do
if current_in_CS = j then
current_in_CS := ⊥;

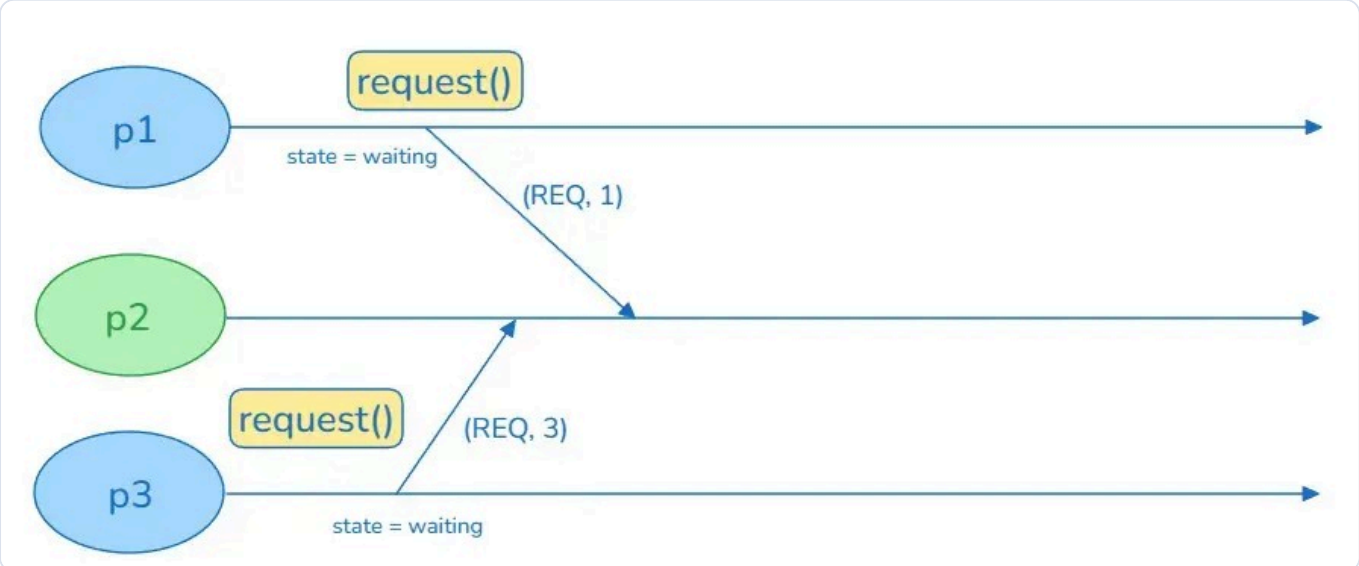
This is the code executed exclusively by the process elected as coordinator, which acts as a central "traffic policeman" for dispatching permissions.

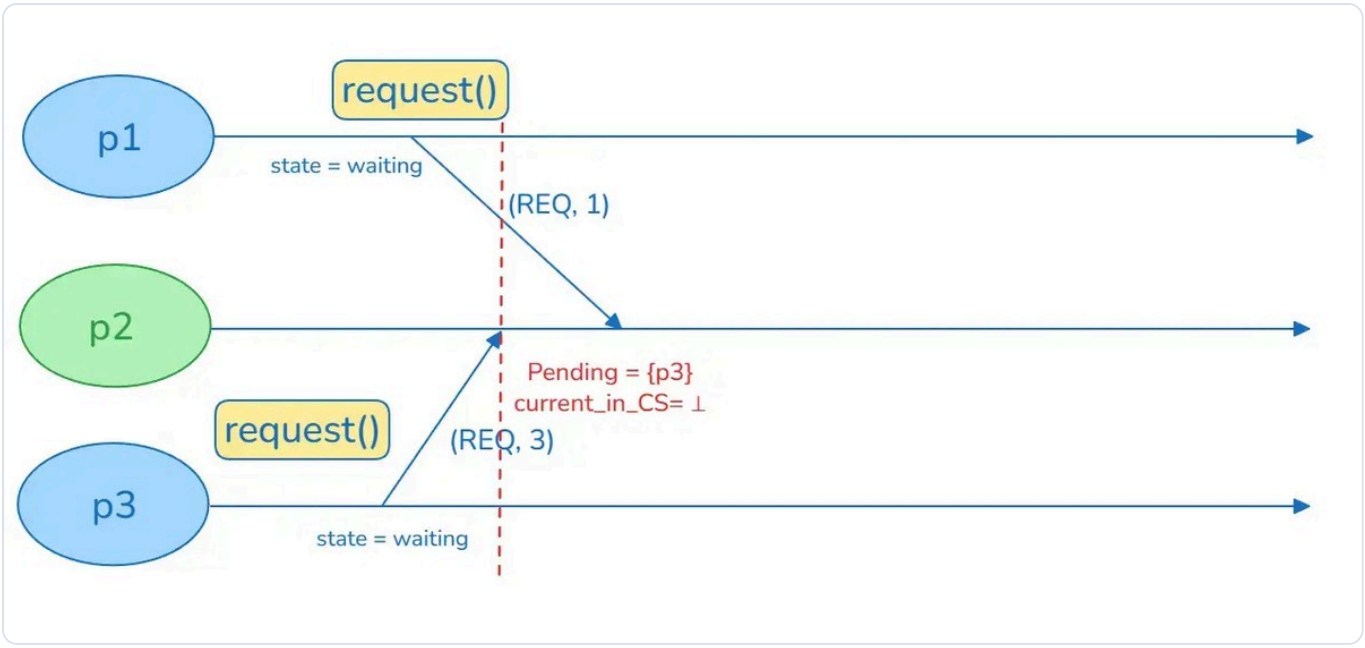
- `Init` **(Initialization):**
  - `pending = ∅`: Initializes a queue or empty pool to collect incoming requests from various nodes.
  - `current_in_CS = ⊥`: A variable that keeps track of which process is currently in the Critical Section (initially empty/null, denoted by ⊥).
- `upon event pp2pDeliver(REQ, j) from pj` **(When the coordinator receives a request):**
  - `pending = pending ∪ {p_j}`: Takes the requesting process p_j and adds it to the pool of pending requests.
- **The** `when pending ≠ ∅ and current_in_CS = ⊥` **control block (The scheduling logic):**
  - This check is executed continuously (or triggered on an event-by-event basis): if there are pending requests (`pending` is not empty) **and** the resource is currently free (`current_in_CS = ⊥`), the coordinator performs three actions:
    1. `candidate = select_process(pending)`: Selects a process from the waiting list (e.g., in FIFO order).
    2. `pending = pending \ candidate`: Removes the selected process from the waiting list.
    3. `current_in_CS = candidate`: Assigns the resource to the candidate.
    4. `trigger pp2pSend(GRANT_CS) to candidate`: Sends the `GRANT_CS` authorization message directly to that process.
- `upon event pp2pDeliver(REL, j) from pj` **(When the coordinator receives the release):**

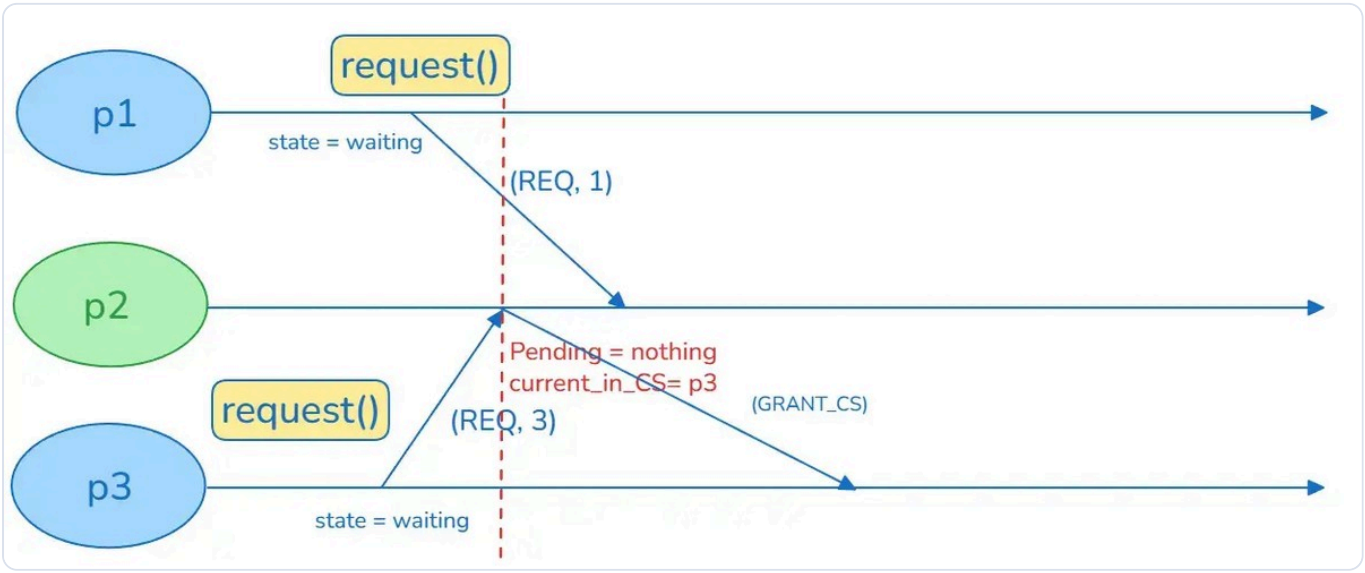
- `if current_in_CS = j`: Verifies that the process that was using the resource is actually releasing it.
- `current_in_CS = ⊥`: Frees the resource by setting it to ⊥. (This immediately unlocks the `when` lock described above, allowing the coordinator to serve the next process in the queue.)

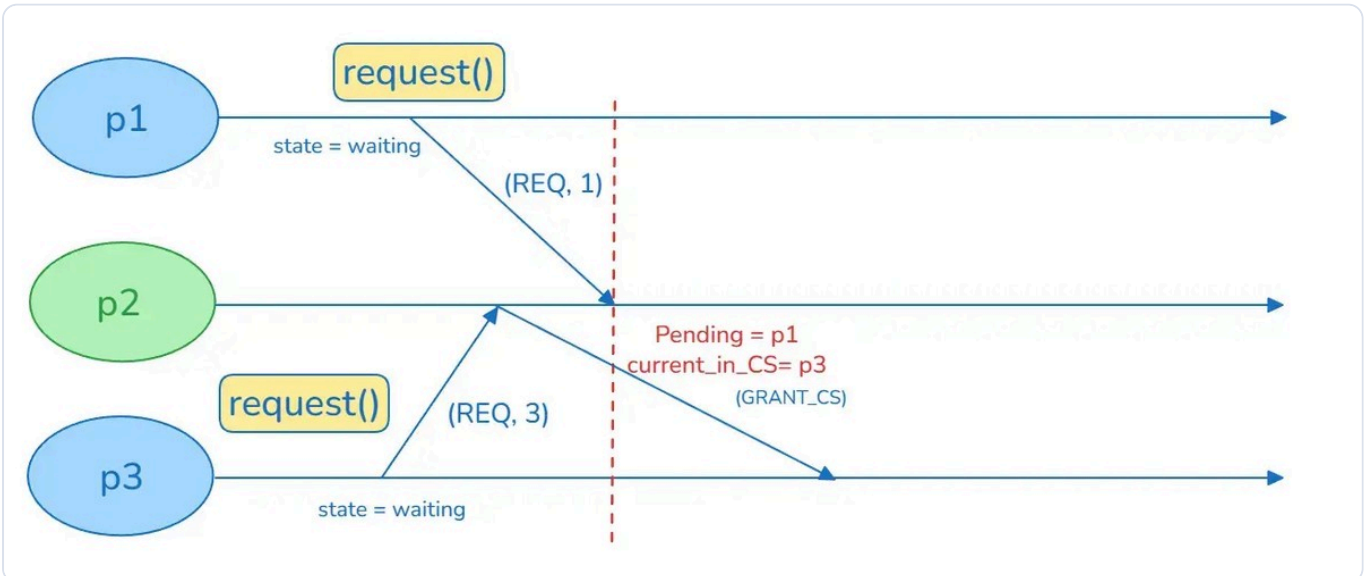
Example:

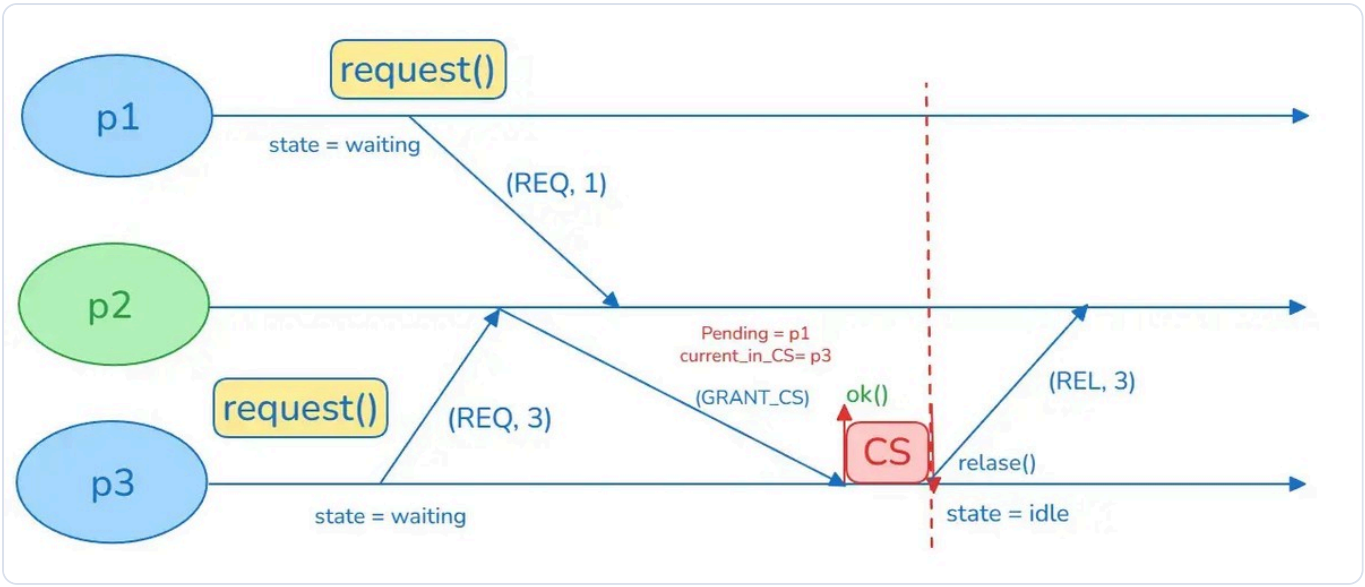


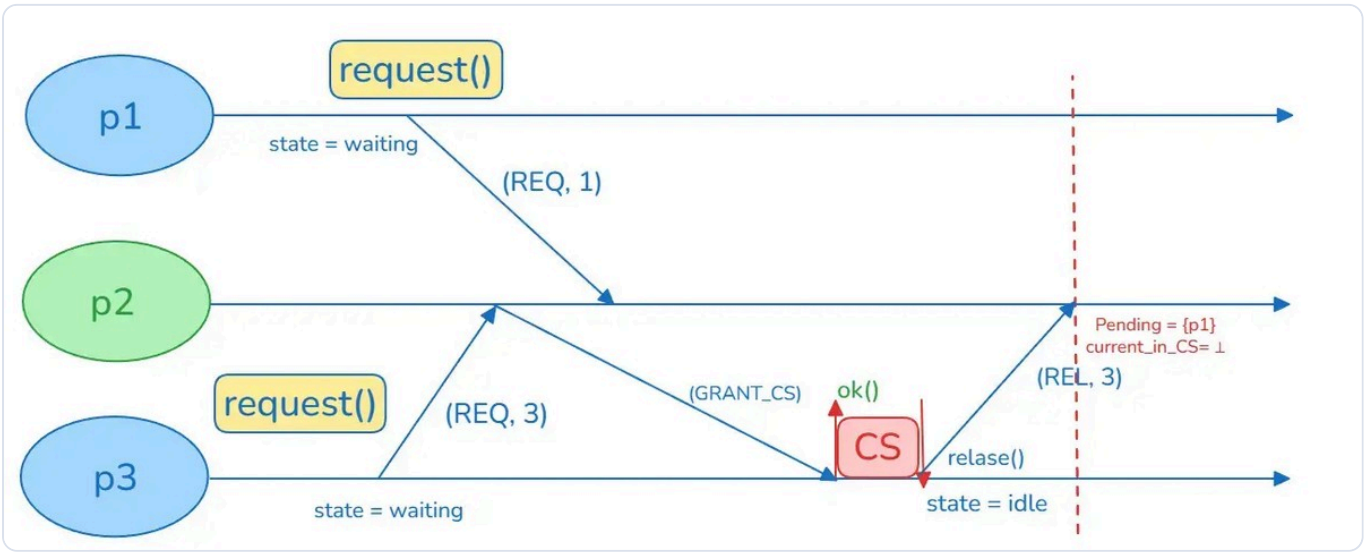


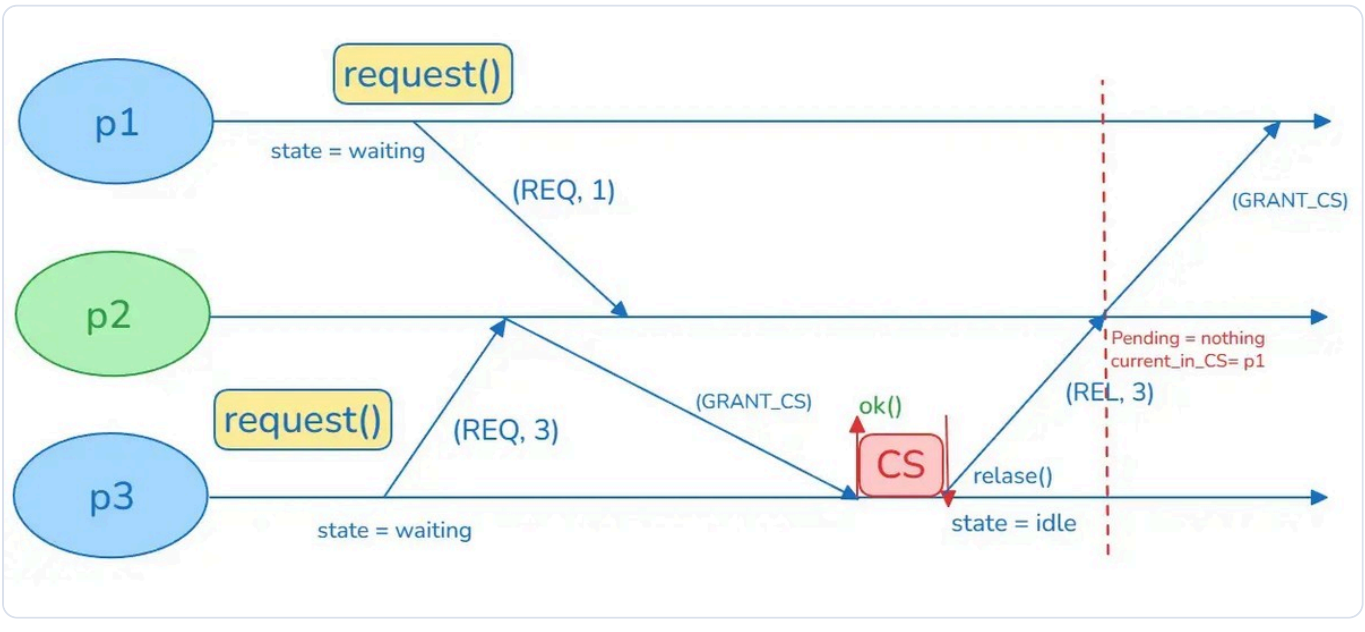

request()
p1
state = waiting
(REQ, 1)
p2
Pending = {p3}
current_in_CS= ⊥
request()
(REQ, 3)
p3
state = waiting

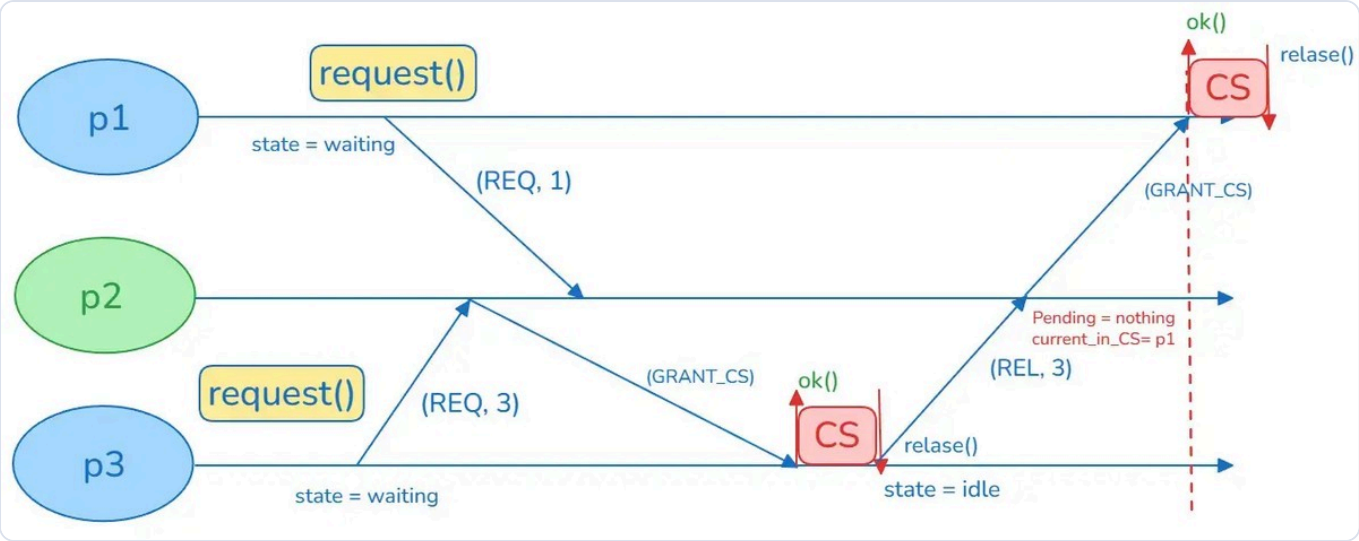

request()
p1
state = waiting
(REQ, 1)
p2
Pending = nothing
current_in_CS= p3
(GRANT_CS)
request()
(REQ, 3)
p3
state = waiting


request()
p1
state = waiting
(REQ, 1)
p2
Pending = p1
current_in_CS= p3
(GRANT_CS)
request()
(REQ, 3)
p3
state = waiting


p1
request()
state = waiting
(REQ, 1)
p2
Pending = p1
current_in_CS= p3
request()
(REQ, 3)
(GRANT_CS)
ok()
(REL, 3)
CS
p3
relase()
state = waiting
state = idle


p1
request()
state = waiting
(REQ, 1)
p2
Pending = {p1}
current_in_CS= ⊥
request()
(REQ, 3)
(GRANT_CS)
ok()
(REL, 3)
CS
p3
relase()
state = waiting
state = idle


p1
request()
state = waiting
(REQ, 1)
(GRANT_CS)
p2
Pending = nothing
current_in_CS= p1
request()
(REQ, 3)
(GRANT_CS)
ok()
(REL, 3)
CS
p3
relase()
state = waiting
state = idle



Pro/Cons

- **Performance**: Entering the CS requires exactly **2 messages** (1 `REQ` and 1 `GRANT`), equivalent to a single RTT (Round Trip Time). Releasing the resource requires **1** `REL` message.
- **Pros**: Extremely lightweight and easy to implement protocol.
- **Cons**:
- Scalability issues if traffic increases on the coordinator.
- **Single Point of Failure (SPOF)**: If the coordinator crashes, the entire mutual exclusion system crashes.

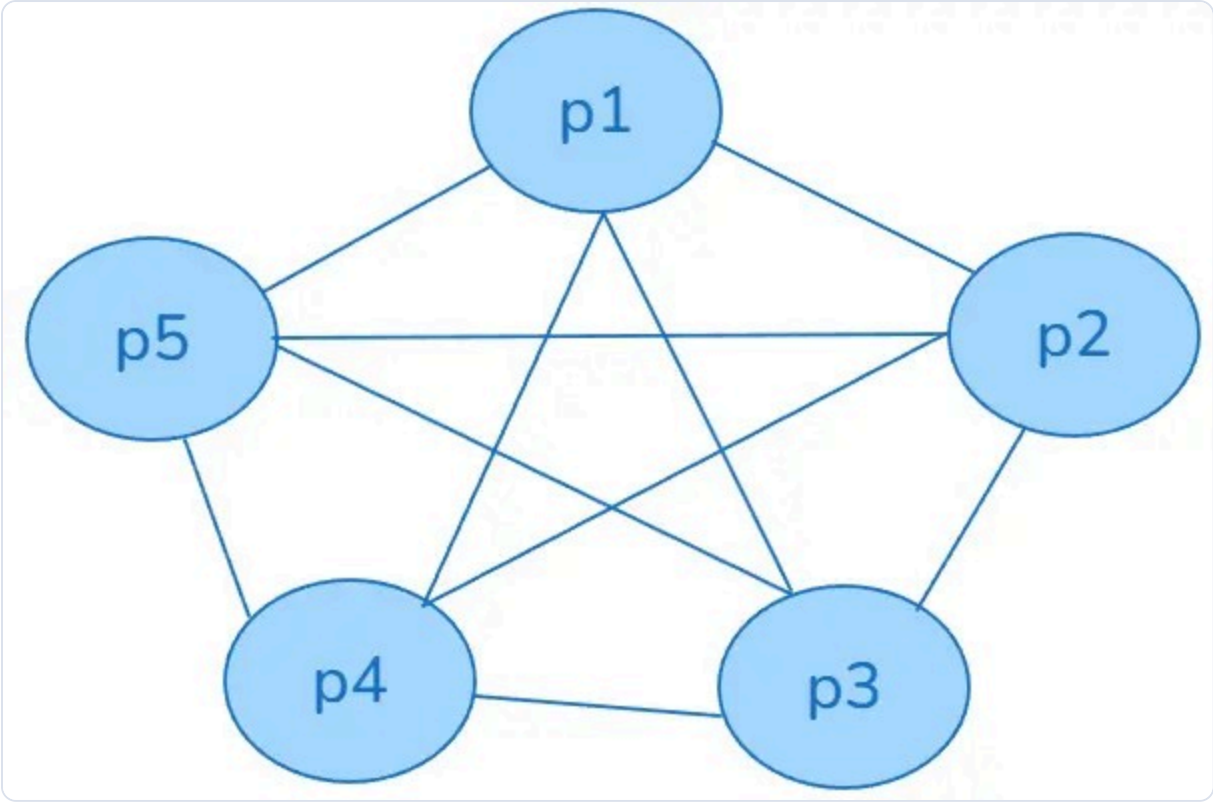
Token-based Algorithm on Logical Ring

The processes are organized in a logical ring topology (overlay network) using existing point-to-point communication channels. A single special message called a Token continuously circulates along the ring: only those who possess it are authorized to enter the Critical Section.

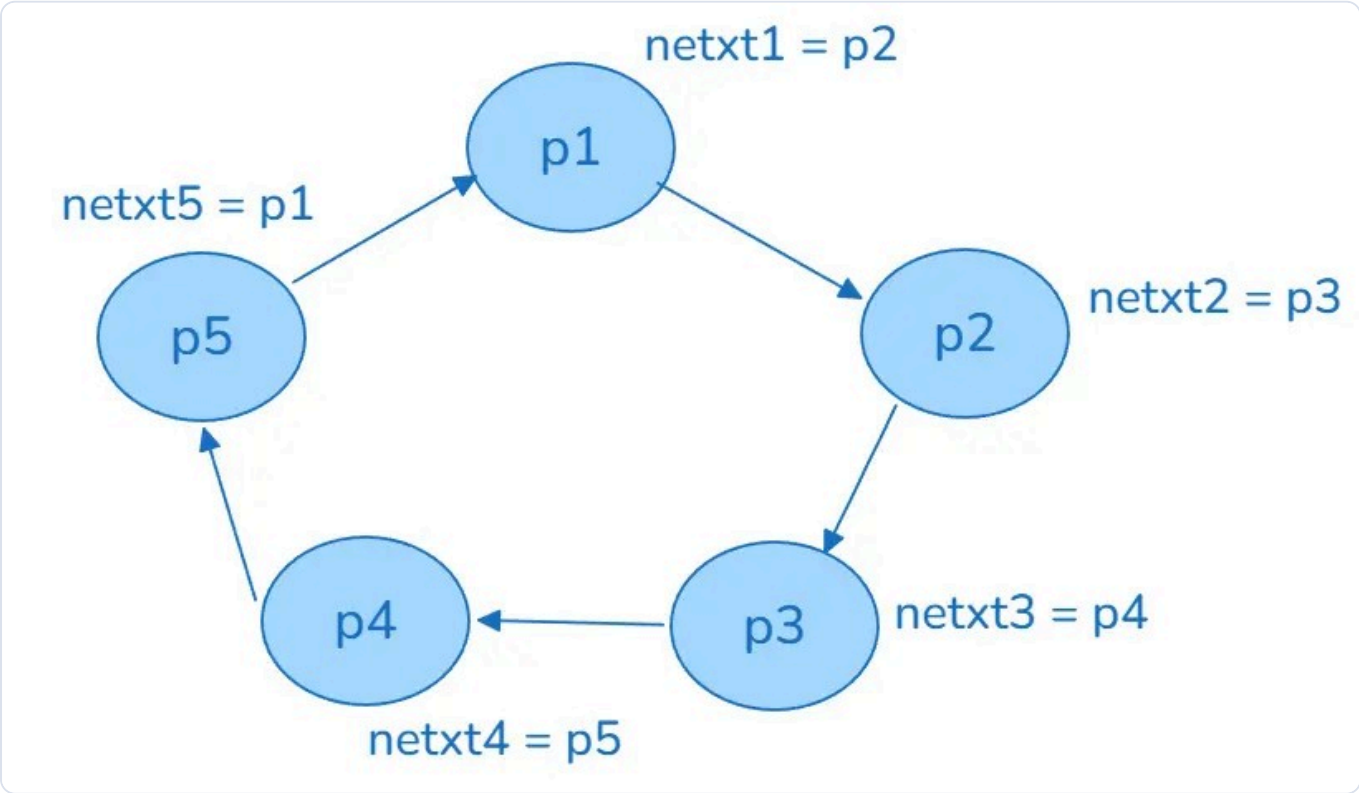
How it works

1. **Ring construction**: Each process p_i stores the name of the next process in a local variable (`next = p_{(i+1) \bmod N}`).
2. **Initialization**: A single token is created (usually generated by an initial node like p_0) and inserted into the ring.
3. **Request handling**:

- When a process wants to access the CS, it enters the `waiting` state and **waits to receive the token**.
- When the token arrives, if the process was waiting, it enters the CS (notifying `ok()`). When it is finished (`release()`), it passes the token to the next node (`next`).
- If the process receives the token but is *not* interested in the CS, it simply **immediately** forwards it to the next process in the ring.

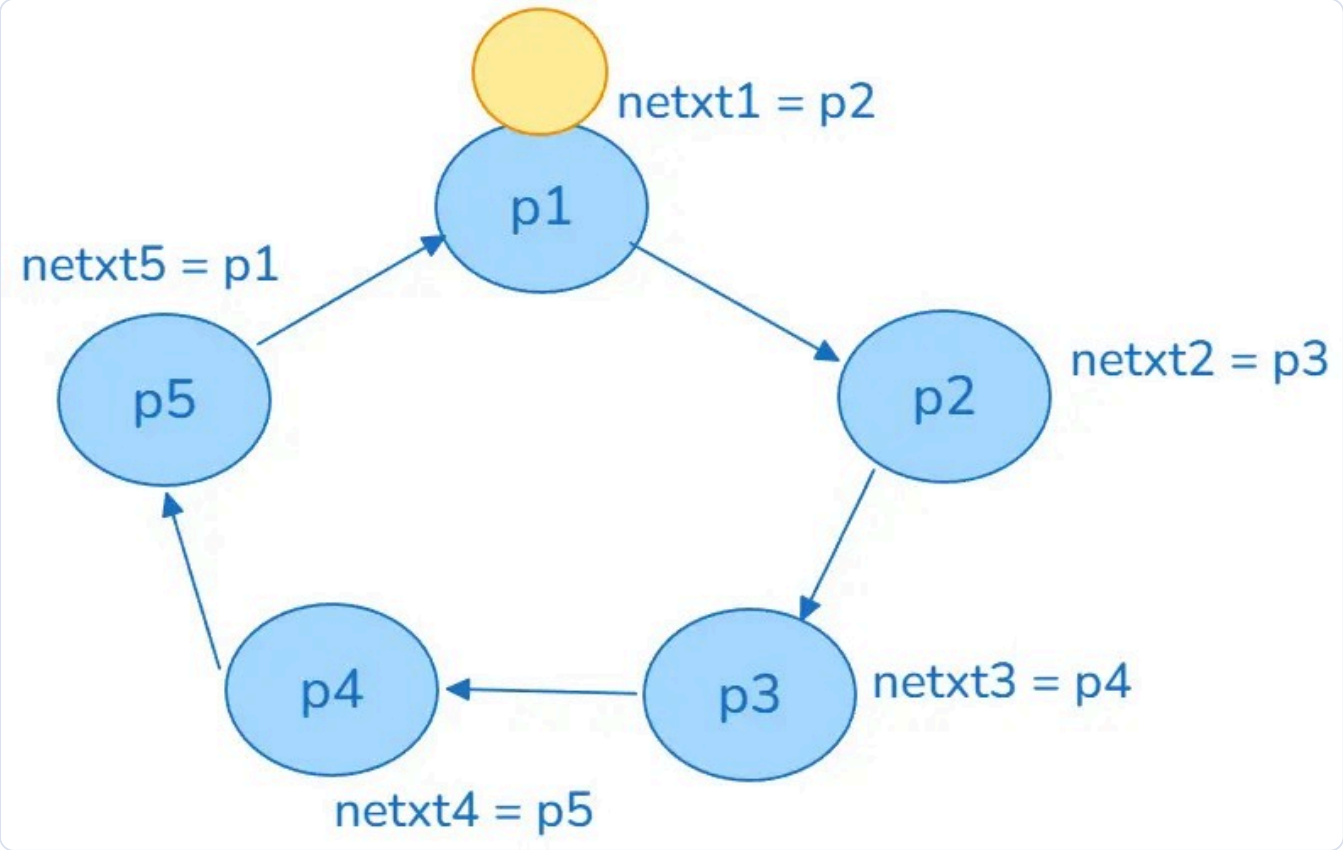


1. We construct an overlay (i.e., a logical network) as a ring exploiting existing point-to-point communication channels. The ring is obtained by:
   - Storing in a local variable the name of the next process in the ring;
   - Allowing communication only with the next node.

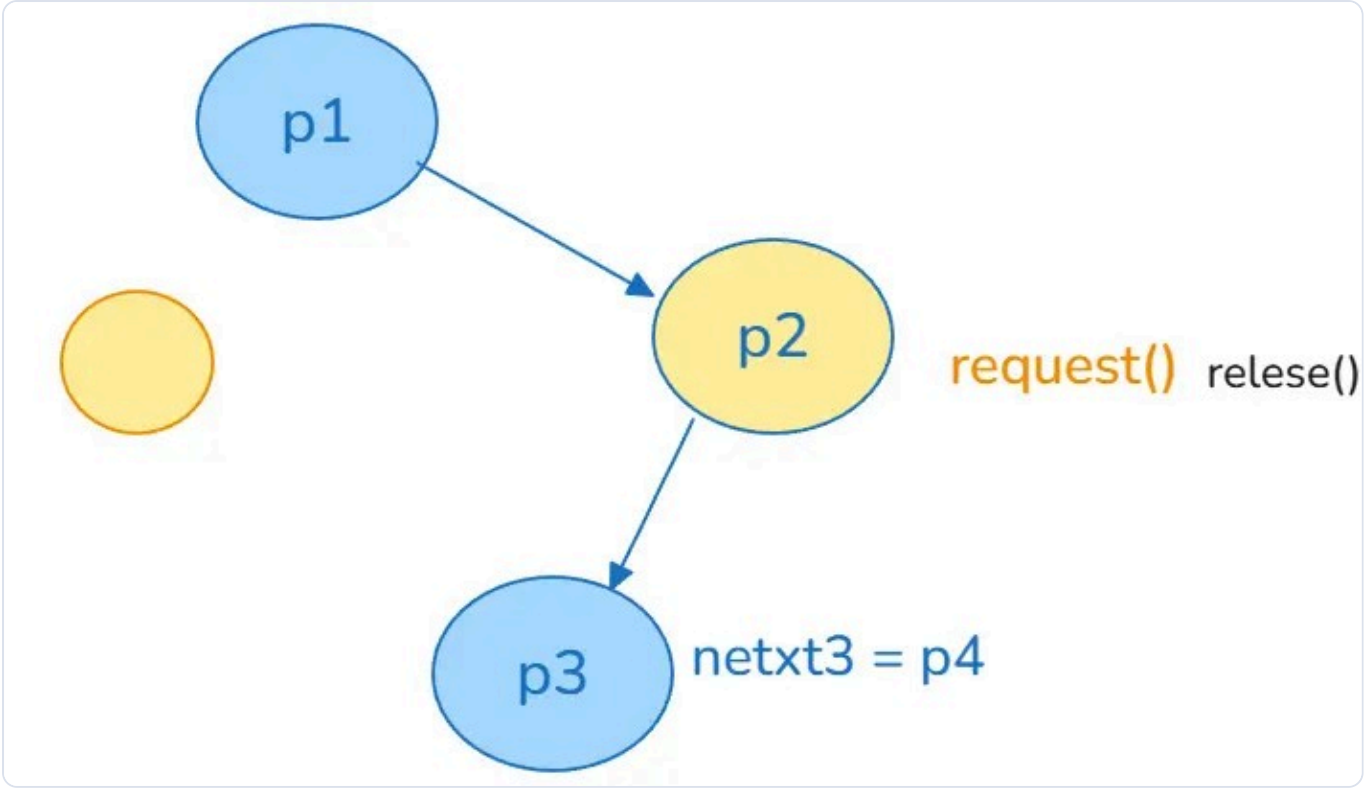


Init:
state := idle;
next := $p_{(i+1) \bmod N}$;

2. A token is created and propagated in the ring during the initialization phase (i.e., it is assigned to a process of the system). WARNING: The token must be unique to guarantee mutual exclusion. Only one process (selected through a deterministic function) can create the token during the init.



Init
state := idle
next := $p_{(i+1) \bmod n}$
if self = p_0 **then**
trigger pp2pSend(TOKEN) **to next**




```
upon event request() do
  state := waiting;
upon event pp2pDeliver(TOKEN) do
  if state = waiting then
    state := CS;
    trigger ok();
  else
    trigger pp2pSend(TOKEN) to next; ;
upon event release() do
  state := idle;
  trigger pp2pSend(TOKEN) to next;
```

So we obtain

```
Init:
  state := idle;
  next := p_{(i+1) mod N};
  if self = p_0 then
    trigger pp2pSend(TOKEN) to next;;
upon event request() do
  state := waiting;
upon event pp2pDeliver(TOKEN) do
  if state = waiting then
    state := CS;
    trigger ok();
  else
    trigger pp2pSend(TOKEN) to next;;
upon event release() do
  state := idle;
  trigger pp2pSend(TOKEN) to next;;
```

- **Main disadvantage**: The token continues to cycle incessantly, consuming network resources and bandwidth even when no process is interested in accessing the CS.
- **Latency**: The waiting time (delay) varies from 0 (the token arrives immediately) to N messages if the token has just been passed and must complete the entire loop before becoming available again.

Quorum-based Algorithm – Maekawa's voting algorithm

Instead of asking permission from *all* nodes (as in Lamport) or from a single coordinator, a process entering the CS must get acknowledgement (vote) only from a **specific subset of processes**, called the **Quorum or Voting Set (V_i)**.

The Properties of Voting Sets (V_i)

To ensure mutual exclusion and correct operation, voting sets must adhere to strict geometric/mathematical rules:

1. $p_i \in V_i$ (each process is part of its own voting set).
2. The *intersection* between any two sets is never empty: $V_i \cap V_j \neq \emptyset$. This ensures that there is always at least one "arbiter node" in common between two processes claiming to enter the CS, preventing conflicts.
3. All sets have the same size $|V_i| = K$ (fair load principle).
4. Each process p_j belongs to the same number of sets M (fair responsibility principle).

Init:
  state := released;
  voted := false;
  V_i := get_voting_set(i);
  replies := $\emptyset$;
  pending := $\emptyset$;

upon event request() **do**
  state := wanted;
  for each $p_j \in (V_i \setminus \{p_i\})$ **do**
    trigger pp2pSend(REQ, i) to p_j;

upon event pp2pDeliver(REQ, j) **do**
  if state = held $\vee$ voted = true **then**
    pending := pending $\cup \{p_j\}$;
  else
    trigger pp2pSend(ACK, i) to p_j;
    voted := true;

upon event pp2pDeliver(ACK, j) **do**
  replies := replies $\cup \{p_j\}$;

when |replies| = $|V_i| - 1$ **do**
  state := held;

upon event release() **do**
  state := released;
  replies := $\emptyset$;
  for each $p_j \in (V_i \setminus \{p_i\})$ **do**
    trigger pp2pSend(REL, i) to p_j;

upon event pp2pDeliver(REL, j) **do**
  if |pending| > 0 **then**
    candidate := select_next(pending);
    pending := pending $\setminus$ {candidate};
    trigger pp2pSend(ACK, i) to candidate;
    voted := true;
  else
    voted := false;

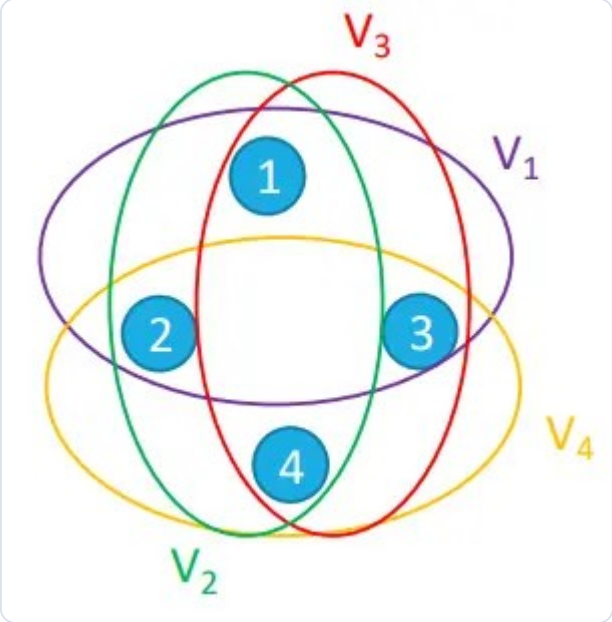
Maekawa showed that to minimize the number of messages and balance the load:

$$K \approx \sqrt{N} \quad \text{and} \quad M = K$$

A rule of thumb for constructing the groups (V_i) is to arrange the N processes in a matrix $\sqrt{N} \times \sqrt{N}$ and define the voting set V_i as **the union of the row and column** in which the process p_i is located.

Example: Let us consider a system composed by N = 4 processes. $M = K \approx \sqrt{4} = 2$
Let's place processes in the matrix and compute V_i

Process id	V
1	p_1, p_2, p_3
2	p_1, p_2, p_4
3	p_1, p_3, p_4
4	p_2, p_3, p_4



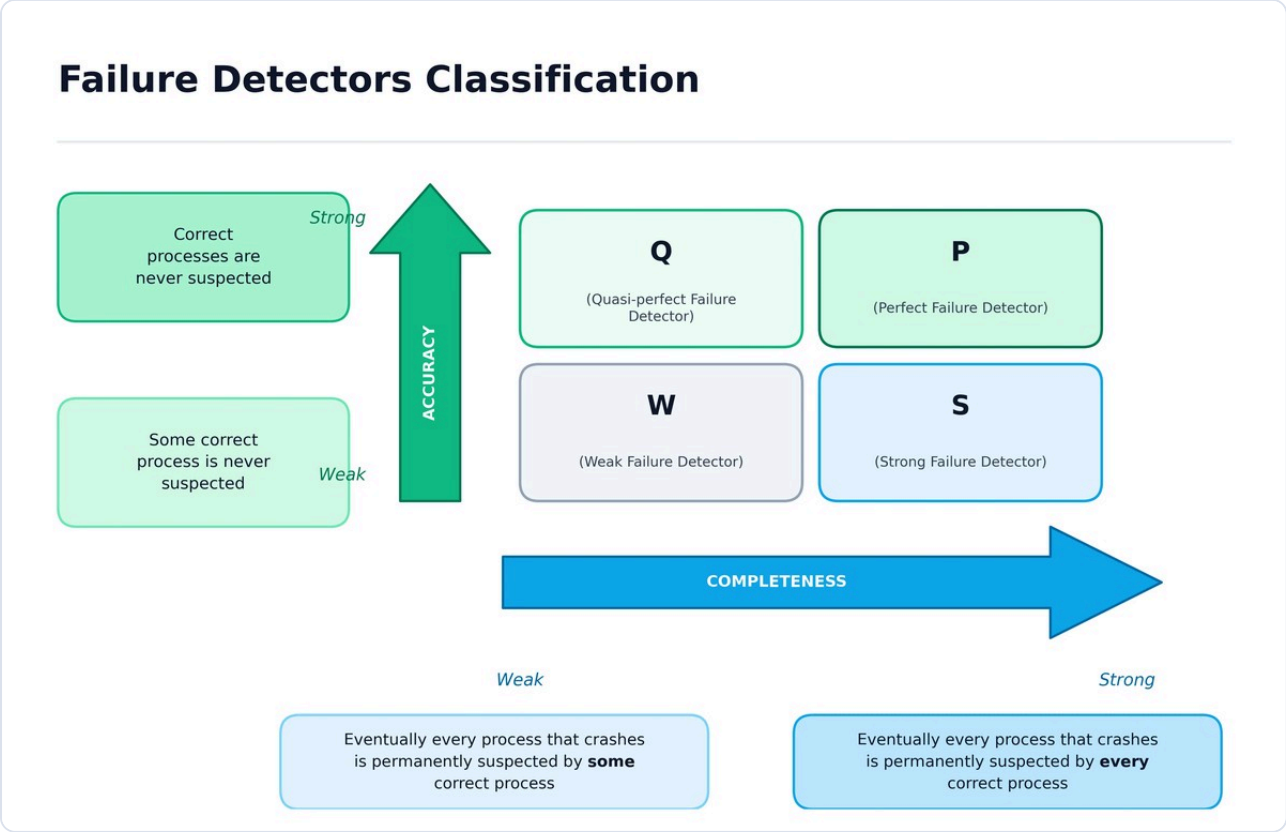
> **WARNING**
>
> Maekawa's algorithm is not deadlock-free! If two processes (p_1 and p_4) simultaneously request the resource and block each other by obtaining partial votes from the common nodes without being able to complete their quorum, the system becomes stuck in a structural deadlock.

Failure Detection and Leader Election

Failure detection

A **Failure Detector** is an oracle providing information about the failure state of a process. Failure detector is generally described by two properties:

- **Accuracy**: the ability to avoid mistakes in the detection;
- **Completeness**: informally is ability to detect all failures.



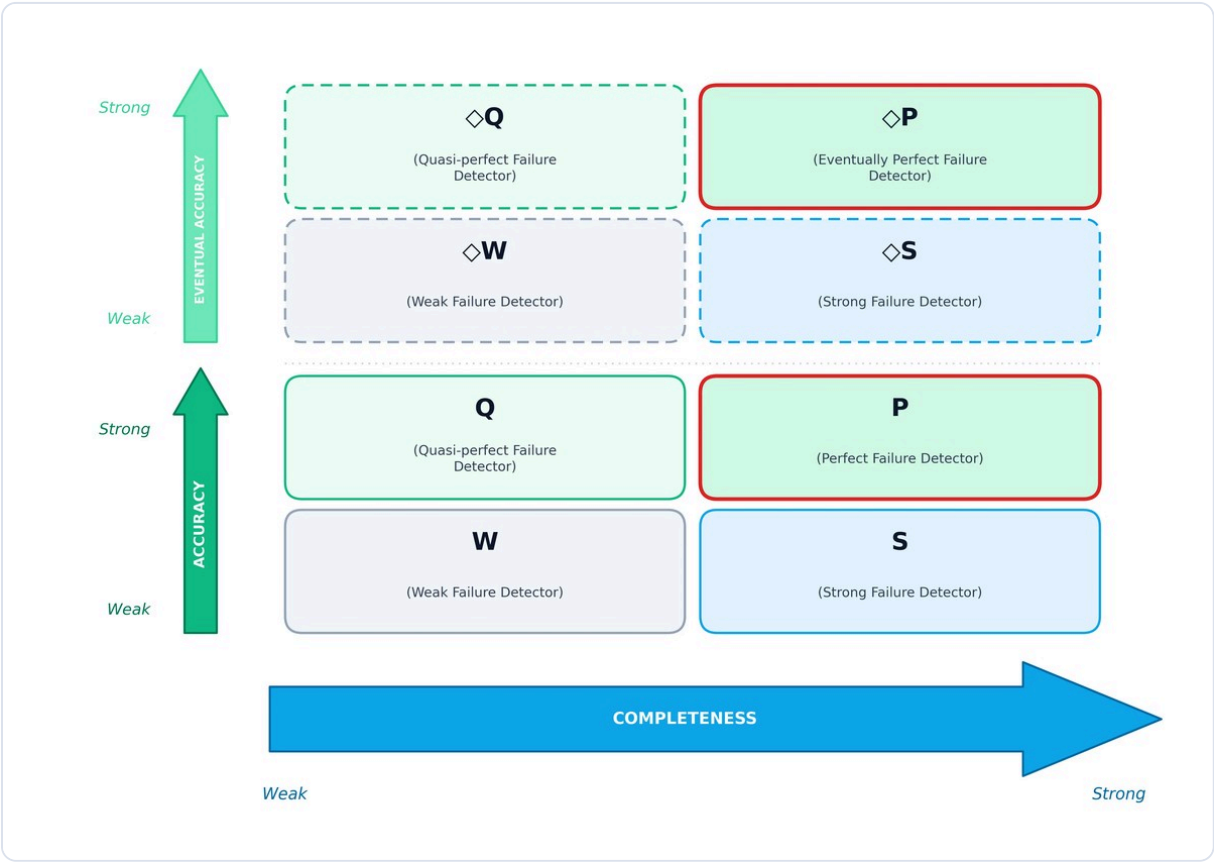
Failure detectors classification

Combining the different levels of Accuracy and Completeness, detectors are divided into classes:

- **Completeness**:
- *Strong*: Sooner or later, every process that crashes is permanently suspected by *all* healthy processes.
- *Weak*: Sooner or later, every process that crashes is permanently suspected by *at least* one healthy process.
- **Accuracy**:
- *Strong*: Healthy processes are **never** suspected.
- *Weak*: There is at least one healthy process that is never suspected.
- **Eventual Accuracy (◇)**: Accuracy is not required immediately, but applies "after a certain time" (*eventually*):
- **Eventual Strong Accuracy** ($\diamond S$): There is a time after which no correct process is suspected by another correct process.

THE CLASS MATRIX:

- **P (Perfect Failure Detector)**: Strong completeness + Strong accuracy (requires a synchronous system).
- **S (Strong Failure Detector)**: Strong completeness + Weak accuracy.
- **Q (Quasi-perfect Failure Detector)**: Weak completeness + Strong accuracy.
- **W (Weak Failure Detector)**: Weak completeness + Weak accuracy.
- The diamond versions ($\diamond P, \diamond S, \diamond Q, \diamond W$) indicate that the accuracy properties hold only partially (typical of partial synchrony).



Perfect failure detector

They operate in synchronous systems with crash-stop failures. Using its own clock and the bound of the synchrony model, a process can infer if another process is crashed.

- *PFD1 (Strong Completeness)*: Every process that crashes is permanently detected by every correct process.
- *PFD2 (Strong Accuracy)*: If one process p is detected by any process, then p has crashed.

Algorithm 2.5: Exclude on Timeout
Implements: PerfectFailureDetector, instance $\mathcal{P}$.
Uses: PerfectPointToPointLinks, instance pl.

upon event $\langle \mathcal{P}, \text{Init} \rangle$ **do**
  $\text{alive} := \Pi$;
  $\text{detected} := \emptyset$;
  $\text{starttimer}(\Delta)$;

upon event $\langle \text{Timeout} \rangle$ **do**
  forall $p \in \Pi$ **do**
    if $(p \notin \text{alive}) \land (p \notin \text{detected})$ **then**
      $\text{detected} := \text{detected} \cup \{p\}$;
      trigger $\langle \mathcal{P}, \text{Crash} \mid p \rangle$;
    trigger $\langle pl, \text{Send} \mid p, [\text{HEARTBEATREQUEST}] \rangle$;
  $\text{alive} := \emptyset$;
  $\text{starttimer}(\Delta)$;

upon event $\langle pl, \text{Deliver} \mid q, [\text{HEARTBEATREQUEST}] \rangle$ **do**
  trigger $\langle pl, \text{Send} \mid q, [\text{HEARTBEATREPLY}] \rangle$;

upon event $\langle pl, \text{Deliver} \mid p, [\text{HEARTBEATREPLY}] \rangle$ **do**
  $\text{alive} := \text{alive} \cup \{p\}$;

This algorithm implements a **Perfect Failure Detector** ($\mathcal{P}$) using point-to-point communication channels (pl). Its purpose is to monitor processes in the system and detect crashes with absolute accuracy and completeness.

- **Initialization (`Init`)**:
  - `alive := Π`: Initially assumes all processes in the system are alive.
  - `detected := ∅`: No process is marked as crashed yet.
  - `starttimer(Δ)`: Starts a periodic heartbeat timer with a interval Δ.
- **Timeout Handling (`Timeout`)**:
  - The algorithm periodically checks every process $p \in \Pi$.
  - If a process p is neither in the `alive` set nor already marked in `detected`, it means p failed to send a heartbeat within the last interval. The detector marks it as crashed (`detected := detected ∪ {p}`), triggers a crash indication ($\langle \mathcal{P}, \text{Crash} \mid p \rangle$), and sends a heartbeat request to check its status.
  - It then resets `alive := ∅` for the next round and restarts the timer.
- **Responding to Heartbeats (`HEARTBEATREQUEST` / `HEARTBEATREPLY`)**:
  - When a process receives a `HEARTBEATREQUEST`, it immediately replies with a `HEARTBEATREPLY`.
  - When a process receives a `HEARTBEATREPLY` from p, it adds p back to the `alive` set (`alive := alive ∪ {p}`), confirming that p is still operational.

Eventually perfect failure detectors

It operates under conditions of **partial synchrony**. Before an unknown time t, the system behaves asynchronously (the detector can make errors and incorrectly suspect live nodes). After time t, the system becomes synchronous and the suspicions become accurate.

- *EPFD1 (Strong Completeness)*: Every crashed process is permanently suspected.
- *EPFD2 (Eventual Strong Accuracy)*: After time t, no valid process is suspected. To avoid false alarms due to the variability of the asynchronous network, the algorithm adopts an incremental timeout: each time a node is incorrectly suspected of being alive due to a timeout that is too short, the delay value is increased ($\text{delay} = \text{delay} + \Delta$), ensuring convergence to accuracy when the network stabilizes.

Algorithm 2.7: Increasing Timeout
Implements: EventuallyPerfectFailureDetector, instance $\diamond\mathcal{P}$.
Uses: PerfectPointToPointLinks, instance pl.

upon event $\langle \diamond\mathcal{P}, \text{Init} \rangle$ **do**
  alive $:= \Pi$;
  suspected $:= \emptyset$;
  delay $:= \Delta$;
  starttimer(delay);

upon event $\langle \text{Timeout} \rangle$ **do**
  if alive $\cap$ suspected $\neq \emptyset$ then
    delay $:=$ delay $+ \Delta$;
  forall $p \in \Pi$ do
    if $(p \notin \text{alive}) \wedge (p \notin \text{suspected})$ then
      suspected $:=$ suspected $\cup \{p\}$;
      trigger $\langle \diamond\mathcal{P}, \text{Suspect} \mid p \rangle$;
    else if $(p \in \text{alive}) \wedge (p \in \text{suspected})$ then
      suspected $:=$ suspected $\setminus \{p\}$;
      trigger $\langle \diamond\mathcal{P}, \text{Restore} \mid p \rangle$;
    trigger $\langle pl, \text{Send} \mid p, [\text{HEARTBEATREQUEST}] \rangle$;
  alive $:= \emptyset$;
  starttimer(delay);

upon event $\langle pl, \text{Deliver} \mid q, [\text{HEARTBEATREQUEST}] \rangle$ do
  trigger $\langle pl, \text{Send} \mid q, [\text{HEARTBEATREPLY}] \rangle$;

upon event $\langle pl, \text{Deliver} \mid p, [\text{HEARTBEATREPLY}] \rangle$ **do**
  alive $:=$ alive $\cup \{p\}$;

This algorithm implements an **Eventually Perfect Failure Detector ($\diamond\mathcal{P}$)**. Unlike a *perfect* failure detector (which makes no mistakes), an eventually perfect failure detector can make temporary mistakes during periods of asynchrony (such as suspecting a process that is still alive), but guarantees that it eventually becomes accurate once the system reaches periods of synchrony.

1. **Initialization** (`Init`)

- `alive := Π` : Initially, the algorithm assumes that all processes in the system set (Π) are alive.
- `suspected := ∅` : No process is initially suspected of having crashed.
- `delay := Δ` : Sets an initial timeout duration.
- `starttimer(delay)` : Starts the timer using the current delay value.

2. **Timeout Handling** (`Timeout`) This is the core mechanism that makes the algorithm adaptive:

- **Checking for past mistakes (`alive \cap suspected \neq \emptyset`)**: If a process that was previously suspected of crashing successfully replies during the current interval (meaning it is actually in the `alive` set), it implies the timeout interval was too short for network conditions. Consequently, **the delay is increased** (`delay := delay + Δ`).
- **Updating Suspicions (`forall p \in \Pi`)**:
  - If a process p is neither in `alive` nor already in `suspected` , it is added to the `suspected` set, and a `Suspect` indication is triggered (`trigger` < $\diamond\mathcal{P}, \text{Suspect} \mid p >$).
  - If a process was previously suspected but has now responded (meaning it is back in `alive`), it is removed from the `suspected` set, and a `Restore` indication is triggered (`trigger` $< \diamond\mathcal{P}, \text{Restore} \mid p >$).
- **Broadcasting Heartbeats (`HEARTBEATREQUEST`)**: The process sends a new heartbeat request to all nodes, resets the `alive` set to empty (`alive := ∅`), and restarts the timer with the updated (or maintained) `delay` .

2. **Message Delivery** (`Deliver`)

- When a node receives a `HEARTBEATREQUEST` , it immediately responds with a `HEARTBEATREPLY` .
- When a monitoring node receives a `HEARTBEATREPLY` from p, it adds p back into the `alive` set (`alive := alive ∪ {p}`).

Correctness

1. **Strong Completeness**
   - **What this means:** _"If a process crashes, it stops sending messages. Consequently, the process will be suspected by every healthy process, and no process will change its mind again."
   - **Explanation:** This property is **always true** from the beginning (it doesn't need the word "eventually"). If a node truly dies, it will no longer respond to heartbeat checks, so sooner or later all healthy nodes will notice its absence and permanently add it to the suspect list.
2. **Eventual Strong Accuracy**
   - **What this means:** _"After time T, the system becomes synchronous (i.e., messages between healthy processes arrive within a guaranteed maximum time). If p was suspected in error by q, then q will revise its suspicion."
   - **Why is it "eventual"?** This is where the temporal aspect of $\diamond\mathcal{P}$ comes into play.
     - *Before time* T (when the network is asynchronous and unpredictable), the system can make mistakes: a live node can be mistakenly suspected simply because the network delayed a message or because the timeout was set too short.

- *After time T, the magic of **eventually** comes true*: the network stabilizes and becomes synchronous. Messages travel within a certain time limit. If q had wrongly suspected a live node p, as soon as it receives a real message from p, it will realize its error and **revise its suspicion**.

Leader election

In distributed systems, continuously monitoring the state of all nodes can be cumbersome. Often, to coordinate actions (e.g., in consensus or transactional protocols), it's enough to know that **at least one process is alive** and acting as the coordinator (*leader*).

- **LE1 (Eventual Detection)**: Either there are no valid processes, or a valid process is eventually elected as the leader.
- **LE2 (Accuracy)**: If a process is the leader, all previously elected leaders have certainly failed. **Monarchical Implementation ("Monarchical Leader Election")**: Uses a Perfect Failure Detector ($\mathcal{P}$). Each node maintains a list of suspected processes and chooses as the leader the alive process with the highest identifier (or rank) among the unsuspected ones ($\text{maxrank}(\Pi \setminus \text{suspected})$).

Algorithm 2.6: Monarchical Leader Election
Implements: LeaderElection, instance le.
Uses: PerfectFailureDetector, instance $\mathcal{P}$.

upon event $\langle le, \text{Init} \rangle$ **do**
  suspected $:= \emptyset$;
  leader $:= \perp$;

upon event $\langle \mathcal{P}, \text{Crash} \mid p \rangle$ **do**
  suspected $:=$ suspected $\cup \{p\}$;

upon leader $\neq \text{maxrank}(\Pi \setminus \text{suspected})$ **do**
  leader $:= \text{maxrank}(\Pi \setminus \text{suspected})$;
  trigger $\langle le, \text{Leader} \mid \text{leader} \rangle$;

This algorithm implements a **Monarchical Leader Election** mechanism (`le`), which elects a unique leader among a set of processes Π using a Perfect Failure Detector ($\mathcal{P}$). The core idea is that processes are ranked (e.g., by their identifier or priority), and the correct process with the highest rank is always chosen as the leader.

1. **Initialization** (`Init`)

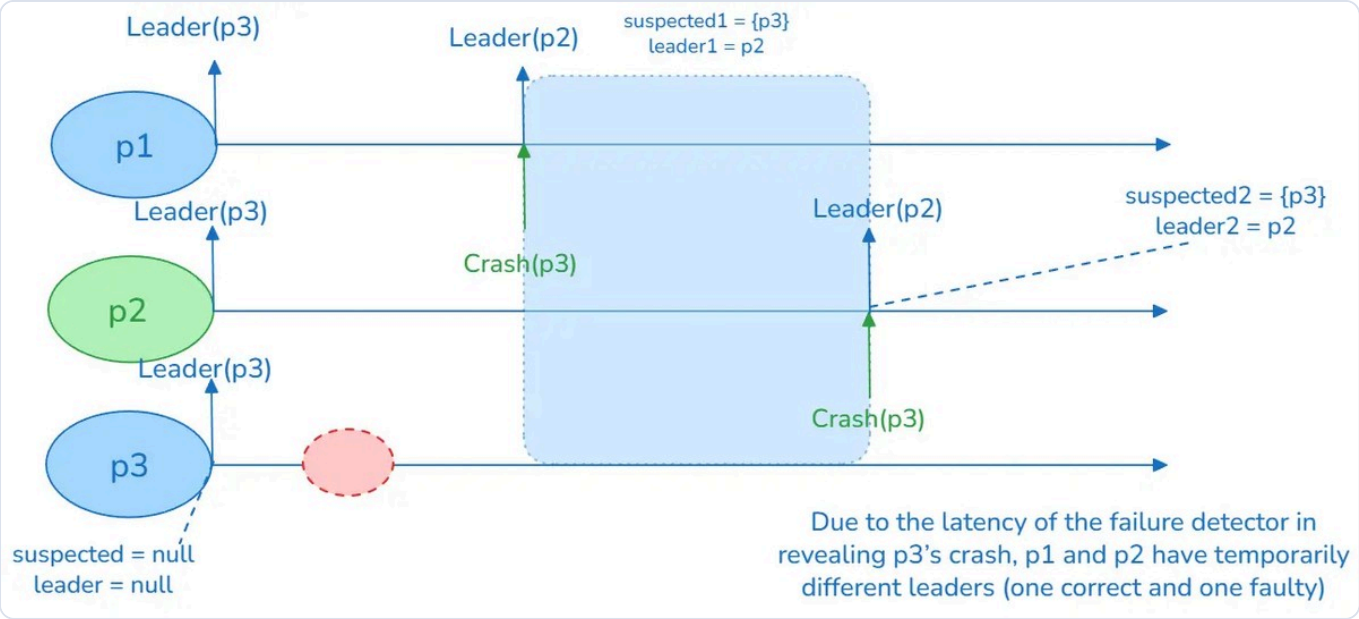
- `suspected` $:= \emptyset$: Initially, no processes are suspected of having crashed.
- `leader` $:= \perp$: The leader variable is undefined ($\perp$) at startup.

1. **Handling Process Crashes** $\langle \mathcal{P}, \text{Crash} \mid p \rangle$

- When the underlying Perfect Failure Detector ($\mathcal{P}$) detects that a process p has crashed, it triggers an event.
- The algorithm catches this event and adds p to the `suspected` set (`suspected := suspected \cup \{p\}`).

1. **Continuous Guard** (leader $\neq \text{maxrank}(\Pi \setminus \text{suspected})$)

- This is a reactive guard condition (`upon ... do`) that constantly monitors the set of active (non-suspected) processes: $\Pi \setminus \text{suspected}$.
- **Selecting the Leader**: It computes `maxrank(...)`, which identifies the process with the highest rank among the currently alive processes.
- **Updating and Notifying**: If the current `leader` differs from this newly calculated highest-rank process, the variable is updated (`leader := maxrank(...)`), and a notification is broadcasted via `trigger` $\langle le, \text{Leader} \mid \text{leader} \rangle$ to inform the system of the new leader.



Eventual leader election Ω

In asynchronous or partially synchronous systems, it is not possible to guarantee a single, error-free leader from the outset. Therefore, the oracle Ω (*Eventual Leader Detector*) is introduced:

- *ELD1 (Eventual Accuracy)*: There is a time after which all correct processes place their trust (*trust*) in the same correct process.
- *ELD2 (Eventual Agreement)*: There is a time after which no two correct processes choose different correct leaders.

Important note: Ω does not guarantee immediate stability; leaders can temporarily change before converging on the final election (*stabilization*). There always exists at least one correct process, otherwise Ω cannot be built. **Crash-Recovery Implementation**: When considering systems with crash-recovery failures (nodes can restart), epoch-based algorithms (generation/restart numbers) and fair-loss links are used to make nodes exchange heartbeats associated with the active epoch, ultimately electing the leader with the correct epoch and rank.

Algorithm 2.8: Monarchical Eventual Leader Detection
Implements: EventualLeaderDetector, instance Ω.
Uses: EventuallyPerfectFailureDetector, instance $\diamond\mathcal{P}$.

upon event $\langle \Omega, \text{Init} \rangle$ **do**
  $\text{suspected} := \emptyset$;
  $\text{leader} := \perp$;

upon event $\langle \diamond\mathcal{P}, \text{Suspect} \mid p \rangle$ **do**
  $\text{suspected} := \text{suspected} \cup \{p\}$;

upon event $\langle \diamond\mathcal{P}, \text{Restore} \mid p \rangle$ **do**
  $\text{suspected} := \text{suspected} \setminus \{p\}$;

upon $\text{leader} \neq \text{maxrank}(\Pi \setminus \text{suspected})$ **do**
  $\text{leader} := \text{maxrank}(\Pi \setminus \text{suspected})$;
  trigger $\langle \Omega, \text{Trust} \mid \text{leader} \rangle$;

1. **Initialization** (`Init`)

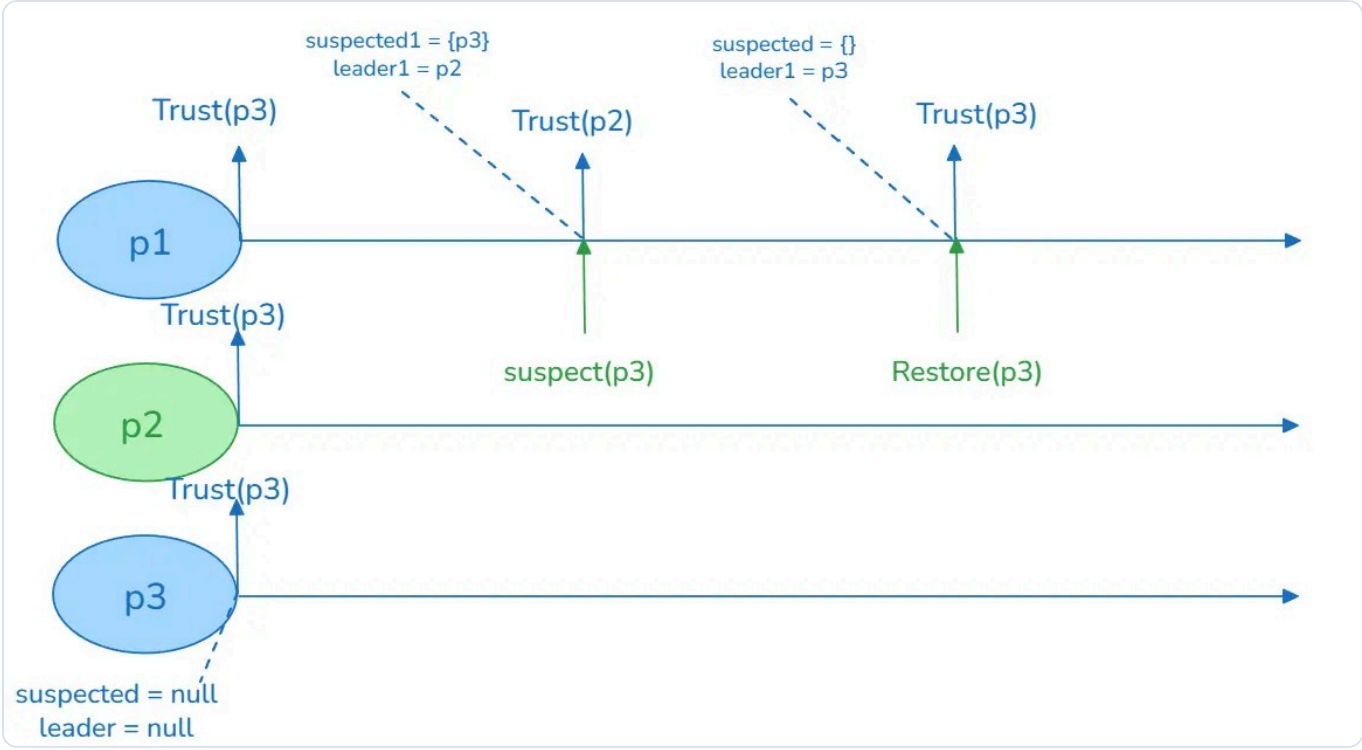
- `suspected := ∅` : Initially, no processes are suspected of having crashed.
- `leader := \bot` : The local leader variable is undefined ($\perp$) at startup.

2. **Handling Failure Detector Events** (`Suspect` / `Restore`)

- **`upon event`** $\langle \diamond\mathcal{P}, \text{Suspect} \mid p \rangle$: When the underlying Eventually Perfect Failure Detector ($\diamond\mathcal{P}$) suspects that a process p has crashed, the algorithm adds p to the `suspected` set (`suspected := suspected \cup \ {p\}`).
- **`upon event`** $\langle \diamond\mathcal{P}, \text{Restore} \mid p \rangle$: If the failure detector later realizes it made a mistake and restores p, the algorithm removes p from the `suspected` set (`suspected := suspected \setminus \{p\}`).

3. **Continuous Guard and Trust** ($\text{leader} \neq \text{maxrank}(\Pi \setminus \text{suspected})$)

- This reactive guard continuously monitors the set of currently trusted (non-suspected) processes: $\Pi \setminus \text{suspected}$.
- **Selecting the New Leader**: It computes `maxrank(...)` to find the process with the highest rank among the active processes.
- **Updating and Trusting**: If the calculated highest-rank process differs from the current `leader`, the local variable is updated, and a notification is sent out via `trigger` $\langle \Omega, \text{Trust} \mid \text{leader} \rangle$ to signal that the process now trusts this new leader.

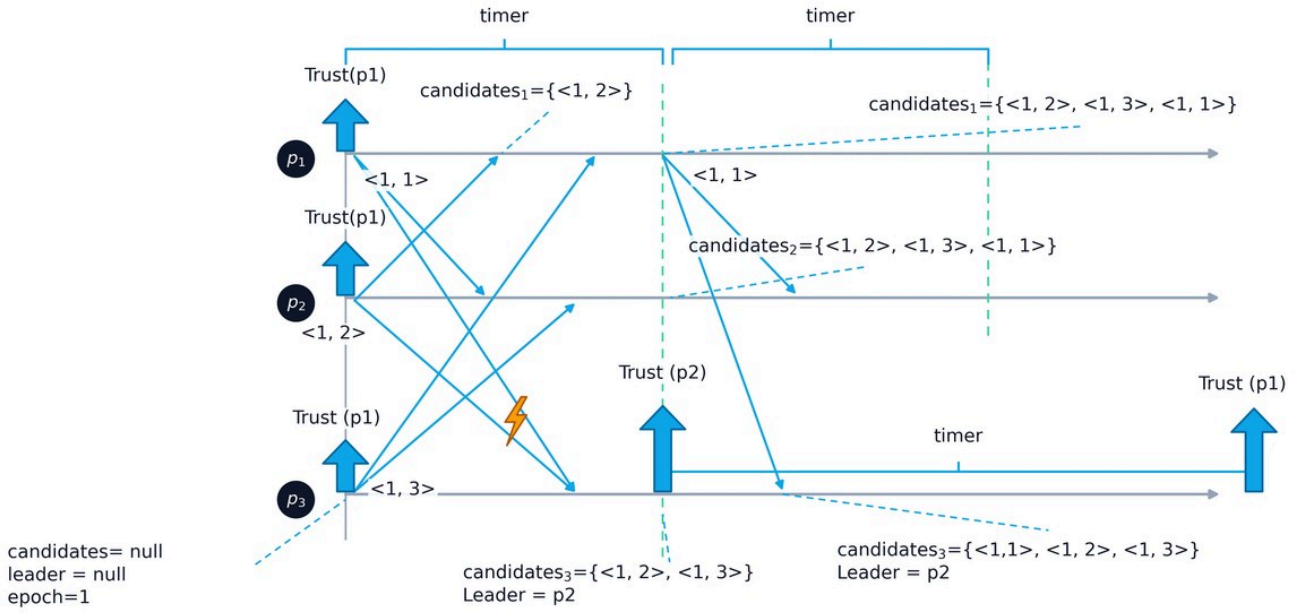

suspected1 = {p3}
leader1 = p2
suspected = {}
leader1 = p3
Trust(p3)
Trust(p2)
Trust(p3)
p1
Trust(p3)
suspect(p3)
Restore(p3)
p2
Trust(p3)
p3
suspected = null
leader = null

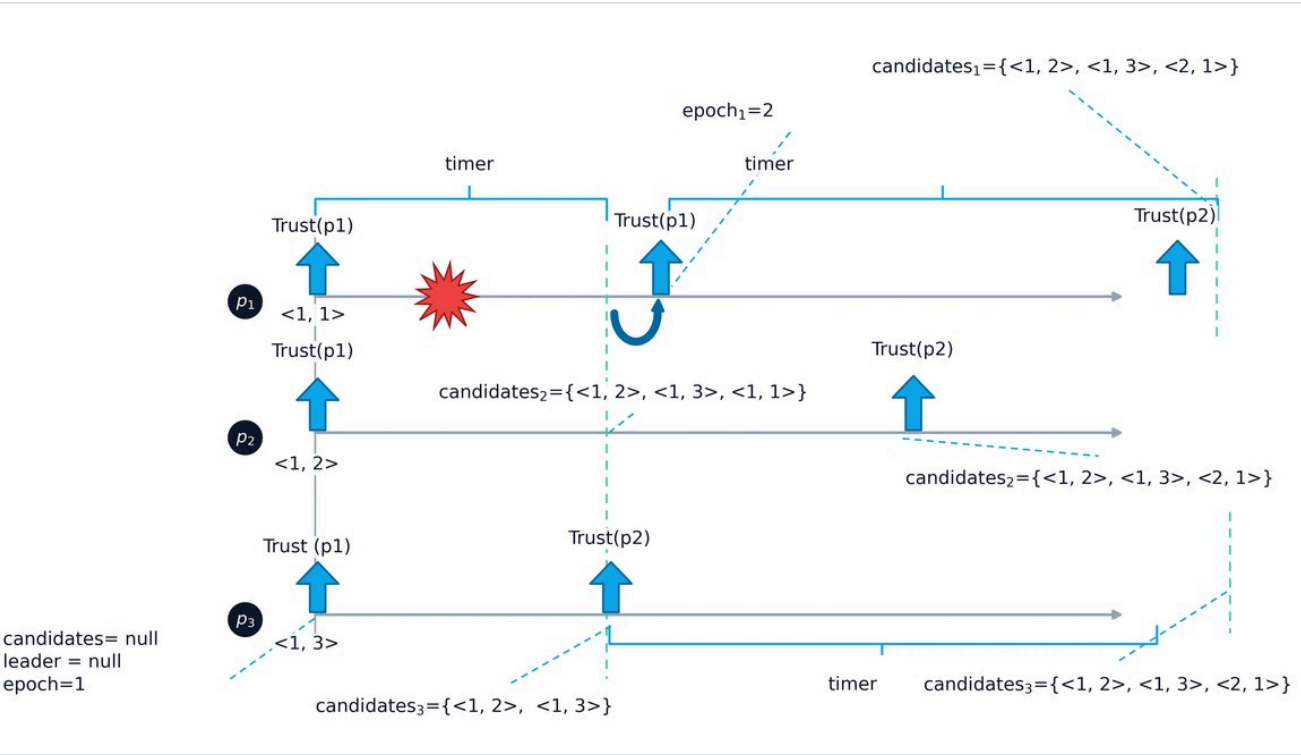
Algorithm 2.9: Elect Lower Epoch
Implements: EventualLeaderDetector, instance Ω
Uses: FairLossPointToPointLinks, instance fll

upon event $\langle \Omega, \text{Init} \rangle$ do
  epoch := 0;
  store(epoch);
  candidates := $\emptyset$;
  trigger $\langle \Omega, \text{Recovery} \rangle$;

upon event $\langle \Omega, \text{Recovery} \rangle$ do
  leader := maxrank(Π);
  trigger $\langle \Omega, \text{Trust} \mid \text{leader} \rangle$;
  delay := Δ;
  retrieve(epoch);
  epoch := epoch + 1;
  store(epoch);
  forall $p \in \Pi$ do
    trigger $\langle fll, \text{Send} \mid p, [\text{HEARTBEAT}, \text{epoch}] \rangle$;
  candidates := $\emptyset$;
  starttimer(delay);
upon event $\langle \text{Timeout} \rangle$ do
  newleader := select(candidates);
  if newleader $\neq$ leader **then**
    delay := delay + Δ;
    leader := newleader;
    trigger $\langle \Omega, \text{Trust} \mid \text{leader} \rangle$;
  forall $p \in \Pi$ do
    trigger $\langle fll, \text{Send} \mid p, [\text{HEARTBEAT}, \text{epoch}] \rangle$;
  candidates := $\emptyset$;
  starttimer(delay);

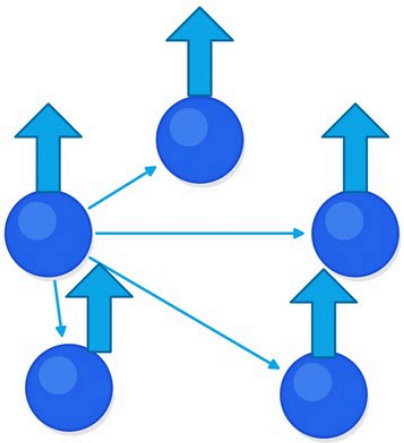
upon event $\langle fll, \text{Deliver} \mid q, [\text{HEARTBEAT}, ep] \rangle$ do
  if exists $(s, e) \in$ candidates such that $s = q \wedge e < ep$ **then**
    candidates := candidates $\setminus \{(q, e)\}$;
  candidates := candidates $\cup (q, ep)$;


timer
timer
Trust(p1)
candidates1={<1, 2>}
candidates1={<1, 2>, <1, 3>, <1, 1>}
p1
<1, 1>
<1, 1>
Trust(p1)
candidates2={<1, 2>, <1, 3>, <1, 1>}
p2
<1, 2>
Trust (p2)
Trust (p1)
Trust (p1)
timer
p3
<1, 3>
candidates= null
leader = null
epoch=1
candidates3={<1, 2>, <1, 3>}
Leader = p2
candidates3={<1,1>, <1, 2>, <1, 3>}
Leader = p2

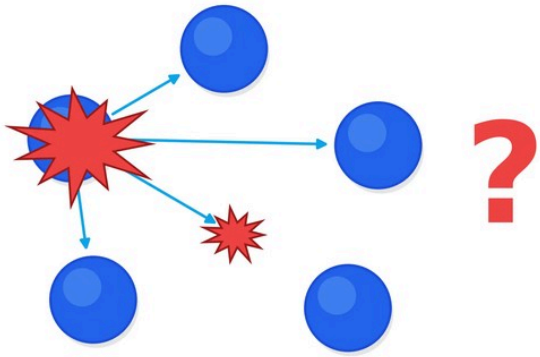

candidates1={<1, 2>, <1, 3>, <2, 1>}
epoch1=2
timer
timer
Trust(p1)
Trust(p1)
Trust(p2)
p1
<1, 1>
Trust(p1)
Trust(p2)
candidates2={<1, 2>, <1, 3>, <1, 1>}
p2
<1, 2>
candidates2={<1, 2>, <1, 3>, <2, 1>}
Trust (p1)
Trust(p2)
p3
<1, 3>
candidates= null
leader = null
epoch=1
timer
candidates3={<1, 2>, <1, 3>, <2, 1>}
candidates3={<1, 2>, <1, 3>}

Broadcast communications

Until now we studied how a process can communicate with another process, but now we want to study how a process can communicate with an entire group of processes. This type of communication is called *broadcast communications*. So, now, we want that a process can communicate with a group of processes ensuring certain properties even when some processes may fail.



No Failures



Crash Failures

Let's remember some useful things for this chapter:

- System model: the model that describes the system and his characteristics;
- A distributed system is composed by many processes that communicate over a network. Processes can perform operations independently and do not necessarily share the same memory.
- When we define a communication abstraction, we need to specify which properties it must respect:
  - *Safety*;
  - *Liveness*.
- Until now we studied point-to-point communication abstractions like:
  - *Fair-loss* links;
  - *Stubborn* links;
  - *Perfect* links (reliable delivery, no duplication, no creation).
- The following were also addressed:
  - *Physical clocks*: physical clocks used to measure time.
  - *Logical clocks*: logical clocks used to establish an ordering of events.
  - *Failure Detector* (FD): component that allows you to suspect that a process has failed.
  - *Leader Election*: procedure that allows processes to elect a leader.

For a communication broadcast we can immagine a system with three processes: $\Pi = \{p_1, p_2, p_3\}$, if p_1 want to send a message m to everyone, it must make sure that it is received by p_1, p_2, p_3. But we can ave no failures and

crash failures so we reach the goal → How can we ensure that processes have a consistent view of the messages received, even in the presence of failures?

Best effort broadcast (BEB)

Idea: When a process wants to send a message, the system tries to deliver it to all processes. The two main operations are:

- `BEB_Broadcast(msg)`: Used by a process to broadcast a message to everyone.
- `BEB_Deliver(msg)`: Used when a process receives and delivers the message to the application. BEB has three properties:

1. **BEB1**: *Validity*, if a correct process broadcast a message m, then every correct process eventually delivers m;
2. **BEB2**: *No duplication*, no message is delivered more than once;
3. **BEB3**: *No creation*, if a process delivers a message m with sender s, then m was previously broadcast by process s.

System model:

- Asynchronous system;
- Perfect links;
- Crash failures. The important thing is that BEB doesn't use an inter-process re-transmission mechanism to recover messages that the sender failed to send before a crash. It simply sends the message to everyone via Perfect Links.

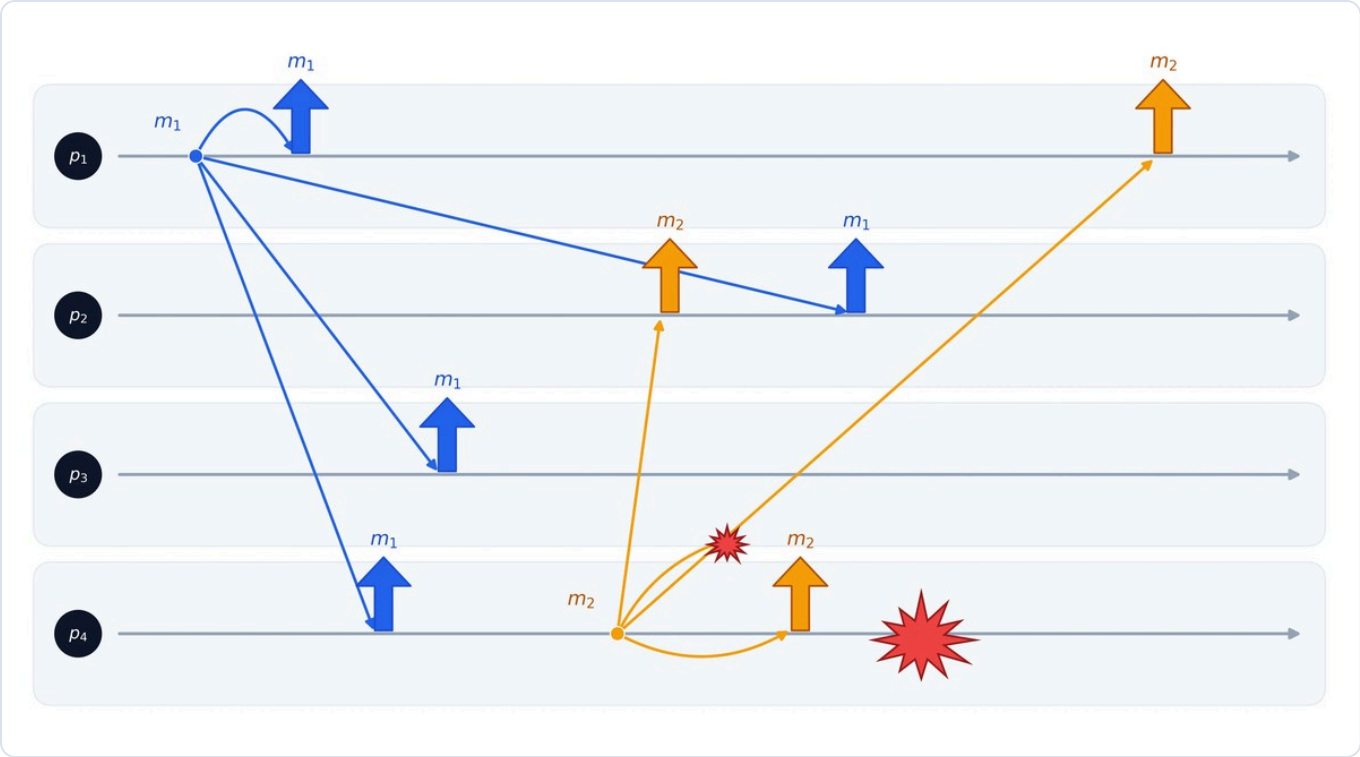
Algorithm 3.1: Basic Broadcast
Implements: BestEffortBroadcast, instance beb.
Uses: PerfectPointToPointLinks, instance pl.

upon event $\langle beb, \text{Broadcast} \mid m \rangle$ **do**
  forall $q \in \Pi$ **do**
    trigger $\langle pl, \text{Send} \mid q, m \rangle$;

upon event $\langle pl, \text{Deliver} \mid p, m \rangle$ **do**
  trigger $\langle beb, \text{Deliver} \mid p, m \rangle$;

Why BEB is correct?

The validity ensure that if a correct process broadcast a message m then all correct process eventually delivery m . Why? because the sender process send the message at all the processes using perfect links. If the sender is correct then can complete the delivery to everyone. Furthermore, Perfect Links guarantee delivery to the correct recipients. Therefore, validity derives directly from the reliable delivery of Perfect Links and from the fact that the sender sends the message to every process. The no-duplication ensure that a message can be delivered at most one time by the same process. We know that the message have unique identifier so the different messages must be distinguishable. The no-creation assure that it must not be possible to deliver a message that no process has ever sent.

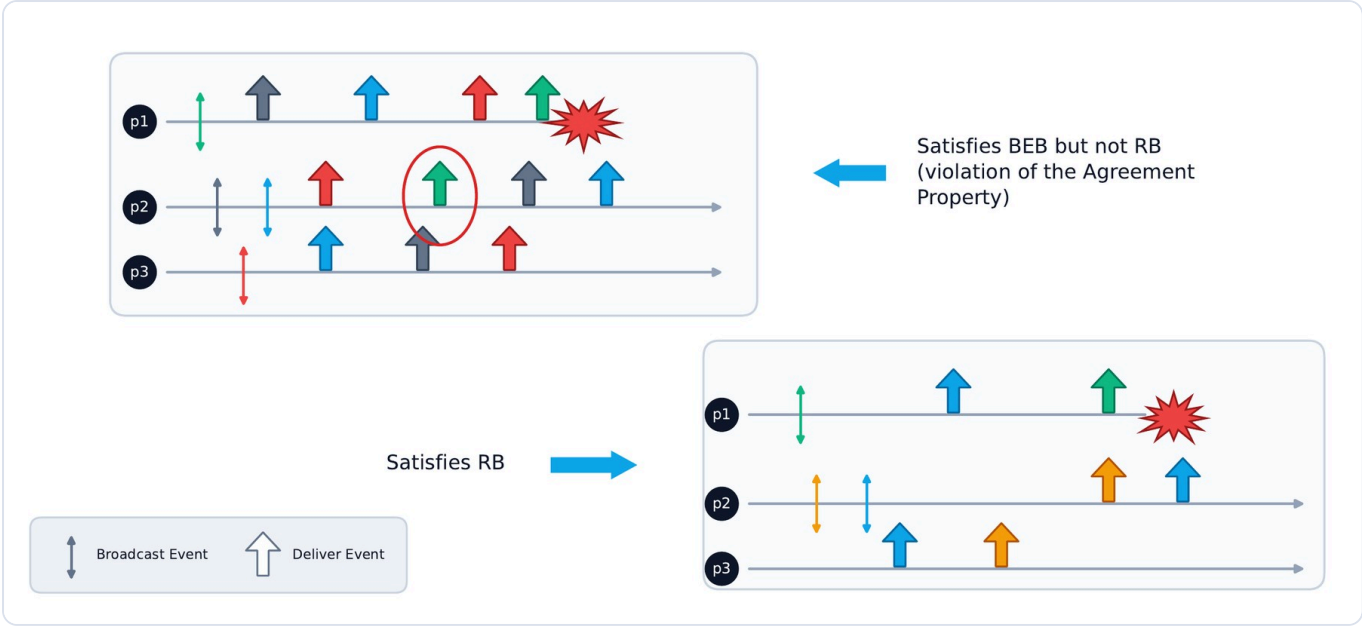


So we can notice:

1. BEB guarantees message delivery as long as the sender doesn't fail. If the sender is correct, it successfully sends the message to all processes, and thanks to Perfect Links, the correct recipients receive it.
2. If the sender fails, the processes may disagree about whether the message was delivered or not.

Reliable broadcast (RB)

To overcome the BEB limitation we introduce **Reliable Broadcast**. **RB** retains the same properties as BEB, but adds a key property: **Agreement**. **Agreement**: If a correct process delivers a message (m), then all correct processes eventually deliver (m). If even one correct process has delivered m, then all other correct processes must eventually deliver it. It doesn't matter if the original sender subsequently fails. This is a fundamental difference from BEB.



We have: $[p_1, p_2, p_3]$ Process p_1 sends m_1, but fails before completing the sending. With BEB:

- p_2 delivers m_1;
- p_3 does not deliver m_1. This behavior is permitted by BEB, but not by RB. Why? Because p_2 is a correct process and has delivered m_1. Consequently, according to Agreement, p_3, if correct, must also deliver it eventually. The **Agreement** property does not mean that all processes must deliver every message transmitted by a failing process. It means that, once a correct process has delivered a message, no other correct process can remain without delivering it.

> **WARNING**
>
> **BEB** ensures the reliability of communication when the sender is correct. **RB** also ensures agreement between the correct processes on message delivery.

Lazy reliable broadcast

To understand why it is necessary, we must first remember the problem that RB is designed to solve: if a sender fails after sending a message to only some processes, we must ensure that the other correct processes can still receive it. Consider a set of three processes:

$$\Pi = \{p_1, p_2, p_3\}$$

Suppose p_1 broadcasts a message m_1:

- p_1 succeeds in sending the message to p_2, but **crashes** before it can transmit it to p_3.
- With the **Best-Effort Broadcast (BEB)** abstraction:
  - p_2 receives and delivers m_1.
  - p_3 will never receive m_1.

AGREEMENT VIOLATION

This situation violates the **Agreement** property of *Reliable Broadcast*: if a correct process (p_2) delivers a message, then all correct processes (and thus also p_3) must eventually deliver it. To honor the **Agreement**, process p_2, after receiving the message, must cooperate by forwarding m_1 to other processes to fill the gap left by the failed sender.

ROLE OF THE PERFECT FAILURE DETECTOR ($\mathcal{P}$)

The key question is: *when should a process intervene to retransmit?* In a synchronous system, we rely on the oracle of the **Perfect Failure Detector** ($\mathcal{P}$), characterized by two formal properties:

1. **Strong Completeness**:

$$\forall p \in \Pi, \quad p \text{ crash} \implies \Diamond(\forall q \in \text{Correct}, \ q \text{ suspects } p)$$

Every process that fails is sooner or later permanently suspected (detected) by all correct processes.

2. **Strong Accuracy**:

$$\forall p \in \text{Correct}, \quad \forall q \in \Pi, \quad q \text{ never suspects } p$$

No correct process is ever wrongly suspected.

INTUITION

When $\mathcal{P}$ notifies the event $\langle \mathcal{P}, \text{Crash} \mid p \rangle$, there is mathematical certainty that p has indeed crashed, avoiding unnecessary or premature retransmissions. This deterministic guarantee is achievable in the **synchronous model**, where the upper bounds on transmission and computation times are known a priori.

Algorithm 3.2: Lazy Reliable Broadcast
Implements: ReliableBroadcast, instance rb.
Uses:
BestEffortBroadcast, instance beb;
PerfectFailureDetector, instance $\mathcal{P}$.

upon event $\langle rb, \text{Init} \rangle$ **do**
correct $:= \Pi$;
from$[p] := [\emptyset]^N$;

upon event $\langle rb, \text{Broadcast} \mid m \rangle$ **do**
trigger $\langle beb, \text{Broadcast} \mid [\text{Data}, self, m] \rangle$;

upon event $\langle beb, \text{Deliver} \mid p, [\text{Data}, s, m] \rangle$ **do**
if $m \notin \text{from}[s]$ **then**
trigger $\langle rb, \text{Deliver} \mid s, m \rangle$;
from$[s] := \text{from}[s] \cup \{m\}$;
if $s \notin$ correct **then**
trigger $\langle beb, \text{Broadcast} \mid [\text{Data}, s, m] \rangle$;

upon event $\langle \mathcal{P}, \text{Crash} \mid p \rangle$ **do**
correct $:=$ correct $\setminus \{p\}$;
forall $m \in \text{from}[p]$ **do**
trigger $\langle beb, \text{Broadcast} \mid [\text{Data}, p, m] \rangle$;

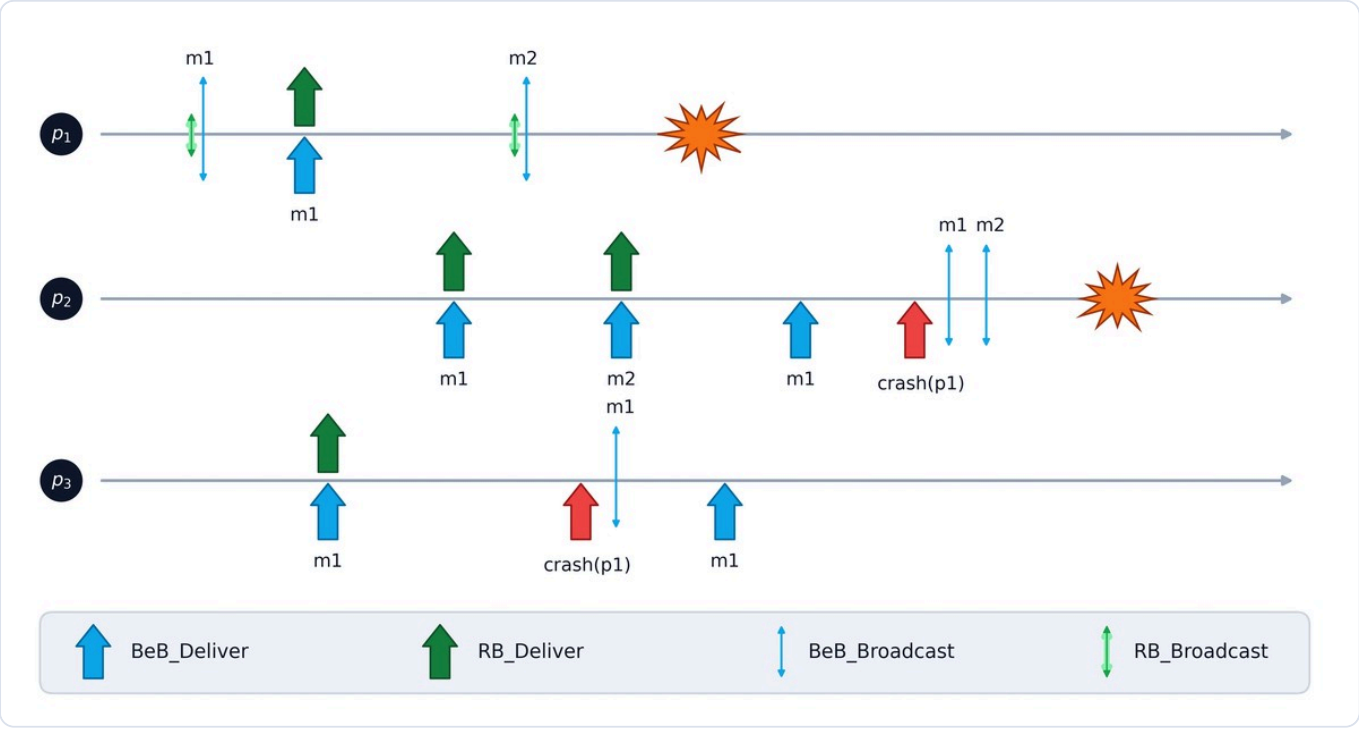
The **Lazy Reliable Broadcast** algorithm implements a reliable broadcast abstraction (rb) using a **Best-Effort Broadcast** (beb) protocol and a **Perfect Failure Detector** ($\mathcal{P}$). Its key characteristic is "laziness": instead of eagerly re-broadcasting every message to all processes immediately (which would consume high network bandwidth), processes only re-broadcast a message if its original sender is suspected or detected to have crashed. This ensures **Reliable Broadcast** properties (Validity, Agreement, Integrity) with lower message overhead under normal, crash-free conditions.

Components and State Variables:

- `correct`: A set initialized to Π (all processes in the system). It tracks which processes are currently believed to be correct (not crashed).
- `from[p]`: An array/dictionary mapping each process p to a set of messages received from p. This prevents duplicate deliveries and keeps track of what has been heard from whom.
- `beb` **(Best-Effort Broadcast)**: A lower-level communication primitive that guarantees message delivery if the sender does not crash, but does not guarantee atomic agreement across all correct processes on its own.
- $\mathcal{P}$ **(Perfect Failure Detector)**: A failure detector module that outputs permanent crash suspicions (never makes mistakes). **Detailed Event Breakdown**

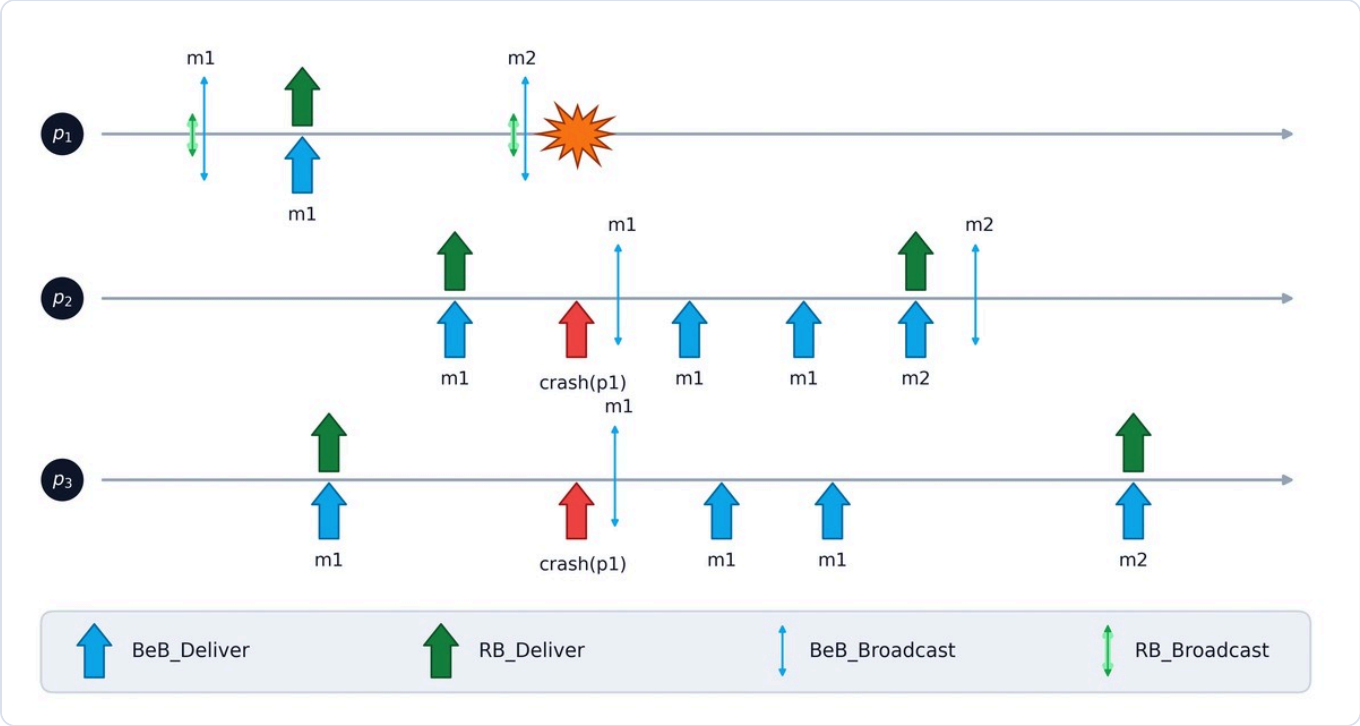
1. **Initialization**: When the initialization event $\langle rb, \text{Init} \rangle$ occurs, the system sets the set of correct processes to all nodes by letting $correct := \Pi$, and initializes the message-tracking structure with empty sets for each process by setting $\text{from}[p] := [\emptyset]^N$.

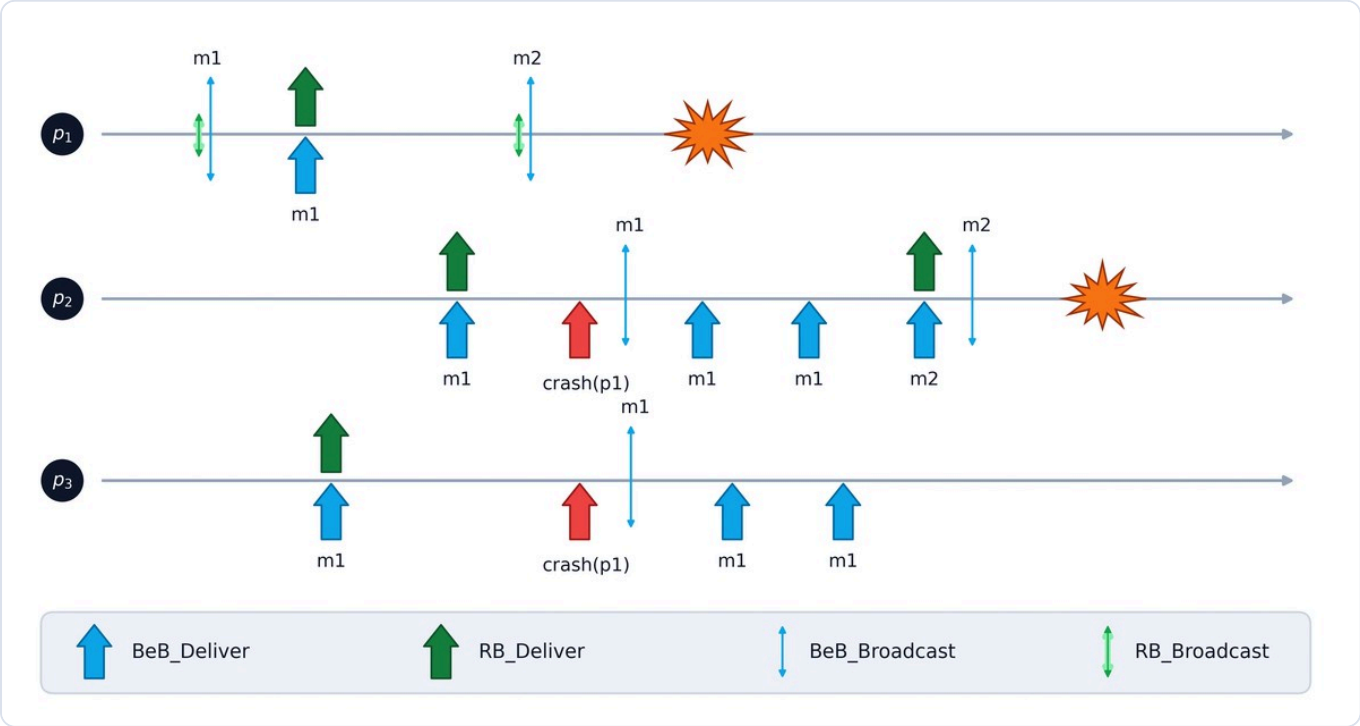
2. **Initiating a Broadcast**: When a process wants to broadcast a message m via $\langle rb, \text{Broadcast} \mid m \rangle$, it triggers a best-effort broadcast by encapsulating the message with its own identifier, activating the event $\langle beb, \text{Broadcast} \mid [\text{Data}, self, m] \rangle$. In this way, the message is sent only once without immediately forcing others to repeat it.
3. **Receiving a Message**: When a process receives a message via $\langle beb, \text{Deliver} \mid p, [\text{Data}, s, m] \rangle$, it performs a check to see if the message is new by checking the condition $m \notin from[s]$. If the condition is true:
    1. The process locally delivers the message by triggering $\langle rb, \text{Deliver} \mid s, m \rangle$.
    2. It updates the history by adding the message with $\text{from}[s] := \text{from}[s] \cup \{m\}$.
    3. It checks whether the original sender is considered incorrect by checking the condition $s \notin correct$. If the sender turns out to have crashed, the current process takes responsibility for forwarding the message by triggering $langlebeb, \text{Broadcast} \mid [\text{Data}, s, m] \rangle$.
4. **Handling Process Crashes**: When the failure detector signals the crash of a process through the event $\langle \mathcal{P}, \text{Crash} \mid p \rangle$:
    1. The process is removed from the set of healthy nodes by updating $\text{correct} := \text{correct} \setminus \{p\}$.
    2. The system loops through all messages previously received from that process by executing $\forall m \in \text{from}[p]$ and re-broadcasting them via $\langle beb, \text{Broadcast} \mid [\text{Data}, p, m] \rangle$. This guarantees that if p crashed mid-transmission, its pending messages are recovered and propagated to all remaining correct processes.



Consider the following sequence: p_1 performs an RB Broadcast of m_1, the message reaches p_2, subsequently p_1 performs an RB Broadcast of m_2, and before everyone has received the messages, p_1 crashes. Let us observe what happens. For m_1: p_2 has received m_1 and delivered it via RB; when p_1's crash is detected, p_2 knows that p_1 originated m_1, and consequently, it retransmits m_1 via BEB, allowing other processes that had received m_1 to also participate in its dissemination. For m_2: the message was originated by p_1, but not all processes had received it yet; however, if at least one correct process had already received it, this process can retransmit it after

the crash. The lesson to understand is that any process that has received the message becomes responsible for its further dissemination when the sender fails.





The idea behind the algorithm is:

- Normally, a process does not retransmit messages it receives.
- However, if it discovers that the original processing of a message has failed, it re-transmits the messages that process had already sent. Therefore, re-transmission occurs only when necessary. Hence the name Lazy.

PERFORMANCE OF LAZY RB ALGORITHM

- Best case: 1 BEB message per one RB message;
- Worst case: n-1 BEB message per one RB (this is the case with n-1 failures).

Reliable broadcast implementation in asynchronous system

This algorithm is called ""Eager reliable broadcast". It doesn't use a failure detector, this is important because we are in an asynchronous system. The logic of this algorithm is very easy: every process that receive a message for the first time re-send this message (immediately) to all the processes. So we don't have to wait that a process fail and we don't have to know if a process is failed or either. This is where the name Eager comes from, meaning "immediate". **Why does Eager RB guarantee Agreement?** Let's look at an example. We have three processes: $[p_1, p_2, p_3]$ Suppose p_1 sends m_1 to p_2, but fails before reaching p_3. With BEB, p_2 receives m_1. With Eager RB, however, p_2 doesn't just deliver it: it immediately retransmits it to everyone. So p_3 can receive m_1 even if p_1 has failed. **This is exactly the behavior we need to guarantee Agreement.**

Algorithm 3.3: Eager Reliable Broadcast
Implements: ReliableBroadcast, instance rb.
Uses: BestEffortBroadcast, instance beb.

upon event $\langle rb, \text{Init} \rangle$ **do**
  delivered $:= \emptyset$;

upon event $\langle rb, \text{Broadcast} \mid m \rangle$ **do**
  trigger $\langle beb, \text{Broadcast} \mid [\text{DATA}, \text{self}, m] \rangle$;

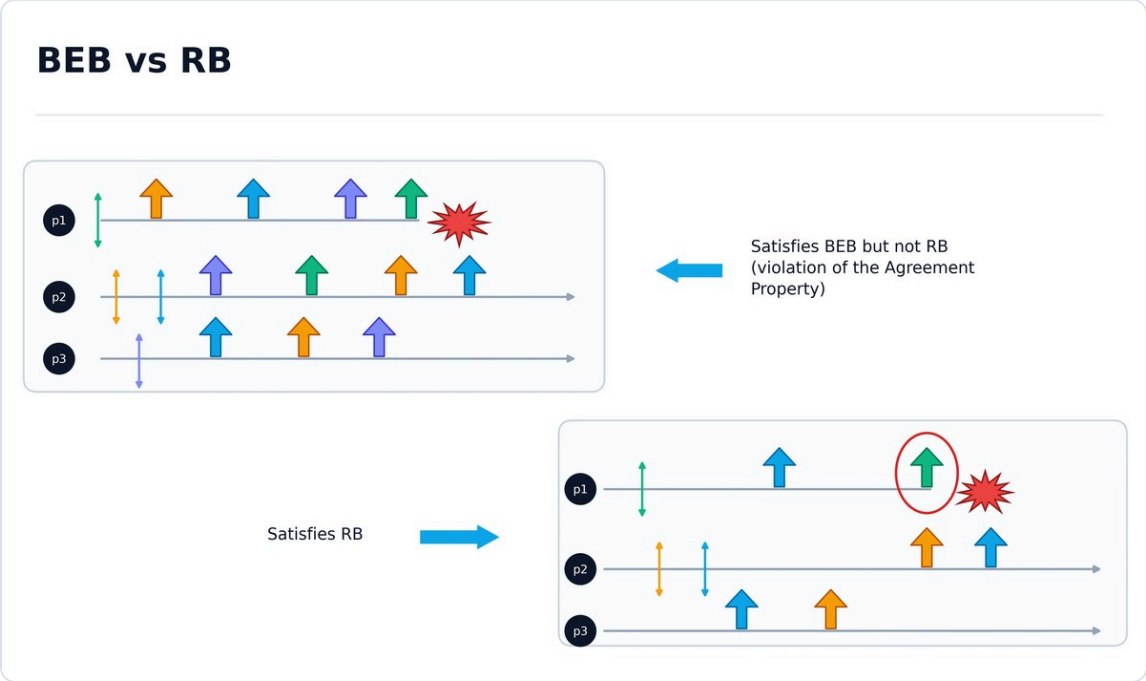
upon event $\langle beb, \text{Deliver} \mid p, [\text{DATA}, s, m] \rangle$ **do**
  if $m \notin$ delivered **then**
    delivered $:=$ delivered $\cup \{m\}$;
    trigger $\langle rb, \text{Deliver} \mid s, m \rangle$;
    trigger $\langle beb, \text{Broadcast} \mid [\text{DATA}, s, m] \rangle$;

PERFORMANCE OF EAGER RB ALGORITHM

Best case = worst case Why? because every process that receive the message for the first time re-transmits it. In a system with n processes, we have:

- An initial diffusion from the sender;
- A diffusion from every other process that receive the message for the first time. So the total number of broadcast operations is n. This feature incurs a higher communication cost than Lazy RB when there are no failures. However, we don't need a Failure Detector.

Charcteristic	Lazy RB	Eager RB
System	Sync	Async
Failure Detector	Perfect FD	Not necessary
Re-transmit	Only when necessary	At every first receive
Best case	1 BEB per RB	n BEB per RB
Worst case	$n-1$ BEB per RB	n BEB per RB
Communication cost	Depend on failure	Consistent with failures



Uniform reliable broadcast (URB)

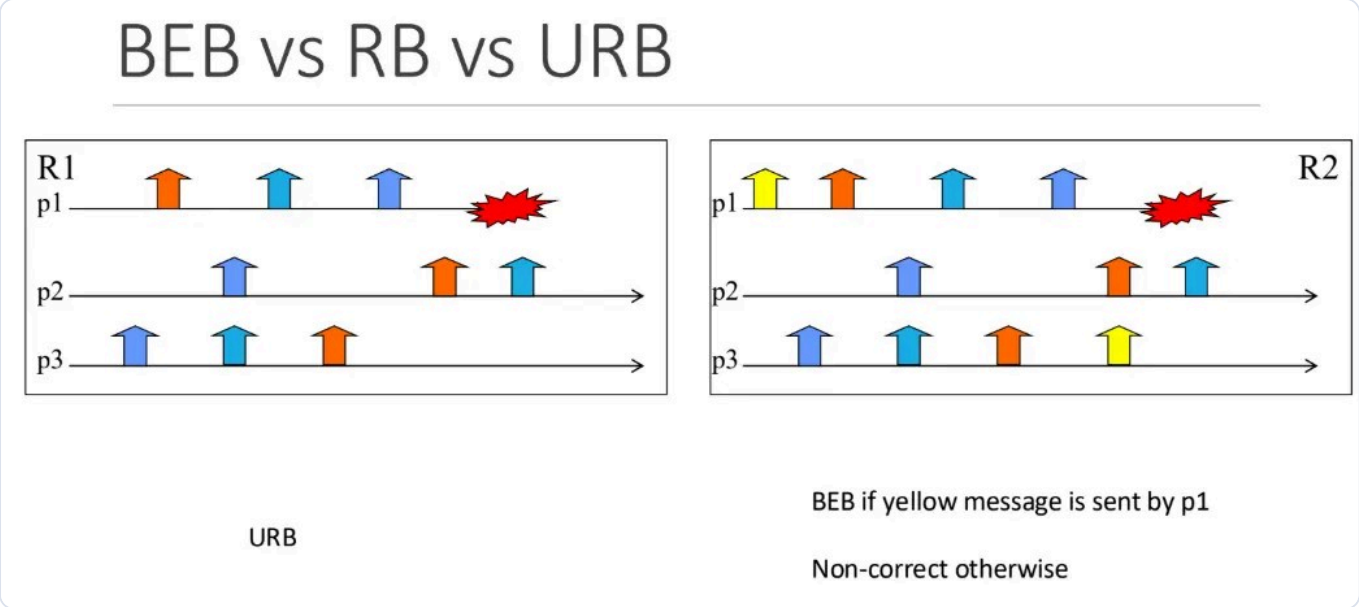
The agreement property only considers processes that don't fail. But what happens if a process delivers a message and then fails? Regular reliable broadcast does not require that all correct processes deliver that message. Example Consider three processes: $[p_1, p_2, p_3]$ Suppose:

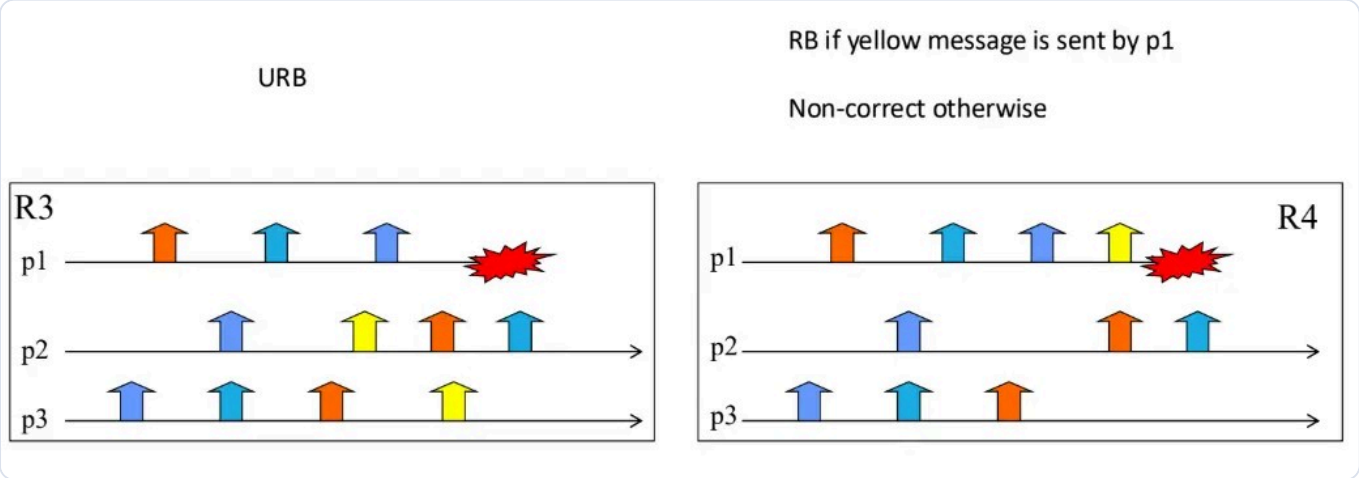
1. p_1 broadcasts m.
2. p_2 receives and delivers m.
3. p_2 fails.
4. p_1 and p_3 have not delivered m. The Agreement property of RB is not violated! Why? Because the process that delivered the message, p_2, failed. Agreement requires propagation only if the message was delivered by a correct process. Therefore, other correct processes may never receive that message. But from an application perspective, this can be a problem: one process has already performed an operation that the others are unaware of.

URB introduces a stronger property called **Uniform Agreement**: If any process, whether correct or failing, delivers a message m, then all correct processes must eventually deliver m. URB maintains the properties of BEB

and RB, but replaces Agreement with Uniform Agreement.

> **DIFFERENCE BETWEEN RB AND URB**
>
> RB considers delivery made by a correct process. URB considers delivery made by any process, correct or faulty. For this reason, Uniform Agreement is a stronger property than Agreement.





The goal is to verify which behaviors are allowed by the three abstractions. The yellow message represents the message whose delivery is being analyzed. The slide highlights two situations:

1. **First situation**:

The yellow message was sent by p_1. In this case, the situation can respect BEB but not necessarily RB. If p_1 fails after sending the message to only a few processes, BEB allows the processes to have received different information. RB, however, requires Agreement if a correct process delivered the message.

2. **Second situation**:

The slide shows that, if the process that delivered the message is destined to fail, the situation can respect RB but not URB. This is because RB does not enforce Agreement when the message was delivered only by one process

that subsequently fails. URB, however, requires all correct processes to deliver the message.

The criteria to be applied to the examples is always the same:

- *BEB*: checks Validity, No Duplication, and No Creation.
- *RB*: also checks Agreement.
- *URB*: Uniform Agreement verification, which also considers faulty processes.

IMPLEMENTATION IN A SYNCHRONOUS SYSTEM

Algorithm 3.4: All-Ack Uniform Reliable Broadcast.
Implements: UniformReliableBroadcast, instance urb.
Uses: BestEffortBroadcast, instance beb.
PerfectFailureDetector, instance $\mathcal{P}$.

upon event $\langle urb, \text{Init} \rangle$ **do**
  delivered $:= \emptyset$;
  pending $:= \emptyset$;
  correct $:= \Pi$;
  forall m **do** ack$[m] := \emptyset$;

upon event $\langle urb, \text{Broadcast} \mid m \rangle$ **do**
  pending $:=$ pending $\cup \{(\text{self}, m)\}$;
  trigger $\langle beb, \text{Broadcast} \mid [\text{DATA}, \text{self}, m] \rangle$;

upon event $\langle beb, \text{Deliver} \mid p, [\text{DATA}, s, m] \rangle$ **do**
  ack$[m] :=$ ack$[m] \cup \{p\}$;
  if $(s, m) \notin$ pending **then**
    pending $:=$ pending $\cup \{(s, m)\}$;
    trigger $\langle beb, \text{Broadcast} \mid [\text{DATA}, s, m] \rangle$;

upon event $\langle \mathcal{P}, \text{Crash} \mid p \rangle$ **do**
  correct $:=$ correct $\setminus \{p\}$;

function candeliver(m) **returns** Boolean **is**
  return (correct $\subseteq$ ack$[m]$);

upon exists $(s, m) \in$ pending **such that** candeliver$(m) \wedge m \notin$ delivered **do**
  delivered $:=$ delivered $\cup \{m\}$;
  trigger $\langle urb, \text{Deliver} \mid s, m \rangle$;

Overview & Dependencies

- **Implements:** `UniformReliableBroadcast`, instance `urb`. This guarantees that if *any* process delivers a message (even if the sender crashes), *all* correct processes eventually deliver it.
- **Uses:**
  - `BestEffortBroadcast`, instance `beb` (to disseminate messages across the network).
  - `PerfectFailureDetector`, instance $\mathcal{P}$ (to accurately track which processes are still alive and which have crashed).

State Variables (`Init`) When the algorithm starts, the local state is initialized as follows:

- $\text{delivered} := \emptyset$: Stores messages that have already been delivered to the application layer.
- $\text{pending} := \emptyset$: Tracks messages that have been broadcast or received but are waiting for the safety condition to be met before final delivery.
- $\text{correct} := \Pi$: The set of all processes in the system (Π) currently presumed to be correct (non-faulty).
- $\text{ack}[m] := \emptyset$: For each message m, a set recording which processes have acknowledged or relayed it.

Core Events & Logic

- **Initiating a Broadcast:**

$$\textbf{upon event } \langle urb, \text{Broadcast} \mid m \rangle \text{ do}$$

- Adds (self, m) to the local `pending` set.
- Triggers a Best-Effort Broadcast (`beb`) to send the data payload `[DATA, self, m]` to all processes.
- **Receiving via Best-Effort Broadcast:**

$$\textbf{upon event } \langle beb, \text{Deliver} \mid p, [\text{DATA}, s, m] \rangle \text{ do}$$

- Adds the sender p to the acknowledgment set for message m ($\text{ack}[m] := \text{ack}[m] \cup \{p\}$).
- If the message-sender pair (s, m) is not yet in `pending`, it adds it and immediately relays it using `beb` to ensure network-wide propagation.
- **Handling Process Crashes:**

$$\textbf{upon event } \langle \mathcal{P}, \text{Crash} \mid p \rangle \text{ do}$$

- When the failure detector $\mathcal{P}$ suspects/detects that process p has crashed, p is removed from the local `correct` set ($\text{correct} := \text{correct} \setminus \{p\}$). This ensures the algorithm stops waiting for acknowledgments from dead processes.
- **Check for Delivery Readiness:**

$$\textbf{function } \text{candeliver}(m) \text{ returns Boolean is}$$

- Returns `true` if and only if every process currently in the `correct` set has acknowledged message m ($\text{correct} \subseteq \text{ack}[m]$).

- **Delivering the Message:**

$$\textbf{upon exists } (s, m) \in \text{pending such that candeliver}(m) \land m \notin \text{delivered do}$$

- Once all correct nodes acknowledge a pending message and it hasn't been delivered yet:
  - Adds m to the `delivered` set (delivered $:=$ delivered $\cup \{m\}$).
  - Triggers the upper-layer delivery event $\langle urb, \text{Deliver} \mid s, m \rangle$.

IMPLEMENTATION IN ASYNCHRONOUS SYSTEM

In an asynchronous system, we can't know for sure whether a process has failed or is simply very slow. Therefore, we can't use a Perfect Failure Detector in the same way. So we put a rule:

> **WARNING**
>
> We need to assume a majority of correct processes.

Supponiamo che il sistema contenga n processi.

The majority of processes is correct when: $\boxed{\#Correct > \frac{n}{2}}$ For example, if we have five processes: $\Pi = \{p_1, p_2, p_3, p_4, p_5\}$ the majority requires at least three correct processes. Therefore:

- 5 correct processes: majority respected;
- 4 correct processes: majority respected;
- 3 correct processes: majority respected;
- 2 correct processes: majority not respected.

Algorithm 3.5 Majority-Ack Uniform Reliable Broadcast

Implements:
UniformReliableBroadcast (urb).

Extends:
All-Ack Uniform Reliable Broadcast (Algorithm 3.4).

Uses:
BestEffortBroadcast (beb).

function canDeliver(m) **returns** boolean **is**
return ($|\text{ack}_m| > N/2$);

// Except for the function above, and the non-use of the
// perfect failure detector, same as Algorithm 3.4.

A majority allows for a large enough set of correct processes to ensure that the confirmations collected don't come exclusively from processes destined to fail. The idea is to use a quorum, i.e., a minimum number of processes that must confirm receipt of the message.

Implementation	Hypothesis
Sync system	Perfect Failure Detector
Async system	Correct processes majority

QUESTION

Can we devise a uniform reliable broadcast algorithm for a partially synchronous system (using an eventually perfect failure detector) but without the assumption of a majority of correct processes?

Probabilistic broadcast